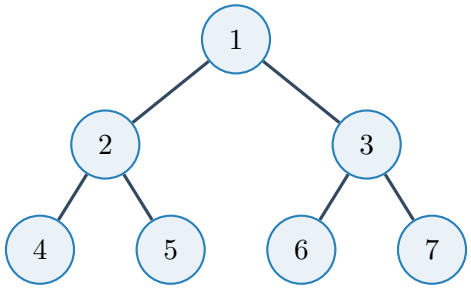
Trees

for Competitive Programming

From Roots and Leaves to Segment Trees and HLD



A rooted binary tree – the first picture of Chapter 1.

23 Chapters • **773 Problems** • **26 Fully Solved**

Written so far: 3 of 23 chapters • 83 problems written • 690 planned in the outline

Comprehensive DSA Guide

October 1, 2026

Contents

How to Read This Book v

Roadmap ix

I Foundations 1

1 Tree Basics and Terminology 2
1.1 What Is a Tree? . . . 3
1.2 Depth, Height and Size . . . 10
1.3 Kinds of Trees . . . 16
1.4 Counting Trees . . . 23
Chapter 1 Summary . . . 29
Common Mistakes Checklist . . . 30
Quick Reference Card . . . 30
Mixed Problem Set . . . 31
Related Topics . . . 31

2 Binary Trees 33
2.1 The Binary Tree Node and Structural Recursion . . . 34
2.2 Top-Down Recursion: Carrying State Down . . . 41
2.3 Bottom-Up Recursion: Returning Values Up . . . 47
Chapter 2 Summary . . . 52
Common Mistakes Checklist . . . 53
Quick Reference Card . . . 54
Mixed Problem Set . . . 54
Related Topics . . . 55

3 Tree Traversals 56
3.1 The Three Depth-First Orders . . . 58
3.2 Iterative Traversals and Morris Threading . . . 65
3.3 Breadth-First Traversal: Level Order . . . 73
3.4 Rebuilding and Serialising Trees . . . 78
Chapter 3 Summary . . . 85
Common Mistakes Checklist . . . 86
Quick Reference Card . . . 87
Mixed Problem Set . . . 88
Related Topics . . . 88

4 Representing Trees in Memory 89
4.1 Pointers vs Arrays . . . 89
4.2 Rooting an Unrooted Tree . . . 89
4.3 Euler Tour and Flattening . . . 89
4.4 Encodings and Formats . . . 90

II Ordered Trees and Heaps **91**

5 Binary Search Trees **92**
5.1 The BST Property . . . 92
5.2 Insertion and Deletion . . . 92
5.3 BST Queries . . . 92
5.4 BSTs in Contests . . . 93

6 Balanced Search Trees **94**
6.1 Why Balance? . . . 94
6.2 Rotations . . . 94
6.3 AVL Trees . . . 94
6.4 Treaps and Implicit Keys . . . 94
6.5 Cartesian Trees . . . 95

7 Heaps and Priority Queues **96**
7.1 The Binary Heap . . . 96
7.2 Building and Using a Heap . . . 96
7.3 Heaps as Trees . . . 96
7.4 Classic Heap Problems . . . 97

8 Tries **98**
8.1 The Trie of Strings . . . 98
8.2 Applications of the String Trie . . . 98
8.3 The Binary Trie . . . 98
8.4 Trie Practice . . . 99

III Rooted Tree Algorithms **100**

9 Height, Diameter and Centroid **101**
9.1 Depth and Height on a General Tree . . . 101
9.2 Tree Diameter . . . 101
9.3 Centroid and Centroid of a Tree . . . 101
9.4 Radius, Eccentricity and Centre . . . 102

10 Tree Dynamic Programming **103**
10.1 DP on Subtrees . . . 103
10.2 DP with Two States . . . 103
10.3 Rerooting DP . . . 103
10.4 Tree DP Practice . . . 104

11 Lowest Common Ancestor **105**
11.1 Ancestors and the LCA . . . 105
11.2 LCA by Euler Tour and RMQ . . . 105
11.3 LCA by Binary Lifting . . . 105
11.4 Distances and Paths via LCA . . . 106

12 Binary Lifting **107**
12.1 Jumping by Powers of Two . . . 107
12.2 Binary Lifting for LCA . . . 107
12.3 Lifting Arbitrary Path Data . . . 107
12.4 Binary Lifting Practice . . . 108

IV Range and Query Structures **109**

13 Segment Trees **110**
13.1 The Segment Tree . . . 110
13.2 Range Queries and Updates . . . 110
13.3 Advanced Segment Trees . . . 110
13.4 Segment Tree Practice . . . 111

14 Fenwick Trees **112**
14.1 The Binary Indexed Tree . . . 112
14.2 Fenwick Variants . . . 112
14.3 Fenwick on Trees . . . 112
14.4 Fenwick Practice . . . 113

15 Lazy Propagation **114**
15.1 Range Updates Done Right . . . 114
15.2 Composing Lazy Values . . . 114
15.3 Lazy on Trees . . . 114
15.4 Lazy Practice . . . 115

16 Heavy–Light Decomposition **116**
16.1 Splitting a Tree into Chains . . . 116
16.2 HLD for Path Queries . . . 116
16.3 HLD Applications . . . 116
16.4 HLD Practice . . . 117

V Expert Trees **118**

17 Union-Find and Minimum Spanning Trees **119**
17.1 Disjoint Set Union . . . 119
17.2 Minimum Spanning Tree . . . 119
17.3 DSU Trees and Offline Tricks . . . 119
17.4 Pruefer Sequences and Counting Trees . . . 120
17.5 Spanning Tree Practice . . . 120

18 Centroid Decomposition **121**
18.1 Building the Centroid Tree . . . 121
18.2 Counting Paths . . . 121
18.3 Centroid Decomposition Applications . . . 121
18.4 Centroid Practice . . . 122

19 Persistent Data Structures **123**
19.1 Persistence by Path Copying . . . 123
19.2 Persistent Segment Trees . . . 123
19.3 Persistent Trees on Trees . . . 123
19.4 Persistence Practice . . . 124

20 Advanced Tree DP **125**
20.1 DP on the DFS Order . . . 125
20.2 Rerooting at Scale . . . 125
20.3 DP with Data Structures . . . 125
20.4 Capstone Problems . . . 126

21 Mo's Algorithm and Virtual Trees **127**
21.1 Mo's Algorithm on Subtrees . . . 127
21.2 Mo's Algorithm on Paths . . . 127
21.3 Virtual (Auxiliary) Trees . . . 127
21.4 Applications and Practice . . . 128

22 Tree Isomorphism and Hashing **129**
22.1 Rooted Tree Isomorphism and AHU . . . 129
22.2 Canonical Forms and the Tree Centre . . . 129
22.3 Tree Hashing . . . 129
22.4 Counting Distinct Subtrees and Shapes . . . 130
22.5 Isomorphism Practice . . . 130

23 Link-Cut Trees **131**
23.1 Dynamic Trees and the Represented Tree . . . 131
23.2 Splay Trees: the Building Block . . . 131
23.3 Preferred Paths and Access . . . 131
23.4 Link, Cut and Queries . . . 132
23.5 Applications and Practice . . . 132

Appendices **133**

A Formula Reference **134**
A.1 Tree Basics . . . 134
A.2 Counting Trees . . . 134
A.3 Distances and Ancestors . . . 135

B Code Templates **136**
B.1 Node Types . . . 136
B.2 General Trees from an Edge List . . . 136
B.3 Height and Depth of a Binary Tree . . . 137
B.4 Level-Order Traversal (BFS) . . . 137
B.5 Three Binary-Tree Traversals . . . 138

C Problem-Solving Strategies **139**
C.1 Recognising a Tree Problem . . . 139
C.2 Constraints Suggest the Method . . . 139
C.3 A Five-Step Framework . . . 140

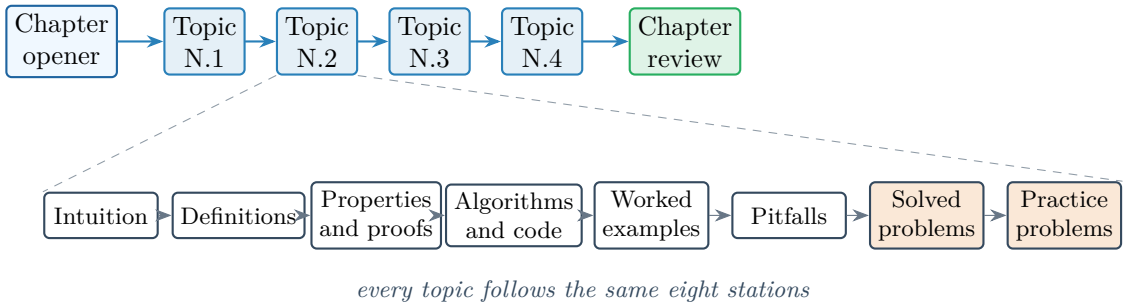
List of Solved Problems **141**

How to Read This Book

This book teaches the tree data structures and tree algorithms behind competitive programming and coding interviews, in the order you need them. Every idea is *explained with a picture*, *stated precisely*, *turned into code*, and then *used on real problems*. The same layout repeats in all 23 chapters, so once you know how one topic is organised you can navigate the whole book.

The shape of a chapter

A chapter covers one area (for example *Tree Traversals*) and is split into **topics**. A topic is a numbered section such as 1.3, and it always walks through the same stations, from intuition to practice.



every topic follows the same eight stations

A chapter is an opener, its topics and a review. Each topic runs from intuition to practice in a fixed order.

- The **opener** says what the chapter is for, what you should know first, and shows a map of how its topics depend on each other.
- Each **topic** starts with a short *In This Section* box, then builds the ideas with pictures, and ends with fully solved problems followed by a graded list of practice problems.
- The **chapter review** collects the key results, a quick-reference card (facts plus code), a set of *mixed* problems that combine the chapter's topics, and links to related chapters.

The boxes

Each coloured box has exactly one job, so you can skim a page by colour.

Definition (Tree)

A **tree** is a connected graph with no cycles. Equivalently, it is a set of nodes joined by edges where any two nodes are linked by exactly one path.

Blue boxes define a term precisely. The term is printed in bold blue where it is introduced.

Theorem (Edges of a tree)

A tree with n nodes has exactly $n - 1$ edges.

Purple boxes state results: theorems, lemmas (stepping stones) and corollaries (immediate consequences).

Proof

Every result is proved or argued, usually right below it. Proofs end with a black square. When a proof uses an earlier result it cites it by number, and the number is a link.

■

Key Insight

Yellow boxes give the idea in one or two sentences: the picture to keep in your head.

Step-by-Step Trace

Green boxes walk through an algorithm by hand on a small tree, step by step, so you can check your understanding (and your code) against it.

Common Mistake

Red boxes flag the mistakes that cost accepted verdicts: forgetting the parent pointer and looping forever, off-by-one on depth vs. height, wrong `null` base case in recursion.

Complexity

Grey boxes state running time and memory, and why.

Note / Contest tip

Light-grey boxes hold side remarks: history, alternative methods, judge-specific advice.

a full binary tree has $L = I + 1$ leaves for I internal nodes — a fact worth memorising

Extra tools for the tricky parts

Some ideas – rotations, binary lifting, heavy–light decomposition, lazy propagation – are hard to grasp from prose alone. Four optional aids exist for exactly those moments. They appear where an author needs them, not in every topic.

File system

A rooted tree is a set of folders: the root is `/`, each folder is a node, its sub-folders are children, and a folder with none is a leaf. "The subtree at v" is everything inside one folder.

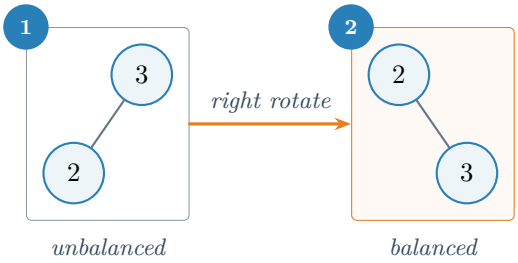
A blue analogy box ties an abstract definition to something you already know.

Where this is used. Chapter 13 (segment trees) and Chapter 14 (Fenwick trees) – the Euler tour that flattens a subtree into a contiguous range is what turns "sum over the subtree of v" into a plain range query.

Depth vs Height

Depth depth(v)
edges measured down from the *root* to v

Height height(v)
edges measured down from v to its deepest *leaf*



A **state sequence** shows an operation happening: numbered frames, a thick orange arrow, and the key result boxed in orange. Harder practice items may also carry a **hint ladder** – a gentle nudge then a stronger one – so you are never stuck on a single clue.

Solved problems and practice problems

A **solved problem** opens with a header like the one below. The solution underneath always has the same steps: *Understanding the problem*, *Key observations*, *Solution*, *Proof of correctness*, *Implementation*, *Complexity*, and *Pitfalls and variants*. Try the problem yourself before you read past the observations.

W1.1.1 Maximum Depth of a Binary Tree

Source: LeetCode 104 — LC Easy
Techniques: recursion, height of a tree

Practice problems come right after each topic, ordered from easy to hard, each with a one-line hint that nudges without spoiling:

Q2.1.6 Balanced Binary Tree – LeetCode 110 — LC Easy

Hint: Compute height bottom-up and return a sentinel (like -1) as soon as a subtree is unbalanced.

Numbering and cross-references

Item	Looks like	Rule
Topic (section)	1.3	chapter.topic
Definition, theorem, ...	Theorem 1.3.4	one shared counter per topic
Figure / table	Figure 1.3.2	numbered per topic
Code listing	Code 1.3.1	numbered per topic
Solved problem	W1.3.2	*W*orked, numbered per topic
Practice problem	Q1.3.5	*Q*uestion, numbered per topic
Mixed problem (chapter review)	M1.4	numbered per chapter

Every number is a clickable link in the PDF. The *List of Solved Problems* at the end of the book gives every W-problem with its source.

Difficulty tags

Problems carry a tag. LeetCode uses its own labels; Codeforces ratings use the Codeforces colour bands; other judges are shown in grey.

LC Easy LC Medium LC Hard CF 900 CF 1300 CF 1500 CF 1700 CF 2000 CF 2200 CF 2600 CSES AtCoder 1580

Notation

n	number of nodes	depth(v)	edges from the root to v
height(v)	longest path down to a leaf	height(T)	height(root)
size(v)	nodes in the subtree of v	level(v)	depth$(v)+1$ (1-based)
deg(v)	number of children / edges	parent(v)	the node above v
children(v)	the nodes just below v	lca(u, v)	lowest common ancestor
dist(u, v)	edges on the u–v path	diam(T)	longest path in the tree

We count depth and height in *edges* unless a problem says otherwise; when we count in nodes we say so. Inside $O(\cdot)$ the base of a logarithm does not matter, so we write $O(\log n)$.

Code conventions

- All code is C++ and is accepted by GCC with `-std=c++17` on Codeforces, AtCoder, CSES and the LeetCode compiler. Listings are complete: they compile as printed.
- Binary trees use the classic node struct with `left` and `right` pointers; `null` marks a missing child.
- General trees are stored as an adjacency list `vector<vector<int>` and walked with DFS, always passing the parent so we never revisit it.
- Recursion depth can reach n; for a chain of 10^5 nodes prefer an iterative traversal or raise the stack limit if the judge allows it.

How to practise with this book

1. Read a topic up to its solved problems. Redo each *trace* box by hand.
2. For each solved problem, read the statement, stop, and try it for twenty minutes. Then read one observation at a time.
3. Solve the practice list in order. Use the hint only after a real attempt.
4. At the end of the chapter, do the mixed set: those problems do not tell you which topic they belong to.

How the notes are organised on disk

The book is split so that each part can be written and compiled on its own: `setup/` holds the whole preamble (packages, boxes, notation); `chapters/chNN_name/` holds one chapter, with one file per topic; `structure/` holds the writing templates used for new chapters; and `tools/preview.py` compiles a single topic in seconds while `tools/build.py` builds the complete book. Problem counts in this book (title page, part pages, chapter headers) are computed automatically.

Roadmap

The book at a glance

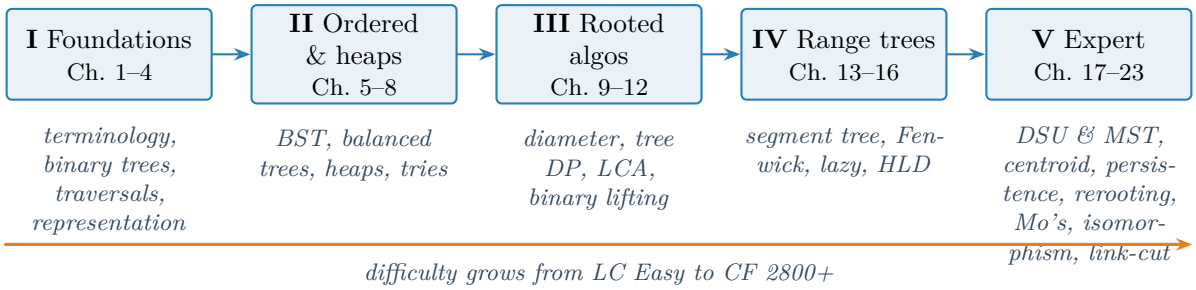
The book has five parts, ordered by difficulty. The chapter and problem counts below are counted automatically from the chapters themselves; an outline chapter contributes its planned problems until it is written.

Part	Title	Level	Chapters	Problems
I	Foundations	Beginner – LC Easy/Medium	1–4	112
II	Ordered Trees and Heaps	LC Medium – CF 1600	5–8	134
III	Rooted Tree Algorithms	CF 1400–2000	9–12	133
IV	Range and Query Structures	CF 1600–2300	13–16	133
V	Expert Trees	CF 1800–2800	17–23	261
	Total		**23**	**773**

Progress. 3 of 23 chapters are written, with 83 problems on paper (26 of them fully solved) and 690 more planned in the outline.

Part I builds the vocabulary every later chapter uses: what a tree is, how binary trees are shaped, how to walk them, and how to store them. Read it first even if you already "know" trees – the notation is fixed here.

How the parts depend on each other



Each part assumes the previous ones. Under each part: the areas it covers.

Study plan

Beginner track – Chapters 1–4 (2–3 weeks)

Focus. Tree vocabulary; binary tree shapes and node counts; DFS and BFS traversals (recursive and iterative); building trees from arrays and traversals.
Goals. Traverse any tree four ways without thinking; compute depth, height and size by recursion; reach LeetCode Easy–Medium on tree problems.

Intermediate track – Chapters 5–8 (3–4 weeks)

Focus. BST search, insert, delete; balanced trees (AVL, red–black ideas); heaps and priority queues; tries for strings.
Goals. Implement a BST and a heap from scratch; know why balance keeps operations $O(\log n)$; reach LC Medium.

Advanced track – Chapters 9–12 (3–4 weeks)

Focus. Diameter and centroids; tree dynamic programming; lowest common ancestor; binary lifting and Euler tours.
Goals. Solve rooted-tree problems with a single DFS; answer LCA in $O(\log n)$; reach CF 1400–2000.

Range-structure track – Chapters 13–16 (3–4 weeks)

Focus. Segment trees; Fenwick trees; lazy propagation; heavy–light decomposition.
Goals. Build, query and update a segment tree fluently; handle path queries on a tree with HLD; reach CF 1600–2300.

Expert track – Chapters 17–23 (4–5 weeks)

Focus. Union–find and minimum spanning trees; centroid decomposition; persistent structures; advanced tree DP and rerooting; Mo's algorithm and virtual trees; tree isomorphism and hashing; link–cut trees.
Goals. Reach CF 1800–2800+. These chapters assume everything before them.

Part I

Foundations

The Language of Trees

Beginner – LC Easy/Medium • 4 chapters • 112 problems

Chapters in this part

1 Tree Basics and Terminology 2

2 Binary Trees 33

3 Tree Traversals 56

4 Representing Trees in Memory 89

1

Chapter

Tree Basics and Terminology

Level: Beginner – LC Easy/Medium • **Problems:** 28 (6 fully solved) • **Status:** Written

Chapter at a Glance

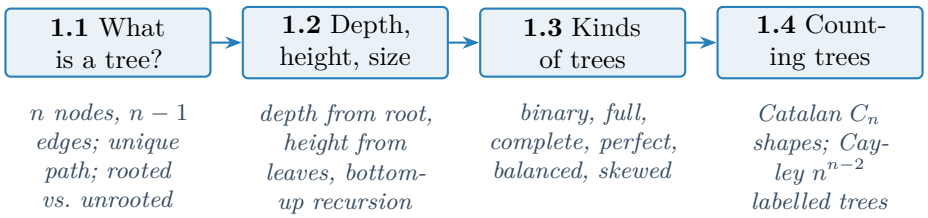
A **tree** is the simplest structure that captures a hierarchy: one thing at the top, everything else hanging below it, and exactly one way to travel from any object to any other. This chapter builds the whole vocabulary we use for the rest of the book – *root, parent, child, leaf, subtree, ancestor, descendant, depth, height, size* – and proves the few facts that every later algorithm quietly relies on: a tree with n nodes has $n-1$ edges, there is exactly one path between any two nodes, and height can be computed bottom-up with a single recursion.

You will learn to

- say precisely which pictures are trees and which are not, and why;
- prove and use the $n-1$ edges and unique-path properties;
- compute depth, height and size of every node in one DFS;
- name the standard shapes (binary, full, complete, perfect, balanced, skewed) and know the counting rule each one gives;
- count tree shapes with the Catalan numbers and labelled trees with Cayley's n^{n-2} formula.

Prerequisites. None – this is the entry point of the book. We only assume you can read a small *C++* function and know what a graph (nodes and edges) is.

In contests and interviews. Every tree problem on LeetCode and most on Codeforces start here: you must know what a root, a leaf and a subtree are, and that a DFS carrying the parent is the workhorse. The counting facts of §1.4 turn up directly in combinatorics and DP problems (Chapter 10).



The four topics of this chapter, left to right, each with the results you take away from it.

Topic	Key results	Solved problems
§1.1 What is a tree?	$n-1$ edges; exactly one path between any two nodes	W1.1.1
§1.2 Depth, height and size	height/size recurrences; one-DFS measurement	W1.2.1, W1.2.2
§1.3 Kinds of trees	perfect $2^{h+1}-1$ nodes; full tree leaves = internal +1	W1.3.1, W1.3.2
§1.4 Counting trees	Catalan C_n shapes; Cayley n^{n-2} labelled trees	W1.4.1

Notation Used in This Chapter

Symbol	Meaning	Symbol	Meaning
n	number of nodes	depth(v)	edges from root down to v
T	a tree	height(v)	edges from v to deepest leaf below
parent(v)	the node above v	size(v)	nodes in the subtree rooted at v
children(v)	nodes directly below v	level(v)	same as depth(v) (root = level 0)
deg(v)	number of neighbours of v	leaves(T)	the set of leaves of T
subtree(v)	the tree under v	`null`	the empty / `null` child
h	height of the whole tree	C_n	n-th Catalan number

Throughout the book height and depth are counted in **edges**, so a single lone node has height 0 and height(`null`) $= -1$. The convention note in Appendix A explains why this matters before you submit.

1.1 What Is a Tree?

In This Section

A tree is a set of nodes joined by edges with no loops and no separate islands – you can get from any node to any other, and there is only one way to do it. File systems, organisation charts, HTML documents and parse trees are all trees, and every data structure in this book is built on this one picture.

Prerequisites. The idea of a graph (nodes and edges) from a first algorithms course.

After this section you can

- tell a tree from a graph that is not a tree, at a glance;
- root a tree and name parents, children, leaves and subtrees correctly;
- prove that a tree on n nodes has $n-1$ edges and one path per pair;
- read a tree from an edge list into an adjacency list in C++.

1.1.1 Intuition

Folders and family

A tree is a computer's folder structure: one root folder, sub-folders inside folders, files at the bottom, and no folder ever nesting back inside its own descendant. A family tree drawn down from a single ancestor is the same picture. That "nothing is its own ancestor" rule is exactly the no-cycles condition we make precise below.

Look at the picture below. The dots are nodes and the lines are edges. There are no closed loops: starting anywhere, you can never come back to where you began without retracing a step. That loop-free, fully-connected shape *is* a tree.

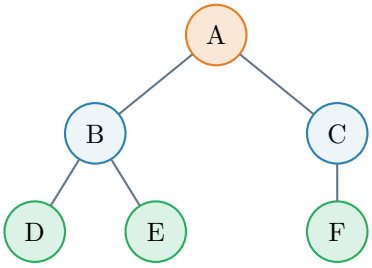


Figure 1.1.1: A tree. The orange node is chosen as the *root*; the green nodes D, E, F are *leaves*. Notice there is exactly one path between any two letters.

Now the three pictures that are **not** trees. A loop (a cycle) is forbidden, and so is a floating piece you cannot reach.

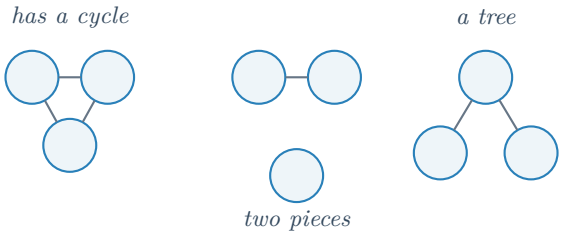


Figure 1.1.2: Left: a cycle, so not a tree. Middle: disconnected (node isolated), so not a tree. Right: connected and loop-free, so a tree.

The same unrooted tree becomes a *rooted* tree the moment we pick a starting node and let gravity decide who is above whom.

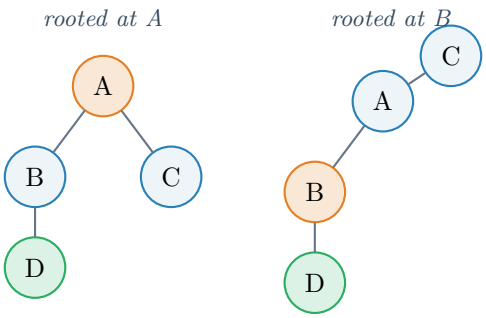


Figure 1.1.3: One tree, two roots. Rooting at B makes A a child of B and D a child of B; rooting at A makes B a child of A. The edges do not change, only our view of parent and child does.

1.1.2 Definitions and Notation

Definition 1.1.1 (Tree and forest)

A **tree** is a connected graph with no cycles. A **forest** is a disjoint union of trees – it drops only the "connected" requirement. Equivalently, a graph on n nodes is a tree if any (and hence all) of these hold: it is connected with $n-1$ edges; it has no cycle but adding any missing edge creates one; every pair of nodes is joined by exactly one path.

Definition 1.1.2 (Rooted tree: parent, child, leaf)

A **rooted tree** is a tree with one distinguished node called the **root**. Directing every edge away from the root gives the relations: if u sits above v and they are adjacent, u is the **parent** of v and v is a **child** of u; a node with no child is a **leaf**; a node that is not a leaf is **internal**. The **subtree** at v, written subtree(v), is v together with all its descendants. An **ancestor** of v is any node on the path from the root to v (including v itself); a **descendant** runs the other way. Siblings share a parent.

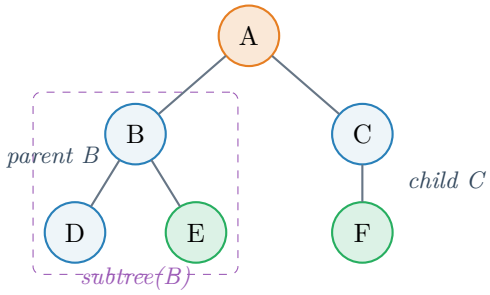


Figure 1.1.4: Anatomy of the rooted tree. B is the parent of D and E (its children); C is a child of A; D, E, F are leaves; the dashed box is the subtree rooted at B. Node A is an ancestor of every other node.

Definition 1.1.3 (Degree, path, distance)

The **degree** $\deg(v)$ of a node is its number of neighbours (in a rooted tree, often split into in-degree = children and out-degree = 1 for the parent). A **path** is a sequence of nodes each adjacent to the next; its **length** is the number of edges. The **distance** $\mathrm{dist}(u, v)$ is the length of the (unique) shortest path between u and v.

1.1.3 Counting Edges, Finding Paths

Two facts about trees come up in almost every proof later in the book. They look obvious, so it is worth seeing exactly why they hold.

Theorem 1.1.4 (Edges of a Tree)

A tree with n nodes has exactly $n-1$ edges.

Proof

Induction on n. For $n = 1$ there are no edges, and $1 - 1 = 0$. For $n \geq 2$, a finite tree always has a leaf: walk a longest simple path, and one of its endpoints must have degree 1 – if it had another neighbour we could either extend the path or close a cycle, both impossible. Call that leaf v; deleting v and its single incident edge leaves a tree on $n-1$ nodes, which

by the induction hypothesis has $n-2$ edges. Putting v back adds exactly one edge, giving $(n-2)+1=n-1$.

■

Key Insight

Read it backwards: $n-1$ edges is the *minimum* needed to connect n nodes. One edge short and you have two islands; one edge too many and you create a cycle. That is why contest statements say "n cities, $n-1$ roads, all connected" and quietly mean a tree.

Theorem 1.1.5 (Unique Path)

Between any two nodes of a tree there is exactly one simple path.

Proof

Existence: the tree is connected, so some path exists between any two nodes. *Uniqueness:* suppose two different simple paths P_1 and P_2 joined u to v. Walk along P_1 from u until the first node w where P_1 and P_2 split, then continue to where they rejoin at some node z. The two different w-to-z pieces together form a closed loop, i.e. a cycle – contradicting that the graph is a tree. Hence the path is unique.

■

Key Insight

Unique paths are why tree problems rarely need "shortest" in the statement: the one path *is* the shortest. They are also why we can talk about "the" path between u and v without qualification, and why removing a single edge splits a tree into exactly two pieces.

1.1.4 Storing a Tree in Memory

Interview binary trees arrive as `TreeNode*` pointers (see Chapter 2). Contest trees arrive as a list of $n-1$ edges over nodes numbered $0..n-1$ or $1..n$; we store them as an adjacency list and remember the parent so the walk never doubles back.

Code 1.1.1: Build an adjacency list from edges and measure sizes with a parent-guard DFS

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<vector<int>> adj;      // adj[v] = neighbours of v
vector<int> sz;               // sz[v] = size of subtree at v

// Root the tree at 0. p is the node we came from; never walk back to it.
void dfs(int v, int p) {
    sz[v] = 1;                        // count v itself
    for (int to : adj[v]) {
        if (to == p) continue;        // the parent guard: without this -> infinite
    ↪ loop
        dfs(to, v);
        sz[v] += sz[to];              // fold each child's subtree size upward
    }
}

```

```
int main() {
    n = 5;
    adj.assign(n, {});
    vector<pair<int,int>> edges = {{0,1}, {0,2}, {1,3}, {1,4}};
    for (auto [u, v] : edges) { adj[u].push_back(v); adj[v].push_back(u); }
    sz.assign(n, 0);
    dfs(0, -1);
    // edges = n - 1 = 4, and sz[0] == n, confirming the two theorems above.
}
```

Complexity

Building the adjacency list is $O(n)$ (each of the $n-1$ edges pushed twice). The DFS visits every node once and every edge twice, so it is $O(n)$ in time and $O(n)$ memory for the list plus the recursion stack. On a path-shaped tree the stack can be n deep; see §1.2 for the iterative alternative.

1.1.5 Worked Examples

Example 1.1.6 (A six-node tree)

Take the tree of Figure 1.1.4 and add a child G under C. It now has $n = 6$ nodes. Count the edges: AB, AC, BD, BE, CF, CG – that is 6 nodes but only $5 = 6 - 1$ edges, matching Theorem 1.1.4. The path from D to G is $D \to B \to A \to C \to G$, length 4, and it is the only such path (Theorem 1.1.5).

Step-by-Step Trace

Running of Code 1.1.1 on a path $0 - 1 - 2 - 3$ rooted at 0 gives $\text{size}(3) = 1$, $\text{size}(2) = 2$, $\text{size}(1) = 3$, $\text{size}(0) = 4$: each node's size is 1 plus its single child's size, so sizes count up from the far leaf.

1.1.6 Common Pitfalls

Forgetting the parent guard

A tree stored as an adjacency list has *undirected* edges, so B appears in `adj[A]` and A appears in `adj[B]`. A DFS that does not skip the node it came from bounces $A \to B \to A \to B \ldots$ forever. The line `if (to == p) continue;` is not an optimisation; it is what makes the traversal terminate.

Connected but cyclic, or acyclic but split

Both failures look tree-like. "No cycle" alone allows a forest; "connected" alone allows a graph with a loop. Only together do they define a tree – and only a tree has the $n-1$ edge count. If your edge count is not $n-1$, you do not have a tree.

1.1.7 Solved Problems

W1.1.1 Same Tree

Source: LeetCode 100 LC Easy
Techniques: tree definition, structural equality, recursion

Same Tree

Given the roots `p` and `q` of two binary trees, decide whether they are *the same tree*: same shape *and* same value at every node. Two empty trees are the same; an empty tree and a non-empty one are not.

➤ Understanding the Problem

"Same" here is the precise structural equality the definition gives: the roots match in value, the left subtrees are the same, and the right subtrees are the same. That is already a recursive description, which is why a two-line recursion solves it and why a flat comparison of node values is not enough – shape matters.

➤ Key Observations

Observation 1. Two trees are the same iff their roots carry the same value *and* their left subtrees are the same *and* their right subtrees are the same.

Observation 2. The base case is the `null` pointer: if either node is `null`, the trees match exactly when *both* are `null` at once.

➤ Solution

Recurse in lockstep on the two trees. At each pair of nodes, if both are null return true; if exactly one is null return false; otherwise compare values and recurse on the two children pairs.

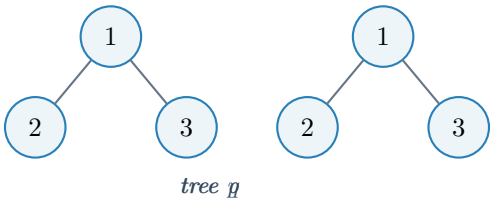


Figure 1.1.5: These two trees are the same: equal values and equal shape, so the answer is `true`. Changing q's right child 3 to a left child would make the shapes differ and the answer `false`.

➤ Proof of Correctness

Induction on tree size

By Theorem 1.1.5 a binary tree's structure is fully described by its root and its two subtrees. Induct on $\text{size}(p) + \text{size}(q)$. Base: both empty – the code returns `p == q`, correct. Step: if exactly one is empty they differ and the code returns false, correct. Otherwise both are non-empty: by the induction hypothesis the recursive calls correctly decide whether the left subtrees match and whether the right subtrees match; the code returns true exactly when the root values equal *and* both recursive answers are true – precisely the definition of "same tree".

■

➤ Implementation

Code 1.1.2: Same Tree (LeetCode 100)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if (p == nullptr || q == nullptr)   // both null -> equal; one null -> not
            return p == q;
        return p->val == q->val
            && isSameTree(p->left,  q->left)
            && isSameTree(p->right, q->right);
    }
};
```

➤ Complexity

Complexity

Time $O(\min(n_p, n_q))$: we stop as soon as a mismatch or a null appears, visiting at most the smaller tree. Memory $O(h)$ for the recursion stack, worst case $O(n)$ for a skewed tree. Because the depth can be n, an iterative BFS-pair version (queue of node pairs) avoids stack overflow on pathological inputs.

➤ Pitfalls and Variants

Catching only value mismatches

Comparing values but not shape passes a tree against its mirror image – wrong. The `p == q` null test is what enforces equal shape; do not replace it with two separate `return true` branches that can drift apart.

Variants: *Symmetric Tree* (LC 101) compares a tree to its mirror; *Subtree of Another Tree* (LC 572) asks whether one tree appears inside another.

1.1.8 Practice Problems

Q1.1.1 Invert Binary Tree – LeetCode 226 LC Easy

Hint: swap the two children at every node – a one-line post-order recursion.

Q1.1.2 Univalued Binary Tree – LeetCode 965 LC Easy

Hint: check every node equals the root; pass the expected value down in a DFS.

Q1.1.3 Leaf- Similar Trees – LeetCode 865 LC Easy

Hint: collect leaf values left-to-right with a DFS, then compare the two lists.

Q1.1.4 Minimum Distance Between BST Nodes – LeetCode 783 LC Easy

Hint: Stretch: an in-order walk of a BST is sorted – compare neighbours. See Chapter 5.

1.2 Depth, Height and Size

In This Section

Three numbers describe every node of a rooted tree: how far it is from the root (**depth**), how far the deepest leaf below it is (**height**), and how many nodes hang beneath it (**size**). Nearly every tree algorithm is "compute these bottom-up, then combine them," so they are the first tools to make automatic.

Prerequisites. §1.1 (parent, child, subtree) and the DFS of Code 1.1.1.

After this section you can

- state depth, height and size for every node of a small tree by hand;
- compute all three for the whole tree in a single $O(n)$ DFS;
- write the height recursion correctly under both the edges and nodes conventions;
- avoid the stack overflow that a skewed tree causes on recursive solutions.

1.2.1 Intuition

Root the tree of Figure 1.1.4 at A and imagine two rulers. One hangs from the root downward and reads the *depth* of each node; the other stands on each node and reaches down to the deepest leaf below it, reading the *height*.

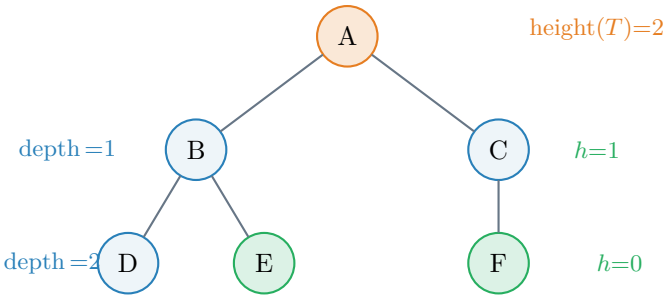


Figure 1.2.1: Blue shows *depth* (measured down from the root, in edges); green shows *height* of a few nodes (edges to the deepest leaf below them). The whole tree has height 2, the height of its root A. Leaves have height 0.

Depth vs Height

Depth depth(v)
edges from the *root* down to v; shared by the walk from the top

Height height(v)
edges from v down to its deepest *leaf*; computed from the bottom up

1.2.2 Definitions and Notation

Definition 1.2.1 (Depth and level)

The **depth** depth(v) of a node is the number of edges on the path from the root to v; the root has depth $= 0$. The **level** of a node is its depth (some books call the root level 1; we always start levels at 0). Siblings share a depth.

Definition 1.2.2 (Height)

The **height** height(v) of a node is the number of edges on the longest path from v down to a leaf in its subtree; a leaf has height $= 0$ and the empty child has height(`null`) $= -1$. The height of the tree is height(T) = height(root).

Definition 1.2.3 (Size)

The **size** size(v) is the number of nodes in subtree(v), counting v itself; so size(`null`) $= 0$ and $\sum$ of nothing is 0.

1.2.3 The Bottom-Up Recurrences

Height and size both "bubble up" from the leaves: a node reads the answers its children computed and folds them together. This is the pattern behind half the book.

Theorem 1.2.4 (Height and Size Recursion)

With height(`null`) $= -1$ and size(`null`) $= 0$, for any node v

$$\text{height}(v) = 1 + \max_{c \in \text{children}(v)} \text{height}(c), \qquad \text{size}(v) = 1 + \sum_{c \in \text{children}(v)} \text{size}(c).$$

A leaf (children(v) $= \emptyset$) gets height $= 0$ and size $= 1$.

Proof

The longest downward path from v must leave v through exactly one child, then follow the longest path from that child; that is $1 + \max_c \text{height}(c)$, and if there is no child the max over the empty set is height(`null`) $= -1$, giving height(v) $= 0$. Every node of subtree(v) is either v itself or a node of exactly one child's subtree, so the counts add to $1 + \sum_c \text{size}(c)$. Both are well-founded because a tree has no cycles, so recursion reaches a leaf in finitely many steps.

■

Key Insight

Depth is a *top-down* quantity (carry it into the recursion as a parameter), while height and size are *bottom-up* (returned from it). One DFS can do all three: push depth in, return height and size out.

Where this is used. Chapter 6 (AVL balance factors), Chapter 9 (diameter) and Chapter 18 (centroid decomposition) – this same "compute a value per node bottom-up, combine children" move returns again and again with a different local formula.

1.2.4 One DFS that Measures Everything

Code 1.2.1: Height of a binary tree (height in edges, height(null) = -1)

```
// height in EDGES: null -> -1, leaf -> 0.  (See the note below for the nodes version .)
int height(TreeNode* r) {
    if (r == nullptr) return -1;
    return 1 + max(height(r->left), height(r->right));
```

```
}
```

Code 1.2.2: One DFS that fills depth, height and size for a general tree

```
vector<vector<int>> adj;
vector<int> depth_, height_, size_;

void measure(int v, int p, int d) {          // p = parent, d = depth of v
    depth_[v]  = d;
    size_[v]   = 1;                           // count v itself
    height_[v] = -1;                          // a leaf ends at -1 + 1 = 0
    for (int to : adj[v]) {
        if (to == p) continue;                // parent guard (see 1.1)
        measure(to, v, d + 1);                // push depth down
        size_[v]   += size_[to];              // pull size up
        height_[v]  = max(height_[v], height_[to]);
    }
    height_[v] += 1;                          // add the edge to the deepest child
}
```

Complexity

Each node is visited once and each edge twice, so the DFS is $O(n)$ time and $O(n)$ memory (arrays plus a recursion stack up to the height of the tree). On a path of n nodes the stack is $O(n)$ deep – see the pitfall below.

Edges vs nodes: an off-by-one you must choose

This book counts height/depth in **edges**: height(`null`) $= -1$, a lone node has height 0. LeetCode's *Maximum Depth* (Code 1.2.3 below) counts in **nodes**: height(`null`) $= 0$, a lone node has depth 1. The two answers differ by exactly one. Read the statement, pick the base case, and never mix them in one program.

1.2.5 Worked Examples

Example 1.2.5 (Reading all three numbers)

In Figure 1.2.1: D has depth 2, height 0, size 1; B has depth 1, height 1, size 3; A has depth 0, height 2, size 6. Note size$(A) = 6$ equals the node count, and height(A) is the tallest root-to-leaf run of edges ($A \to B \to D$ or $A \to B \to E$).

Step-by-Step Trace

Running of Code 1.2.2 on a path $0 - 1 - 2$ rooted at 0 fills depth $\{0, 1, 2\}$, height $\{2, 1, 0\}$ and size $\{3, 2, 1\}$. Height grows from the far leaf back to the root, size collapses onto the root – the two recurrences running in opposite directions along the same walk.

1.2.6 Common Pitfalls

Recursion on a skewed tree

A tree that is really a path of 10^5 nodes makes the DFS recurse 10^5 deep and can overflow the call stack. Prefer an explicit stack (iterative post-order) or raise the stack limit when the judge allows. Every recursive tree routine in this book gets an iterative twin for exactly this reason.

Off-by-one on the base case

Writing height(`null`) $= 0$ when the problem wants edges (or vice-versa) shifts every answer by one. Decide the convention *first*, set the null base case to match, and test on a single-node tree – the fastest way to catch it.

1.2.7 Solved Problems

W1.2.1 Maximum Depth of Binary Tree

Source: LeetCode 104 LC Easy
Techniques: recursion, height of a binary tree

Maximum Depth

Given the root of a binary tree, return the number of *nodes* on the longest path from the root down to the farthest leaf. A single node has depth 1; an empty tree has depth 0.

➤ Understanding the Problem

This is the height of the tree counted in *nodes*, not edges – LeetCode's convention. So the answer is the edge-height of Theorem 1.2.4 plus one. The natural formulation is recursive: the deepest path through a node is one more than the deeper of its two children.

➤ Key Observations

Observation 1. Counting depth in nodes: $\text{depth}(v) = 1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$, with the base case $\text{depth}(\texttt{null}) = 0$.

Observation 2. This mirrors the height recursion exactly; only the base case changes (0, not -1).

➤ Solution

Recurse, returning 0 for a null child and $1 + \max(\text{left}, \text{right})$ otherwise.

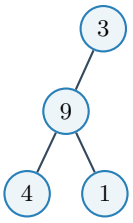


Figure 1.2.2: The longest root-to-leaf path is $3 \to 9 \to 4$ (or $3 \to 9 \to 1$), three *nodes*, so the answer is 3. Counting edges instead would give 2.

➤ Proof of Correctness

Induction on size(v)

Base: $v =$ `null`, the code returns 0, correct. Step: assume the calls on the children return their correct node-depths. The deepest path from v goes to the deeper child and then adds the single node v, so $1 +$ max is exactly the longest root-to-leaf node count at v – the same recurrence the theorem states, shifted by the base case.

■

➤ Implementation

Code 1.2.3: Maximum Depth (LeetCode 104)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == nullptr) return 0;                    // nodes convention
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }
};
```

➤ Complexity

Complexity

Time $O(n)$ (every node visited once). Memory $O(h)$ recursion stack, worst-case $O(n)$ on a skewed tree; a level-order (BFS) count of how many levels there are gives the same answer iteratively with $O(w)$ queue space ($w =$ widest level).

➤ Pitfalls and Variants

Edges vs nodes

Subtracting one here (to answer in edges) or adding one to an edge-height solution is the classic wrong answer. Re-read whether the problem counts nodes or edges before submitting.

Variant: *Minimum Depth* (W1.2.2) below; *Diameter of Binary Tree* (LC 543) uses the same bottom-up idea on two paths at once (Chapter 9).

W1.2.2 Minimum Depth of Binary Tree

Source: LeetCode 111 LC Easy
Techniques: recursion, the one-child trap

Minimum Depth

Return the number of nodes on the shortest path from the root to the nearest *leaf* (a node with no children). The path must end at a real leaf.

➤ Understanding the Problem

The trap: a naive $\min(\text{left}, \text{right})$ with $\text{depth}(\texttt{null}) = 0$ reports depth 1 for a root that has only a right child, because the missing *left* (a null, not a leaf) looks shortest. We must only compare branches that actually reach a leaf.

➤ Key Observations

Observation 1. A null child is *not* a leaf and must never win the min.

Observation 2. If a node has exactly one child, its minimum depth is 1 plus that child's minimum depth – the min has only one candidate.

➤ Solution

Three cases: null returns 0; a node missing one side recurses on the side it has; a node with both children takes $1 + \min$ of the two.

➤ Proof of Correctness

Induction on size(v)

For $v = \texttt{null}$ the answer is 0. If v has exactly one child c, the only root-to-leaf paths go through c, so the minimum is $1 + \text{depth}_{\min}(c)$, which the code returns. If v has two children, every leaf lies under one of them, and the shortest path picks the smaller of the two correct recursive answers plus v. In the remaining case (v is a leaf) both children are null, the two-child formula gives $1 + \min(0, 0)$ only if we let nulls score 0 – which is why we special-case missing children above instead.

■

➤ Implementation

Code 1.2.4: Minimum Depth (LeetCode 111)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
```

```
    int minDepth(TreeNode* root) {
        if (root == nullptr) return 0;
        if (root->left  == nullptr) return 1 + minDepth(root->right);  // only right
        if (root->right == nullptr) return 1 + minDepth(root->left);   // only left
        return 1 + min(minDepth(root->left), minDepth(root->right));   // both
    }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$ recursion stack ($O(n)$ worst case). A BFS that returns as soon as it pops a leaf is often faster in practice (it stops at the first leaf) and is naturally iterative.

➤ Pitfalls and Variants

The one-child case is the whole problem

Collapsing to min(left, right) with nulls scoring 0 fails the moment a node has a single child – a huge fraction of the tests. Keep the two guard lines.

Variant: minimum depth to *any* node with a property (e.g. the nearest leaf matching a predicate) is the same BFS-with-early-exit pattern.

1.2.8 Practice Problems

Q1.2.1 Sum of Left Leaves – LeetCode 404 LC Easy

Hint: recurse, passing a flag for "I am a left child"; add the value when a leaf is left.

Q1.2.2 Tree Diameter (Longest Path) – CSES 1135 CSES

Hint: two BFS/DFS sweeps: farthest node from an arbitrary start, then farthest from that. See Chapter 9.

Q1.2.3 Diameter of Binary Tree – LeetCode 543 LC Easy

Hint: bottom-up: return height, update a global best with left-height + right-height.

Q1.2.4 Binary Tree Longest Consecutive Sequence – LeetCode 298 LC Medium

Hint: Stretch: carry the current run length down in a DFS; compare both children, do not branch-max naively.

1.3 Kinds of Trees

In This Section

"Tree" hides a family of shapes, and each extra restriction buys a counting rule or a faster algorithm. Knowing whether your tree is *full*, *complete*, *perfect* or *balanced* tells you its height from its size, whether an array can store it, and which problems become one line instead of a recursion.

Prerequisites. §1.2 (height, size) and Theorem 1.1.4.

After this section you can

- recognise and name binary, full, complete, perfect, balanced and skewed trees;

- never confuse "full", "complete" and "perfect" again;
- derive the node and leaf counts of perfect and full trees;
- bound the height of any binary tree with n nodes by $\lfloor \log_2 n \rfloor$ and $n - 1$.

1.3.1 Intuition

A **binary tree** restricts every node to at most two children, conventionally called `left` and `right`. That single rule is what makes tree algorithms branch in two and lets us store a tree in an array. The other shapes in this chapter are binary trees with one more promise each.

Single-elimination tournament

A *perfect* binary tree is a knockout bracket: two players per match (the children), the winners fill the next round (level) completely, and the number of matches is fixed by the number of players. Cut the last round short and you get a *complete* tree; let someone get a bye and it is no longer complete. The bracket picture is why the counts come out as powers of two.

1.3.2 Definitions and Notation

Definition 1.3.1 (Binary and m-ary tree)

A **binary tree** is a rooted tree in which each node has at most two children, named `left` and `right` (the two slots are distinguished, so a node with one child keeps track of whether it is the left or the right one). An **m-ary tree** allows at most m children; a general tree is an m-ary tree with unbounded m.

Definition 1.3.2 (Skewed (degenerate) tree)

A binary tree is **skewed** (or **degenerate**) when every internal node has exactly one child, so the tree is really a linked list: all-left or all-right. A skewed tree on n nodes has height $n - 1$ – the worst case for any recursive tree code.

Definition 1.3.3 (Full (proper) binary tree)

A binary tree is **full** (or **proper**, or strict) when every internal node has *exactly two* children – a node has 0 or 2 children, never 1.

Definition 1.3.4 (Perfect binary tree)

A binary tree is **perfect** when every internal node has two children *and* all leaves are at the same depth. It is a completely filled bracket of height h.

Definition 1.3.5 (Complete binary tree)

A binary tree is **complete** when every level except possibly the last is full, and the last level is filled left to right with no gaps. Every perfect tree is complete; the converse needs the last level to be full.

Definition 1.3.6 (Height-balanced tree)

A binary tree is **(height-)balanced** when for every node the heights of its left and right subtrees differ by at most 1. Balancedness is what keeps the height logarithmic so searches stay fast; it is the property Chapters 5–6 fight to maintain.

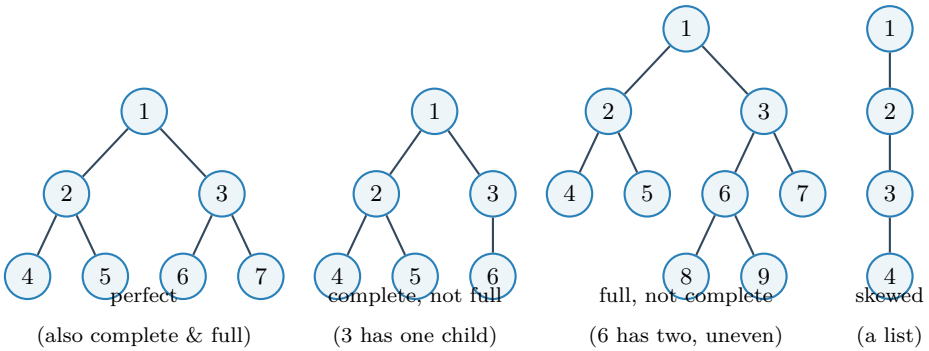


Figure 1.3.1: Four shapes, four promises. Read the boxes under each to see which definition the tree satisfies and which it breaks.

Full vs Complete vs Perfect

Full / Perfect

full: every internal node has two children; perfect adds "all leaves level"

Complete

every level filled left-to-right, last level may be short but has no gaps

1.3.3 Counting Nodes and Leaves

Each extra restriction pins down the size. These identities are the ones interviewers expect cold.

Theorem 1.3.7 (Perfect Tree Counts)

A perfect binary tree of height h (counted in edges) has $2^{h+1} - 1$ nodes, 2^h leaves, and 2^ℓ nodes on level ℓ (levels numbered from 0).

Proof

Level ℓ doubles each step (each node has two children), so it holds 2^ℓ nodes; levels run $0..h$, giving $\sum_{\ell=0}^{h} 2^\ell = 2^{h+1} - 1$ nodes by the geometric series. The only nodes with no children are those on the last level h, so leaves $= 2^h$.

■

Key Insight

Invert it: a perfect tree with n nodes has $h = \log_2(n+1) - 1$, so $n+1$ is a power of two. This is why "is it perfect?" can be answered by counting levels, never by pointer chasing.

Theorem 1.3.8 (Leaves of a Full Binary Tree)

In a full binary tree with I internal nodes and L leaves, $L = I + 1$.

Proof

Count edges two ways. Every internal node has exactly two children, contributing two parent-to-child edges, so the tree has $2I$ edges. But any tree on $n = I + L$ nodes has $n - 1$ edges (Theorem 1.1.4). Therefore $2I = (I + L) - 1$, i.e. $L = I + 1$.

■

Corollary 1.3.9 (Height Bounds of a Binary Tree)

A binary tree with n nodes has height (in edges) $\lfloor \log_2 n \rfloor \le \text{height} \le n - 1$. The lower bound is attained by a perfect (or complete) tree, the upper bound by a skewed tree.

Proof

A tree of height h holds at most $2^{h+1} - 1$ nodes (a perfect tree), so $n \le 2^{h+1} - 1 < 2^{h+1}$, giving $h > \log_2 n - 1$, i.e. $h \ge \lfloor \log_2 n \rfloor$. On the other hand each edge can increase the longest root-to-leaf path by at most one, and there are only $n - 1$ edges, so $h \le n - 1$. Both extremes are the perfect and skewed trees above.

■

Key Insight

"Balanced means height $O(\log n)$." Keeping h near its lower bound is the entire point of AVL and red–black trees (Chapter 6): it turns $O(n)$ lists into $O(\log n)$ searches.

Where this is used. Chapter 6 (self-balancing BSTs), Chapter 7 (heaps in an array) and Chapter 13 (segment trees) – complete and perfect trees are exactly the ones an array stores with no wasted slots, and balanced trees are the ones whose height stays $O(\log n)$.

1.3.4 Worked Examples

Example 1.3.10 (Reading the gallery)

In Figure 1.3.1 the third tree has internal nodes $\{1, 2, 3, 6\}$ ($I = 4$) and leaves $\{4, 5, 8, 9, 7\}$ ($L = 5$): indeed $L = I + 1$, matching Theorem 1.3.8. The first is perfect with $h = 2$, so $2^3 - 1 = 7$ nodes and $2^2 = 4$ leaves – count them to confirm.

Step-by-Step Trace

For $n = 6$ nodes, Corollary 1.3.9 says $\lfloor \log_2 6 \rfloor = 2 \le h \le 5$. A complete tree of six nodes has height 2 (near the best), the skewed tree has height 5 (the worst) – both inside the band.

1.3.5 Common Pitfalls

Confusing full / complete / perfect

They are not synonyms and not even comparable in one direction: a complete tree need not be full (a last-level parent can have one child), and a full tree need not be complete (uneven depth). "Perfect" is the only one that is both. Sketch a two-level and a three-level example of each before you assert a property.

Balanced is per-node, not just root

Checking the height difference at the root only is not enough – W1.3.1 has test cases where the root looks balanced but a deep subtree is not. The condition must hold for *every* node.

1.3.6 Solved Problems

W1.3.1 Balanced Binary Tree

Source: LeetCode 110 LC Easy
Techniques: bottom-up height, early exit, sentinel

Balanced Binary Tree

Given a binary tree, decide whether it is height-balanced: for every node, the heights of its two subtrees differ by at most one.

➤ Understanding the Problem

The naive plan – a full height computation at every node – is $O(n^2)$ (height is $O(n)$ and we call it n times). We can do one bottom-up pass that returns each node's height and simultaneously checks its balance, reusing every child's height instead of recomputing it.

➤ Key Observations

Observation 1. If a subtree is unbalanced the whole tree is unbalanced, so we can abort early by propagating a special "unbalanced" signal upward.

Observation 2. Returning -1 as that signal works because a real height is never negative (with the null $= 0$ nodes convention here).

➤ Solution

A recursive `check` returns the height of a balanced subtree, or -1 if that subtree is unbalanced. A node is balanced iff both children returned non-negative and their heights differ by at most one.

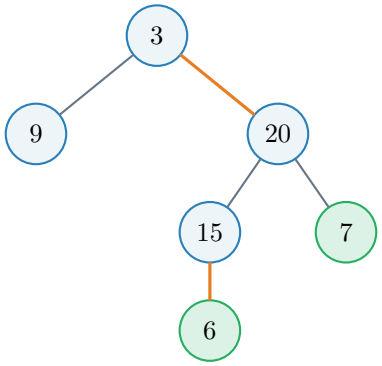


Figure 1.3.2: Not balanced: at node 20 the left subtree (height 2 through the orange edges down to 6) and the right subtree (height 0) differ by more than one, even though the root itself looks fine.

➤ Proof of Correctness

Induction on size(v)

By Theorem 1.2.4 the returned height is correct whenever the subtree is balanced. If a child returns -1, its subtree violates the condition at some node (induction hypothesis) and v correctly propagates -1. Otherwise v's subtree is balanced exactly when both children's subtrees are (induction) *and* v itself satisfies $|\text{height(left)} - \text{height(right)}| \leq 1$, which the code tests. The root's return is negative iff the whole tree is unbalanced.

■

➤ Implementation

Code 1.3.1: Balanced Binary Tree (LeetCode 110)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int check(TreeNode* r) {                // height if balanced, -1 if not
        if (r == nullptr) return 0;
        int lh = check(r->left);   if (lh == -1) return -1;
        int rh = check(r->right);  if (rh == -1) return -1;
        if (abs(lh - rh) > 1) return -1;
        return 1 + max(lh, rh);
    }
    bool isBalanced(TreeNode* root) { return check(root) != -1; }
};
```

➤ Complexity

Complexity

Time $O(n)$: each node is visited once and its children's heights are reused, not recomputed (versus $O(n^2)$ for the root-only-height approach). Memory $O(h)$ stack, worst case $O(n)$.

➤ Pitfalls and Variants

Two traversals double the cost

Writing `isBalanced` to call a separate `height` on every node is correct but $O(n^2)$. The single-pass sentinel version above is what interviewers want to see.

Variant: *Minimum Absolute Difference in BST* or any per-node property uses the same "return a value, abort early" bottom-up shape.

W1.3.2 Count Complete Tree Nodes

Source: LeetCode 222 LC Medium
Techniques: complete-tree structure, perfect shortcut, divide and conquer

Count Complete Tree Nodes

Given the root of a *complete* binary tree, return the number of nodes. Aim for better than $O(n)$: the structure should let you skip whole perfect subtrees.

➤ Understanding the Problem

A plain DFS counting nodes is $O(n)$ and ignores the promise. In a complete tree, compare the length of the left spine to the right spine of a subtree: if they are equal, that subtree is *perfect* and holds $2^{\text{height}} - 1$ nodes (Theorem 1.3.7) – count it instantly. Otherwise recurse, but only down the side that actually changes.

➤ Key Observations

Observation 1. If the left-most and right-most paths from a node have the same length, the subtree is perfect and its size is a power of two minus one.

Observation 2. If they differ, exactly one child subtree is perfect, so each step halves the problem.

➤ Solution

Code 1.3.2: Count Complete Tree Nodes (LeetCode 222)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int leftHeight(TreeNode* n)  { int h = 0; while (n) { ++h; n = n->left;  } return
    ↪ h; }
    int rightHeight(TreeNode* n) { int h = 0; while (n) { ++h; n = n->right; } return
    ↪ h; }

    int countNodes(TreeNode* root) {
        if (root == nullptr) return 0;
        int lh = leftHeight(root), rh = rightHeight(root);
        if (lh == rh) return (1 << lh) - 1;              // perfect subtree: 2^lh - 1
        return 1 + countNodes(root->left) + countNodes(root->right);
    }
};
```

➤ Proof of Correctness

Proof

When the two spine lengths agree ($= h$) the complete tree of height h must have its last level full, hence perfect, so the count is $2^h - 1$ by Theorem 1.3.7. When they differ, the root contributes 1 and we recurse on the (still complete) children; correctness follows by induction on height, since every subtree of a complete tree rooted at a node is again complete.

■

➤ Complexity

Complexity

Spine lengths cost $O(\log n)$ each; we do this on one root-to-leaf path, so $O(\log^2 n)$ time – a large win over $O(n)$ when the tree is deep and full. Memory $O(\log n)$ stack.

➤ Pitfalls and Variants

$1 \ll lh$ overflow

For very tall trees 2^{lh} can exceed `int`; if n may be large use `long long` and $1 \ll lh$ only up to $lh < 31$. Contest node counts keep lh small, but check the constraint.

Variant: the same "measure both spines, shortcut perfect subtrees" idea finds the last (level-order) node or the insertion point in a heap-backed array.

1.3.7 Practice Problems

Q1.3.1 Symmetric Tree – LeetCode 101 LC Easy

Hint: compare a tree with its mirror: recurse on (left.left, right.right) and (left.right, right.left).

Q1.3.2 Binary Tree Level Order Traversal – LeetCode 102 LC Medium

Hint: BFS with a queue; process one whole level per iteration to fill the levels.

Q1.3.3 Convert Sorted Array to BST – LeetCode 108 LC Easy

Hint: pick the middle element as root, recurse on halves – builds a balanced tree. See Chapter 5.

Q1.3.4 Binary Tree Right Side View – LeetCode 199 LC Medium

Hint: Stretch: the last node of each BFS level; or a DFS that visits right before left.

Q1.3.5 Complete Binary Tree Inserter – LeetCode 919 LC Medium

Hint: Stretch: keep a queue of parents that still have a free slot to insert left-to-right.

■ 1.4 Counting Trees

In This Section

"How many trees are there?" has two very different answers depending on what you are counting: the number of *shapes* (Catalan numbers) and the number of ways to put *distinct labels* on those shapes (Cayley's n^{n-2}). Both show up as DP counts and as combinatorics problems, so it pays to know the two formulas and which question each answers.

Prerequisites. §1.3 (binary shapes) and basic binomial coefficients.

After this section you can

- count binary-tree shapes with the Catalan recurrence and closed form;
- state Cayley's formula and say what a labelled tree is;
- compute both modulo a prime without overflow;
- recognise a Catalan count hiding inside a DP problem (Chapter 10).

1.4.1 Intuition

An **ordered** binary tree distinguishes a left child from a right child, so a three-node tree can be arranged in five different shapes – one per way the single "extra" node can hang off a two-node skeleton.

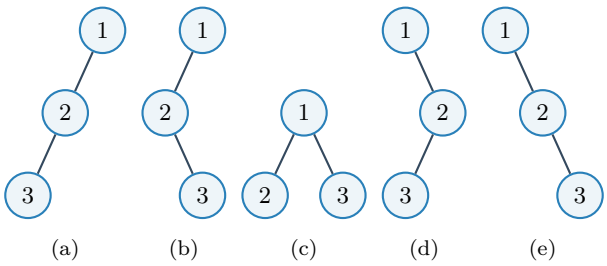


Figure 1.4.1: All five shapes of a three-node binary tree, so $C_3 = 5$. They differ only in which of the four slots the nodes occupy – empty slots are drawn with `[,phantom]`.

1.4.2 Definitions and Notation

Definition 1.4.1 (Shape vs labelled tree)

The **shape** of a rooted (binary) tree is the tree with its values ignored – only the parent–child connections matter. A **labelled tree** on n vertices keeps the shape *and* assigns the distinct names $1, 2, \ldots, n$ to the nodes. Two trees are equal as shapes iff an isomorphism maps one to the other preserving children; they are equal as labelled trees only if the labels match too.

Definition 1.4.2 (Counted objects)

We count three related quantities for n nodes: the number of *binary-tree shapes* (ordered, values ignored); the number of *binary trees labelled* $1..n$ (a shape times the ways to fill it); and the number of *labelled trees* whose edges are chosen freely (not restricted to be binary, and undirected). Each has its own formula below.

1.4.3 Shapes: the Catalan Numbers

Theorem 1.4.3 (Binary Tree Shapes are Catalan)

The number of distinct shapes of a binary tree with n nodes is the Catalan number

$$C_n = \frac{1}{n+1}\binom{2n}{n} = 1, 1, 2, 5, 14, 42, 132, \ldots \quad (n = 0, 1, 2, \ldots)$$

equivalently $C_0 = 1$ and $C_n = \sum_{i=0}^{n-1} C_i \, C_{n-1-i}$.

Proof

Root-splitting counts the recurrence. A tree with $n \geq 1$ nodes spends one node on the root; the remaining $n-1$ split into i nodes in the left subtree and $n-1-i$ in the right, for some $i = 0, \ldots, n-1$. Left and right shapes are chosen independently, giving $C_i \cdot C_{n-1-i}$ trees for that i, and summing over i yields $C_n = \sum_i C_i C_{n-1-i}$ with $C_0 = 1$ (the empty shape). This is exactly the Catalan recurrence, whose closed form is $\frac{1}{n+1}\binom{2n}{n}$ (obtainable by generating functions or by a bijection with balanced parenthesis strings).

■

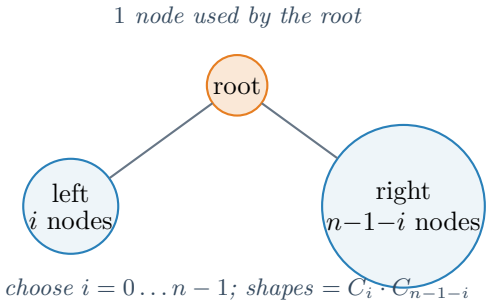


Figure 1.4.2: The Catalan recurrence: the root splits the other $n-1$ nodes between two independent subtrees, and we sum the products over every split.

The number of labelings for a fixed shape

Given a binary shape of n nodes, filling it with labels $1..n$ so that it stays a *BST* is forced (in-order must be sorted), which is why counting BSTs = counting shapes = C_n. Filling it with labels with no BST rule gives $n!$ labelled trees on that shape.

1.4.4 Labelled Trees: Cayley's Formula

Theorem 1.4.4 (Cayley's Formula)

The number of distinct trees on n *labelled* vertices $1, \ldots, n$ (edges arbitrary, not restricted to binary or rooted) is n^{n-2}. Rooted at any vertex, this becomes n^{n-1} labelled rooted trees.

Proof idea (Prüfer sequences)

There is a bijection between labelled trees on n vertices and sequences ("Prüfer codes") of length $n-2$ whose entries are any of the n labels. Since each of the $n-2$ positions holds one of n choices freely, there are n^{n-2} such sequences, hence that many trees. The bijection deletes the smallest leaf repeatedly and records its neighbour; reversing the process rebuilds the tree from the code. The construction and its inverse are built in full in Chapter 17.

■

Key Insight

$n = 1$: one tree (1^{-1} read as 1). $n = 2$: $2^0 = 1$ edge. $n = 3$: $3^1 = 3$. $n = 4$: $4^2 = 16$. Growing far slower than $n!$ – most permutations of a shape collapse to the same unrooted tree.

Where this is used. Chapter 10 and Chapter 20 (DP that counts trees), and Chapter 17 (Prüfer codes) – Catalan-style root-splitting recurrences reappear directly in tree-counting DPs, and the Prüfer bijection behind Cayley's formula – including building a tree from its code – is constructed in Chapter 17.

1.4.5 Computing the Counts

Code 1.4.1: Catalan numbers by DP, $C_0 \dots C_n$ (grows fast – use big integers past $n \approx 30$)

```cpp
#include <bits/stdc++.h>
using namespace std;

// C[0..n], the number of binary-tree shapes with k nodes is C[k].
vector<long long> catalan(int n) {
    vector<long long> C(n + 1, 0);
    C[0] = 1;
    for (int k = 1; k <= n; ++k)
        for (int i = 0; i < k; ++i)
            C[k] += C[i] * C[k - 1 - i];     // root split: i left, k-1-i right
    return C;
}
```

Code 1.4.2: Cayley modulo a prime: $n^{n-2} \bmod m$ by fast exponentiation

```cpp
long long modpow(long long b, long long e, long long m) {
    long long r = 1 % m;
    b %= m;
    for (; e > 0; e >>= 1, b = b * b % m)
        if (e & 1) r = r * b % m;
    return r;
}
// number of labelled trees on n vertices modulo m
long long labelledTrees(long long n, long long m) {
    return n <= 1 ? 1 % m : modpow(n, n - 2, m);   // n^(n-2) mod m
}
```

Complexity

The Catalan DP is $O(n^2)$ time, $O(n)$ memory; use $O(n)$ only if you switch to the closed form with modular inverses. Cayley with binary exponentiation is $O(\log n)$ after the $n-2$ exponent is set. Both counts overflow `long long` almost immediately (C_{35} exceeds 2^{63}), so contests ask for the answer modulo a prime – then reduce every multiply.

1.4.6 Worked Examples

Example 1.4.5 (Small values)

Figure 1.4.1 shows $C_3 = 5$. Extending the root-split, $C_4 = C_0C_3 + C_1C_2 + C_2C_1 + C_3C_0 = 5 + 2 + 2 + 5 = 14$, and $C_5 = 42$. For labelled trees, Cayley gives $n = 4 \Rightarrow 4^2 = 16$ and $n = 5 \Rightarrow 5^3 = 125$.

Step-by-Step Trace

of Code 1.4.1 builds the array $C = [1, 1, 2, 5, 14, 42, 132]$ for $n = 6$: each new entry is the running sum of products from Figure 1.4.2, matching Theorem 1.4.3 line for line.

n	0	1	2	3	4	5	6
C_n (shapes)	1	1	2	5	14	42	132
labelled trees n^{n-2}	–	1	1	3	16	125	1296

1.4.7 Common Pitfalls

Shapes vs labelled trees

C_n counts *shapes* (labels ignored); n^{n-2} counts *labelled* trees (edges free, labels matter, not necessarily binary). Quoting the wrong one is a whole- problem zero. Ask first: do node identities matter, and is binary structure required?

Overflow and modulo

Catalan and n^{n-2} explode; in contests the modulus is part of the task. Never compute $\binom{2n}{n}$ as raw integers – use modular inverses or the DP, all under the modulus.

1.4.8 Solved Problems

W1.4.1 Unique Binary Search Trees

Source: LeetCode 96 LC Medium
Techniques: Catalan numbers, root-splitting DP

Unique BSTs

Given n, count how many *structurally unique* BSTs store the keys $1 \dots n$. For $n = 3$ the answer is 5.

➤ Understanding the Problem

A BST's in-order traversal is the sorted keys, so once we pick which key is the root, the smaller keys are forced left and the larger are forced right. Different shapes give different trees, and two trees with the same shape but different keys can't both be valid BSTs on $1..n$. So counting BSTs is exactly counting binary shapes on n nodes – the Catalan number C_n (Theorem 1.4.3).

➤ Key Observations

Observation 1. Choosing root key r splits keys into $r - 1$ smaller and $n - r$ larger; the two sides form independent subtrees.

Observation 2. So $G(n) = \sum_{r=1}^{n} G(r-1)\,G(n-r)$ with $G(0) = G(1) = 1$ – the Catalan recurrence.

➤ Solution

Bottom-up DP over subtree size, summing the products of left and right counts, exactly Code 1.4.1.

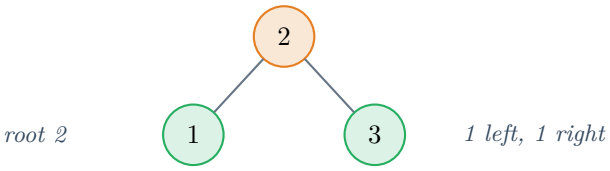


Figure 1.4.3: One of the five BSTs on $\{1, 2, 3\}$: root 2 forces 1 left and 3 right. Pick root 1 and both $2, 3$ go right (two shapes), root 3 mirrors it – 5 in total.

➤ Proof of Correctness

Proof

By the BST in-order property, key r as root uses exactly keys $< r$ on the left and $> r$ on the right, and the subtrees are independent valid BSTs, contributing $G(r-1)G(n-r)$ trees. The cases partition all BSTs by their root, so $G(n) = \sum_r G(r-1)G(n-r)$. The DP computes this with base $G(0) = G(1) = 1$ and returns $G(n) = C_n$.

■

➤ Implementation

Code 1.4.3: Unique Binary Search Trees (LeetCode 96)

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int numTrees(int n) {
        vector<int> dp(n + 1, 0);
        dp[0] = 1; dp[1] = 1;
        for (int nodes = 2; nodes <= n; ++nodes)
            for (int root = 1; root <= nodes; ++root)
                dp[nodes] += dp[root - 1] * dp[nodes - root];
        return dp[n];
    }
};
```

➤ Complexity

Complexity

Time $O(n^2)$, memory $O(n)$. For LC $n \leq 19$ the value fits in `int`; on larger inputs or with a modulus, do the same DP with `long long` and reduce. A closed-form C_n with modular inverses is $O(n)$.

➤ Pitfalls and Variants

Counting vs generating

W1.4.1 only *counts*; *Unique BST II* (LC 95) builds every tree, so the same recurrence is applied to node objects instead of integers – $O(C_n)$ output, and recursion, not a flat DP.

1.4.9 Practice Problems

Q1.4.1 Unique Binary Search Trees II – LeetCode 95 LC Medium

Hint: same root split, but build and combine actual left/right subtree lists.

Q1.4.2 Different Ways to Add Parentheses – LeetCode 241 LC Medium

Hint: each operator as a root splits the expression – a Catalan-shaped interval DP. See Chapter 10.

Q1.4.3 Count Binary Trees With Factors – LeetCode 852 LC Medium

Hint 1: sort values; for each root count factor pairs that multiply to it.

Hint 2: DP: answer[v] = 2 * sum over proper factor pairs (a,b) of answer[a]*answer[b]; memoise with a hash map of value to count.

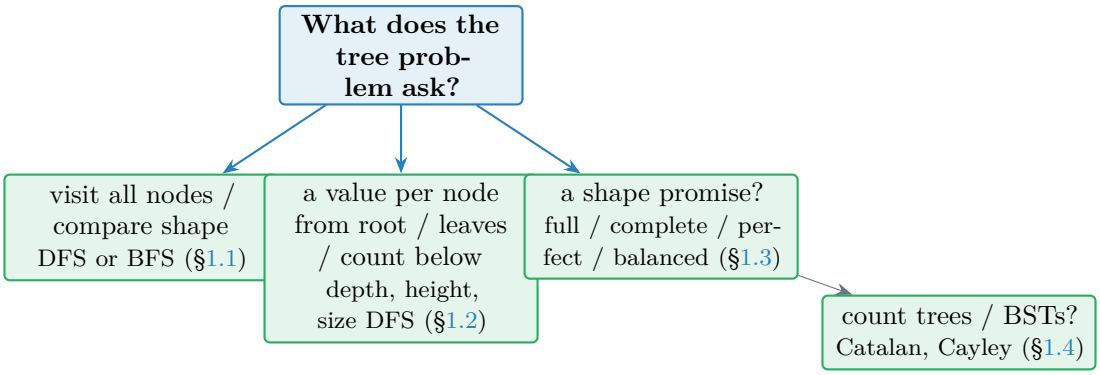
Chapter 1 Summary

Everything in this chapter is a tool the rest of the book assumes. The table collects the results with the exact conditions they need.

Result	Statement (with conditions)	Where
Edges of a tree	A tree on n nodes has exactly $n-1$ edges.	Theorem 1.1.4
Unique path	Exactly one simple path joins any two nodes (connected, acyclic).	Theorem 1.1.5
Height recursion	$\text{height}(\texttt{null}) = -1$, $\text{height}(v) = 1 + \max_c \text{height}(c)$.	Theorem 1.2.4
Size recursion	$\text{size}(\texttt{null}) = 0$, $\text{size}(v) = 1 + \sum_c \text{size}(c)$.	Theorem 1.2.4
Perfect counts	Height h (edges) $\Rightarrow 2^{h+1}-1$ nodes, 2^h leaves.	Theorem 1.3.7
Full tree leaves	Every internal node has 2 children $\Rightarrow L = I + 1$.	Theorem 1.3.8
Height bounds	$\lfloor \log_2 n \rfloor \le \text{height} \le n-1$ (binary, n nodes).	Corollary 1.3.9
Catalan shapes	Binary shapes on n nodes $= C_n = \frac{1}{n+1}\binom{2n}{n}$.	Theorem 1.4.3
Cayley trees	Labelled (free) trees on n vertices $= n^{n-2}$.	Theorem 1.4.4

Algorithms from this chapter.

Algorithm	Time	Memory	Code
Build adjacency from edges	$O(n)$	$O(n)$	Code 1.1.1
One DFS: depth, height, size	$O(n)$	$O(n)$	Code 1.2.2
Structural equality (Same Tree)	$O(\min(n, m))$	$O(h)$	Code 1.1.2
Balanced check (single pass)	$O(n)$	$O(h)$	Code 1.3.1
Count complete-tree nodes	$O(\log^2 n)$	$O(\log n)$	Code 1.3.2
Catalan by DP	$O(n^2)$	$O(n)$	Code 1.4.1



Choosing a Chapter-1 tool from how the problem is phrased. Harder phrasings (path queries, subtree updates) point to later chapters.

Common Mistakes Checklist

- **Missing the `null`/leaf base case.** Every tree recursion must say what a null (or a leaf) returns *before* recursing. Fix: write the base case first.
- **Depth vs height, edges vs nodes.** They differ by one and run in opposite directions. Fix: pick the convention, set the base case to match, test a one-node tree.
- **Forgetting the parent guard.** A DFS on an adjacency list without `if (to == p) continue;` loops forever. Fix: always pass the parent.
- **Recursion on a skewed tree.** A path of 10^5 nodes overflows the stack. Fix: iterative post-order with an explicit stack.
- **Full / complete / perfect mixed up.** They give different counting rules. Fix: sketch a two- and three-level example of each before asserting one.
- **Balanced checked only at the root.** The condition is per-node. Fix: return height bottom-up and a sentinel the moment any node breaks it.
- **Shapes vs labelled trees.** C_n ignores labels; n^{n-2} is unrooted and unbinary. Fix: ask whether identities matter and whether binary structure is required.
- **Overflow on counts.** Catalan and n^{n-2} explode past tiny n. Fix: reduce modulo the given prime at every multiply.

Quick Reference Card

Chapter 1 at a glance

Definitions. Tree = connected + acyclic. Rooted: parent/child/leaf/subtree/ancestor. depth down from root, height down to deepest leaf, size nodes in subtree.

Core identities. n nodes $\Rightarrow$ $n-1$ edges. Unique path between any two nodes. height(`null`)$=-1$, height$(v)=1+\max_c$ height(c); size$(v)=1+\sum_c$ size(c). Perfect: $2^{h+1}-1$ nodes, 2^h leaves. Full binary: $L=I+1$. Height: $\lfloor \log_2 n \rfloor \le h \le n-1$.

Counting. Binary shapes $C_n = \frac{1}{n+1}\binom{2n}{n} = 1, 1, 2, 5, 14, 42, \ldots$; labelled trees n^{n-2}; labelled rooted trees n^{n-1}.

Conventions. Heights/depths in *edges* unless a problem says nodes (LeetCode *Max Depth* counts nodes: subtract 1 to convert).

Code 1.4.4: Chapter 1 code kit: the two shapes you reuse everywhere

```cpp
#include <bits/stdc++.h>
using namespace std;

// (1) General tree from an edge list: DFS with a parent guard, all three measures.
int n; vector<vector<int>> adj; vector<int> dep, hei, sz;
void measure(int v, int p, int d) {
    dep[v] = d; sz[v] = 1; hei[v] = -1;                // edges convention
    for (int to : adj[v]) if (to != p) {
        measure(to, v, d + 1);
        sz[v]   += sz[to];
        hei[v]   = max(hei[v], hei[to]);
    }
    ++hei[v];
}

// (2) Binary tree height (edges) and node-count height (nodes) in two lines.
int heightEdges(TreeNode* r) { return !r ? -1 : 1 + max(heightEdges(r->left),
    ↪ heightEdges(r->right)); }
int heightNodes(TreeNode* r) { return !r ?  0 : 1 + max(heightNodes(r->left),
    ↪ heightNodes(r->right)); }
```

Mixed Problem Set

These blend the chapter's ideas and do not name the topic; decide the tool yourself.

M1.1 Merge Two Binary Trees – LeetCode 617 LC Easy

Hint: recurse in lockstep like Same Tree, but build a new node from the sum at each pair.

M1.2 Path Sum – LeetCode 112 LC Easy

Hint: carry a running total down; a root-to-leaf path must hit the target exactly at a leaf.

M1.3 Subtree of Another Tree – LeetCode 572 LC Easy

Hint: at each node try Same Tree; otherwise recurse into both children.

M1.4 Lowest Common Ancestor of a BST – LeetCode 235 LC Medium

Hint: use the BST order: descend until the two targets split sides. See Chapter 11.

M1.5 Lowest Common Ancestor of a Binary Tree – LeetCode 236 LC Medium

Hint: Stretch: post-order returning which target was found; the node where both appear is the LCA (Chapter 11).

M1.6 Binary Tree Maximum Path Sum – LeetCode 124 LC Hard

Hint 1: Stretch: like the diameter – a global best uses the best path THROUGH each node.

Hint 2: DFS returns the best downward gain from a node $(\max(0, \text{child}))$; update the answer with node + left + right.

Related Topics

- Chapter 2 – Binary trees in depth: the `TreeNode` model, construction, and more traversals.
- Chapter 3 – Preorder, inorder, postorder and level order as reusable walks.
- Chapter 4 – Storing trees: arrays, adjacency lists, parent/child pointers, Euler tour.
- Chapter 6 – Self-balancing trees, where the height-bounds corollary becomes a design goal.
- Chapter 9 – Diameter and centroid, built on the same bottom-up height idea.
- Chapter 10 – Tree DP: the "combine children, return a value" pattern, formalised.
- Chapter 11 – Lowest common ancestor, the path tool this chapter only hinted at.

- Chapter 17 – Prüfer codes: the full bijection behind Cayley's n^{n-2}.

2

Chapter

Binary Trees

Level: LC Easy–Medium • **Problems:** 26 (9 fully solved) • **Status:** Written

Chapter at a Glance

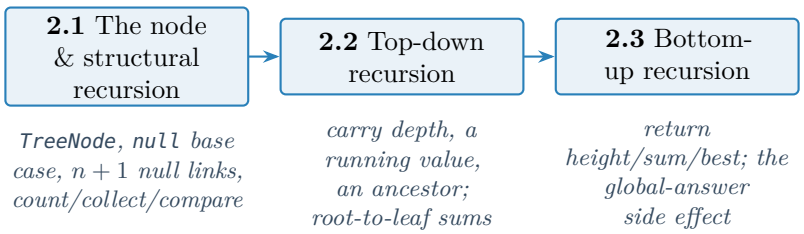
A **binary tree** is the shape your code actually touches in an interview: a `TreeNode` with a value and two pointers, `left` and `right`, either of which may be `null`. Chapter 1 told you what a tree *is* and how to name its parts; this chapter is about *holding one in your hand as pointers* and the one move that solves almost every binary-tree problem – a recursion that trusts the two subtrees to already be solved and only reasons about the node in between. We make that move precise in three guises: pure *structural recursion* (count, collect, compare), *top-down* recursion that carries information from the root, and *bottom-up* recursion that returns an aggregate from the leaves.

You will learn to

- build and read a binary tree from `TreeNode` pointers and reason about its $n + 1$ empty (`null`) links;
- write the structural-recursion template – base case at `null`, combine two already-correct subtree answers – and know why it is correct by induction;
- decide whether a problem wants state carried *down* (depth, a running sum, a grandparent pointer) or an aggregate pulled *up* (tilt, a subtree sum, best path);
- recognise the "return a value *and* update a global best" idiom behind diameter, tilt and longest-path problems.

Prerequisites. Chapter 1 – in particular §1.1 (parent, child, subtree) and §1.2 (the height/size recursion, which we now generalise). The special shapes are already defined in §1.3; we will not redefine them.

In contests and interviews. The LeetCode "Binary Tree" tag is dominated by exactly these three patterns. Once you can name which direction information flows, half the easy/medium tree problems become one function you can write from memory.



The three topics of this chapter. Information flows *into* the recursion in 2.2 and *out of* it in 2.3; 2.1 is the shared skeleton both are built on.

Topic	Key results	Solved problems
§2.1 The node & structural recursion	$n+1$ null links; the trust-the-subtrees template	W2.1.1, W2.1.2, W2.1.3
§2.2 Top-down recursion	carry a parameter in; root-to-leaf accumulation	W2.2.1, W2.2.2, W2.2.3
§2.3 Bottom-up recursion	return an aggregate; global-best side effect	W2.3.1, W2.3.2, W2.3.3

Notation Used in This Chapter

Symbol	Meaning	Symbol	Meaning
`r->left`	pointer to the left child	`r == nullptr`	the `null` / empty child
`r->val`	value stored at node r	size(v)	nodes in subtree of v
`null`	the absent child pointer	height(v)	edges from v to deepest leaf
T	a binary tree	n	number of nodes in T
acc	value carried down (2.2)	$g(v)$	aggregate returned up (2.3)

Throughout, a **binary tree** is the one defined in Definition 1.3.1: every node has at most two, *distinguished* children. Heights and depths stay in **edges** as in Chapter 1, so a lone root has height $= 0$ and height(`null`) $= -1$; LeetCode's *Max Depth* counts nodes and is exactly one more (see the note in §1.2).

2.1 The Binary Tree Node and Structural Recursion

In This Section

Every binary-tree interview answer is one recursive function over a `TreeNode`. This section isolates the bare skeleton that all of them share – handle the `null` base case, recurse on both children, combine the two answers with the current node – and proves it correct by induction. Once the skeleton is automatic, the rest of the chapter is only about *what* you combine.

Prerequisites. §1.1 (parent, child, subtree, `null`) and §1.2 (the height and size recurrences, which are our first examples).

After this section you can

- represent a binary tree with a `TreeNode` struct and link nodes by hand;
- state and prove the "n nodes, $n+1$ null links" counting fact;
- write the structural-recursion template and justify it by induction on size;
- solve count / collect / compare problems (second-min, root-to-leaf paths, parenthesised build) as one template with different combines.

2.1.1 Intuition

A company org chart seen from one manager

Ask any manager "how many people report to you, directly or indirectly?" They do not walk the whole company: they ask their two direct reports, get each one's number, add 1 for themselves, and reply. That single sentence – *trust your reports to answer about their own teams, then add yourself* – is exactly structural recursion on a tree.

A binary tree node holds a value and points to at most two smaller binary trees. That self-similarity is the whole game: any question about the tree at a node can be answered from the *same* question about its left and right subtrees, plus a tiny amount of work at the node itself.

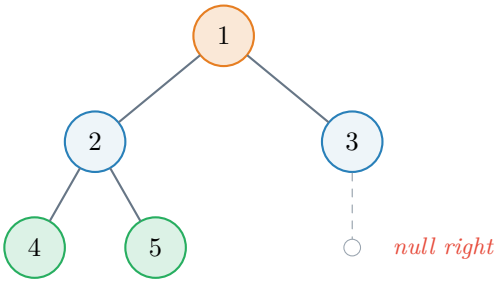


Figure 2.1.1: Node 1 has two children; node 3 has a left child missing (the dashed guide line ends at a small open null node). A `nullptr` is a real thing you must test for, not an afterthought.

2.1.2 Definitions and Notation

Definition 2.1.1 (The `TreeNode` model)

A binary tree is passed around as a pointer to its root. Each node is a `struct TreeNode { int val; TreeNode *left, *right; };` where `left` and `right` are pointers that are `null` (`nullptr`) when that child is absent. The tree is the pointer `root`; the empty tree is `nullptr`. Two children are *distinguished*: a single left child and a single right child are different trees (Definition 1.3.1).

Definition 2.1.2 (Recursive view of a binary tree)

A binary tree T is either empty (`null`) or a triple (val, L, R) where L and R are themselves binary trees (the left and right subtrees). This is the *inductive* definition: it says a tree is a node glued to two smaller trees, and it is why every property of T is proved by reasoning about L and R and the node.

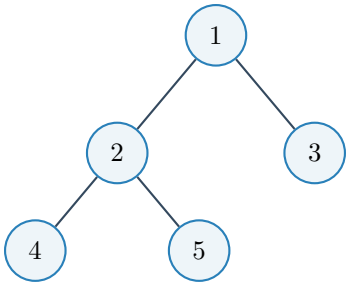


Figure 2.1.2: The same tree as Figure 2.1.1, drawn by `forest` from its recursive specification `[1 [2 [4] [5]] [3 [,phantom]]]`: read a bracket as "a node followed by its subtrees". The `[,phantom]` reserves the missing left slot of 3.

2.1.3 How Many Empty Links?

Before algorithms, one counting fact that explains why `null` handling is unavoidable.

Theorem 2.1.3 (Null Links in a Binary Tree)

A binary tree with n nodes has exactly $n-1$ non-`null` child pointers and $n+1$ `null` (`null`) child pointers.

Proof

Each of the n nodes owns two child-pointer slots, so there are $2n$ slots in total. Every node except the root is pointed to by exactly one non-`null` child pointer (a tree has one parent per non-root node and, by Theorem 1.1.5, no node is pointed at twice), so $n-1$ slots are filled. The remaining slots are null: $2n-(n-1)=n+1$.

■

Key Insight

There is always at least one more null pointer than there are nodes. That is precisely why "what does the code do when the child is `null`?" is the number-one source of wrong answers: your recursion meets $n+1$ nulls but only n real nodes, and each null is a base case you must define. Threaded and array representations (Chapter 4, Chapter 7) exist largely to recycle these $n+1$ dead pointers.

Leaf, null, and "missing one child"

A leaf
a real node `r` with `r->left == r->right == nullptr`; it has a value

A null child
the absent pointer `nullptr`; it has no value and is not a node

2.1.4 The Structural-Recursion Template

Strip away the problem and almost every solution has this skeleton: act on the base case, recurse on both subtrees, combine.

Code 2.1.1: The structural-recursion kit: count nodes, count leaves, and sum values, all with the same skeleton

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

// "Trust the recursion": each function only decides the node, given correct answers
// for the two subtrees. The null base case is what the n+1 nulls force us to define.
int countNodes(TreeNode* r) {                 // size: number of nodes
    if (r == nullptr) return 0;               // base case: empty tree contributes 0
    return 1 + countNodes(r->left) + countNodes(r->right);
}

int countLeaves(TreeNode* r) {                // leaves: nodes with two null children
```

```
    if (r == nullptr) return 0;
    if (r->left == nullptr && r->right == nullptr) return 1;   // r itself is a leaf
    return countLeaves(r->left) + countLeaves(r->right);
}

long long sumValues(TreeNode* r) {            // fold val over the whole tree
    if (r == nullptr) return 0;
    return r->val + sumValues(r->left) + sumValues(r->right);
}
```

Code 2.1.2: Building a small tree by linking pointers – the tree of Figures 2.1 and 2.2

```
TreeNode* buildExample() {
    TreeNode* root = new TreeNode(1);
    root->left  = new TreeNode(2);     // 2 becomes left child of 1
    root->right = new TreeNode(3);     // 3 becomes right child of 1
    root->left->left  = new TreeNode(4);
    root->left->right = new TreeNode(5);
    // node 3 keeps left = right = nullptr automatically (its null links)
    return root;
}
```

Complexity

All three functions are $O(n)$ time: the recursion tree of the traversal has one call per node plus the $n+1$ null base cases, so $2n+1$ calls total. Memory is $O(h)$ for the call stack – $O(\log n)$ when balanced but $O(n)$ when skewed; a path of 10^5 nodes is the stack-overflow warning of §1.2 again.

2.1.5 Worked Examples

Example 2.1.4 (All three counts on one tree)

For the tree of Figure 2.1.2 ($n = 5$): `countNodes` $= 5$; `countLeaves` $= 3$ (nodes $4, 5$ and 3, since 3's children are both null); `sumValues` $= 1+2+3+4+5 = 15$. The null-link lemma predicts $n+1 = 6$ nulls; check the picture: $4, 5, 3$ each contribute two nulls $= 6$.

Step-by-Step Trace

`countNodes` on a two-node left chain root→a returns $1+\text{countNodes}(a)+0 = 1+(1+0+0) = 2$: the leaf a answers 1 first, then the root adds itself – the aggregate rising from the leaves, the same flow as Theorem 1.2.4.

2.1.6 Common Pitfalls

Testing the child, not the pointer

Guard with `if (r == nullptr) return ...;` *before* touching `r->val`. Reading `r->left` on a null `r` is undefined behaviour (a crash), not a false condition. The base case comes first, always.

Confusing "leaf" with "null"

A leaf is a node; a null is the absence of one. `countLeaves` returns 1 only when *both children are null*, and must return 0 when *the node itself is null*. Swapping these two base cases is the classic one-line bug.

2.1.7 Solved Problems

W2.1.1 Second Minimum Node In a Binary Tree

Source: LeetCode 671 LC Easy
Techniques: structural recursion, min-combine, sentinel

Second Minimum Node

Every node here satisfies val = min of its two children, so the root holds the global minimum. Find the second-smallest *distinct* value in the tree, or return -1 if none exists.

➤ Understanding the Problem

The root is the smallest value; the second minimum is the smallest value strictly larger than root → val. Because a subtree rooted at r only ever holds values $\geq$ root → val, we can just scan every node and keep the minimum value that exceeds the root's – a one-value combine over the recursion of Code 2.1.1.

➤ Key Observations

Observation 1. Any node with val > root → val is a candidate for the second minimum, and no descendant can be smaller than it, so we can stop descending there.

Observation 2. The answer is the smallest candidate seen, or -1 if no node ever exceeds the root.

➤ Solution

Carry the fixed root value; recurse, updating a running best over nodes strictly greater than it and pruning below the first such node.

➤ Implementation

Code 2.1.3: Second Minimum Node (LeetCode 671)

```
class Solution {
    long long best;
    void dfs(TreeNode* r, int rootVal) {
        if (!r) return;
        if (r->val > rootVal) { best = min(best, (long long)r->val); return; }
        dfs(r->left, rootVal);      // r->val == rootVal, keep looking below
        dfs(r->right, rootVal);
    }
public:
    int findSecondMinimumValue(TreeNode* root) {
        best = LLONG_MAX;
        dfs(root, root->val);
        return best == LLONG_MAX ? -1 : (int)best;
    }
```

```
};
```

➤ Complexity

Complexity

Time $O(n)$ (pruning makes it faster on typical inputs; worst case all-equal visits every node). Memory $O(h)$ recursion stack. Using `long long`/`LLONG_MAX` for the sentinel avoids colliding with a legitimate value.

➤ Pitfalls and Variants

Sentinel that can equal a real value

Initialising the answer with 0 or `INT_MAX` breaks when those are valid node values. Use `LLONG_MAX` and translate to -1 only at the end.

W2.1.2 Binary Tree Paths

Source: LeetCode 257 LC Easy
Techniques: structural recursion, string build, collect at leaves

Binary Tree Paths

Return every root-to-leaf path as a string of the form `"1->2->3"`. A leaf is a node with no children; every root-to-leaf path must appear.

➤ Understanding the Problem

This is the recursive view directly: the paths through a node are its value glued onto each path of the left and right subtrees, and a leaf contributes exactly one finished path. The combine is string concatenation, and we collect only at leaves – the same "descend, then assemble" shape as Code 2.1.1, just returning a vector.

➤ Key Observations

Observation 1. A node's answer = its value prepended to (to) every path from its children.

Observation 2. A leaf's answer is the single string of its own value; a null's answer is the empty list.

➤ Implementation

Code 2.1.4: Binary Tree Paths (LeetCode 257)

```
class Solution {
public:
    vector<string> binaryTreePaths(TreeNode* r) {
        vector<string> out;
        if (!r) return out;
        if (!r->left && !r->right) { out.push_back(to_string(r->val)); return out; }
        for (auto& s : binaryTreePaths(r->left))  out.push_back(to_string(r->val) + "
    ↪ ->" + s);
        for (auto& s : binaryTreePaths(r->right)) out.push_back(to_string(r->val) + "
    ↪ ->" + s);
```

```
        return out;
    }
};
```

➤ Complexity

Complexity

Time $O(n \cdot h)$ in the worst case (each of the $O(n)$ path strings can be $O(h)$ long); the accumulate-a-string-in-one-DFS variant with backtracking is also $O(n)$ nodes visited. Memory $O(n \cdot h)$ to store all paths. Passing the running string by value (as above) copies it – a reference plus explicit backtracking saves the copies on deep trees.

W2.1.3 Construct String from Binary Tree

Source: LeetCode 606 LC Easy
Techniques: structural recursion, parentheses, empty-left rule

Construct String

Encode a tree as `value(left)(right)` with two rules: a leaf prints just its value; if the left child is null but the right is not, print `()` for the missing left, e.g. `"1(2)(())"` style is disallowed – you must keep the empty `()` when only a right child exists.

➤ Understanding the Problem

Another pure combine: `recurse left` and `recurse right` and splice the strings in with parentheses. The only subtlety is the empty-left exception – it exists precisely because without it a "left-only" and a "right-only" one-child node would encode identically, destroying the distinguished-children rule of Definition 1.3.1.

➤ Implementation

Code 2.1.5: Construct String from Binary Tree (LeetCode 606)

```
class Solution {
public:
    string tree2str(TreeNode* r) {
        if (!r) return "";
        string s = to_string(r->val);
        if (r->left || r->right) s += "(" + tree2str(r->left) + ")";     // left
    ↪ always shown when a child exists
        if (r->right)            s += "(" + tree2str(r->right) + ")";    // right only
    ↪  when present
        return s;
    }
};
```

➤ Complexity

Complexity

Time $O(n^2)$ worst case because each `+` copies a growing string on a skewed tree; $O(n)$ if you append into a single `string` buffer passed by reference. Memory $O(h)$ stack plus the output. String-building problems are exactly where append-in-place beats concatenate-and-return.

2.1.8 Practice Problems

Q2.1.1 Maximum Difference Between Node and Ancestor – LeetCode 1026 LC Medium

Hint: return the min and max value in each subtree bottom-up; update a global answer with the spread at every node.

Q2.1.2 Most Frequent Subtree Sum – LeetCode 508 LC Medium

Hint: post-order: a node's subtree sum is val + left-sum + right-sum; tally sums in a hash map.

Q2.1.3 Lowest Common Ancestor of Deepest Leaves – LeetCode 1123 LC Medium

Hint 1: compute each subtree height bottom-up; the LCA is where the two deepest sides tie.

Hint 2: return (height, lca) pairs; if left height > right, the lca bubbles up from the left, and so on.

2.2 Top-Down Recursion: Carrying State Down

In This Section

In a **top-down** recursion the answer at a node needs information that only its *ancestors* know – the depth so far, the running value of the path, a grandparent pointer – so we pass that state *in* as a function parameter and refine it on the way down. This is the mirror image of the height recursion of §1.2 (which pulled values *up*); here the values flow from the root to the leaves.

Prerequisites. §2.1 (the structural-recursion skeleton and the `null` base case) and Definition 1.1.2 (ancestor, parent, depth).

After this section you can

- spot a top-down problem: the answer depends on the path *above* the node;
- thread an accumulator, a depth, or ancestor pointers through the recursion;
- backtrack a shared path buffer correctly (push before, pop after) so siblings do not contaminate each other;
- solve root-to-leaf numbers, all paths with a target sum, and the even-grandparent sum in one clean DFS.

Where this is used. Chapter 11 (LCA via parent pointers), Chapter 12 (binary lifting carries an ancestor table down) and Chapter 16 (HLD passes a head node in) – almost every "path from the root" or "as a function of depth" query is a top-down DFS; getting the parameter-passing habit now pays off in all of them.

2.2.1 Intuition

Hiking with a running logbook

Descending a trail from the summit, each camp writes down how many hours it has been since the top and the list of camps already passed. It does not have to re-count from the bottom: the number simply arrives from the camp above, gets incremented, and is passed on. That incremented logbook is the carried state of a top-down DFS.

Consider turning root-to-leaf digits into numbers. When you stand at a node you already know the value of everything above you (the partial number); you push one more digit down and add up at the leaves. The information clearly flows downward.

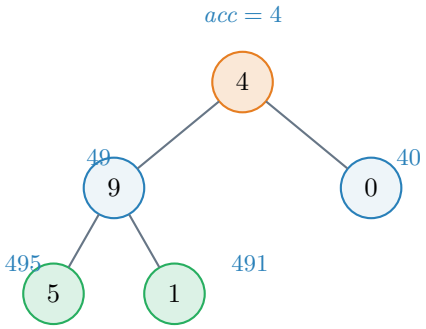


Figure 2.2.1: The accumulator acc = acc × 10 + val flows down. Leaves read 495, 491, 40; the answer is their sum 1026. Nothing flows back up.

2.2.2 Definition

Definition 2.2.1 (Top-down recursion)

A recursion $f(\text{node}, \text{state})$ where *state* is derived from the ancestors and passed *into* the call. The node computes (or records) its answer from state $\cup$ {node}, then calls its children with an updated state. It is correct when the state at a node is exactly the information the path from the root to that node provides; when state is mutated in place, it must be **restored** before the second child so siblings see the same input.

2.2.3 The Template

Code 2.2.1: General top-down skeleton: refine the carried state, recurse, and undo any in-place change

```
// State is whatever the path above contributes: depth, an accumulator, a vector, ancestors.
void topDown(TreeNode* r, State& s) {
    if (r == nullptr) return;              // base case: the n+1 null links (Lemma 2.1)
    record(s, r);                          // answer this node using the state that arrived
    s.extend(r);                           // refine state for the children (e.g. s.val += ...)
    topDown(r->left,  s);
    topDown(r->right, s);
    s.undo(r);                             // back-track: leave s as you found it for siblings
}
```

Top-down vs bottom-up

Top-down (this section)
state flows *in* as a parameter; answer often recorded at leaves or per node

Bottom-up (§2.3)
answer flows *out* as a return value; combine children to build the parent's answer

Complexity

A top-down DFS visits all n nodes once – $O(n)$ – with $O(h)$ call-stack depth plus the size of the carried state. If the state is a path *vector copied at every node*, you pay $O(h)$ per copy and become $O(nh)$; mutate-and-restore keeps it $O(n)$ with one shared buffer.

2.2.4 Worked Examples

Example 2.2.2 (Tracing the accumulator on Figure 2.2.1)

At the root acc $= 4$. Left child 9: acc $= 4 \cdot 10 + 9 = 49$. Its children $5, 1$ give $495, 491$ (both leaves, so both are added). Right child 0: acc $= 40$ and it is itself a leaf. Sum $= 495 + 491 + 40 = 1026$.

Step-by-Step Trace

For Path Sum II, the shared path buffer evolves as $[4] \to [4,9] \to [4,9,5]$ (leaf: test) $\to [4,9] \to [4,9,1] \to [4] \to [4,0]$ – the `pop_back` after each child is what stops node 1 from seeing path $[4,9,5,1]$.

2.2.5 Common Pitfalls

Forgetting to undo shared state

If you push onto a shared `path` before the left child but forget to pop before recursing right, the right subtree inherits the left subtree's nodes. Every mutate-then-recurse needs a matching recurse-then-restore.

Recomputing what you could carry

Passing the depth as a parameter is $O(1)$ per node; re-walking from the root to find the depth at every node is $O(h)$ each and turns an $O(n)$ DFS into $O(nh)$. If the value is already known on the way in, *carry* it.

2.2.6 Solved Problems

W2.2.1 Sum Root to Leaf Numbers

Source: LeetCode 129 LC Medium
Techniques: top-down accumulator, root-to-leaf

Sum Root to Leaf Numbers

Each root-to-leaf path spells a decimal number by its node values (e.g. path $4 \to 9 \to 5$ is 495). Return the sum of all such numbers. Leaves are nodes with no children.

➤ Understanding the Problem

The number for a path is built digit by digit from the root, so at a node we hold the partial value acc and pass $10 \cdot \text{acc} + \text{val}$ downward – the flow of Figure 2.2.1. At a leaf the partial value is a complete number, so it is added to the total. This is a textbook top-down recursion (Definition 2.2.1).

➤ Key Observations

Observation 1. $\text{acc}_{\text{child}} = 10 \cdot \text{acc}_{\text{parent}} + \text{child->val}$, with $\text{acc}_{\text{root}} = \text{root->val}$.

Observation 2. Only leaves contribute; an internal node's partial number is never added.

➤ Implementation

Code 2.2.2: Sum Root to Leaf Numbers (LeetCode 129)

```
class Solution {
    int dfs(TreeNode* r, int acc) {
        if (!r) return 0;
        acc = acc * 10 + r->val;                // carry refined value down
        if (!r->left && !r->right) return acc;   // leaf: complete number
        return dfs(r->left, acc) + dfs(r->right, acc);
    }
public:
    int sumNumbers(TreeNode* root) { return dfs(root, 0); }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$. Because the accumulator is passed by *value*, no back-tracking is needed – each child gets its own copy, which is the cheapest way to avoid the shared-state pitfall when the state is a single integer.

W2.2.2 Path Sum II

Source: LeetCode 113 LC Medium
Techniques: top-down, shared path buffer, backtracking

Path Sum II

Given a target sum, return every root-to-leaf path whose node values add to it. Each path is a list of values; a leaf ends a path.

➤ Understanding the Problem

Unlike W1.1.1, we must *output* the paths, not just decide existence, so we carry the current path down. Carry the remaining budget too: subtract val as we descend and a leaf with remaining val is a hit. The path is shared state, so it must be restored (a push/pop DFS exactly as in Figure 2.2.1).

➤ Key Observations

Observation 1. A leaf is on a valid path iff the target minus all strict ancestors equals its own value.

Observation 2. Record a *copy* of the path at the leaf; the live buffer keeps changing.

➤ Implementation

Code 2.2.3: Path Sum II (LeetCode 113)

```
class Solution {
    vector<vector<int>> ans;
    vector<int> path;
    void dfs(TreeNode* r, int rem) {
        if (!r) return;
        path.push_back(r->val);                          // extend state
        rem -= r->val;
        if (!r->left && !r->right && rem == 0) ans.push_back(path);  // leaf hit
        dfs(r->left, rem);
        dfs(r->right, rem);
        path.pop_back();                                 // restore for siblings
    }
public:
    vector<vector<int>> pathSum(TreeNode* root, int target) {
        dfs(root, target);
        return ans;
    }
};
```

➤ Complexity

Complexity

Time $O(n)$ to visit nodes plus $O(h)$ to copy each found path, worst case $O(n \cdot h)$ when many paths qualify. Memory $O(h)$ for the live path and stack (the answer itself can hold $O(n \cdot h)$ values). The `pop_back` is exactly the "undo shared state" rule – without it the second child sees the first child's node.

W2.2.3 Sum of Nodes with Even-Valued Grandparent

Source: LeetCode 1315 LC Medium
Techniques: top-down, carrying ancestor pointers

Even-Valued Grandparent Sum

Return the sum of values of nodes whose *grandparent* (two levels up) has an even value. If no such node exists the sum is 0.

➤ Understanding the Problem

The predicate about a node depends on its grandparent – pure ancestor information, so we carry grandparent and parent pointers down (Definition 2.2.1). At each node, if the carried grandparent has even value, add the node's value.

➤ Implementation

Code 2.2.4: Sum of Nodes with Even-Valued Grandparent (LeetCode 1315)

```
class Solution {
    int dfs(TreeNode* r, TreeNode* parent, TreeNode* grand) {
        if (!r) return 0;
        int total = (grand && grand->val % 2 == 0) ? r->val : 0;   // decide at this
    ↪ node
        total += dfs(r->left,  r, parent);   // this node becomes parent, old parent
    ↪ becomes grand
        total += dfs(r->right, r, parent);
        return total;
    }
public:
    int sumEvenGrandparent(TreeNode* root) { return dfs(root, nullptr, nullptr); }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$. Carrying `parent`/`grand` as parameters beats storing them in a map: two pointers per call, updated for free on the way down. (A bottom-up variant returning grandchild sums also works – see §2.3.)

2.2.7 Practice Problems

Q2.2.1 Cousins in Binary Tree – LeetCode 993 LC Easy
Hint: carry (depth, parent) down; two nodes are cousins iff equal depth but different parents.

Q2.2.2 Count Good Nodes in Binary Tree – LeetCode 1448 LC Medium
Hint: a node is good if no ancestor exceeds it; carry the max-so-far down and compare.

Q2.2.3 Deepest Leaves Sum – LeetCode 1302 LC Medium
Hint: carry depth down and keep the current deepest level; add values when you reach a new max depth.

Q2.2.4 All Nodes Distance K in Binary Tree – LeetCode 863 LC Medium

Hint 1: you must go up as well as down – build parent pointers first, then BFS from the target.
Hint 2: treat the tree as an undirected graph via a parent map; a two-hop BFS reaches both descendants and ancestors within K.

2.3 Bottom-Up Recursion: Returning Values Up

In This Section

In a **bottom-up** recursion each node asks its children for an answer, combines the two, and *returns* the result to its own parent – the return value is a subtree aggregate. The height and size recurrences of Theorem 1.2.4 were already bottom-up; here we generalise them and add the one idiom that unlocks the hardest tree mediums: a function that *returns* one quantity (so the parent can use it) while *silently updating a global best* (the real answer). That is exactly how diameter, tilt and longest-path problems collapse into a single DFS.

Prerequisites. §2.1 (structural recursion) and Theorem 1.2.4 (return-and-combine).

After this section you can

- tell a bottom-up problem instantly: the answer at a node is a function of its two subtrees alone;
- return a subtree aggregate (sum, height, pruned pointer, chain length) and combine it correctly;
- use the "return one value, update a second global answer" split for diameter/tilt/longest-path;
- solve binary tree tilt, prune tree, and longest univalue path in one pass each.

Where this is used. Chapter 9 (diameter and centroid), Chapter 10 (tree DP) and Chapter 20 (advanced tree DP) – tree DP *is* bottom-up recursion with a richer returned state; mastering the return-plus-global split now is literally the first lesson of every DP-on-trees chapter.

2.3.1 Intuition

An org chart reporting up

A VP does not decide the company total herself: she waits for both directors under her to report their numbers, adds them to her own, and passes one total up to the CEO. The value travels from the bottom (leaves) to the top (root) – the opposite direction to §2.2. Sometimes, though, a manager also shouts "biggest team so far!" to a whiteboard on the wall: that side note is the global-best update made during the upward pass.

Take subtree sums. To know the sum at a node you must already know the sums of both children, so the information is forced to flow upward. We draw the returns on the tree.

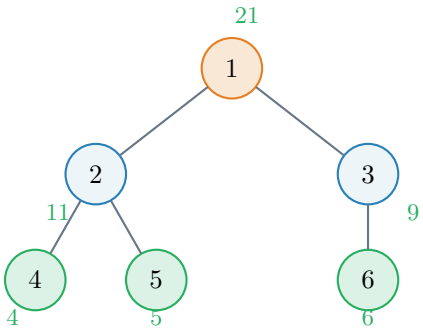


Figure 2.3.1: Subtree sums returned upward (green): leaves return their own value, node 2 returns 2+4+5 = 11, node 3 returns 3+6 = 9, the root returns 1+11+9 = 21. The parent cannot be computed before the children – the definition of bottom-up.

2.3.2 Definition and the Aggregate Property

Definition 2.3.1 (Bottom-up recursion)

A recursion whose *return value* is a function of the return values of the children plus the node itself: $g(v) = \phi(v,\ g(v_{\text{left}}),\ g(v_{\text{right}}))$ with a base value at `null`. The answer to the whole problem is either $g(\text{root})$ or a quantity recorded while computing the g's.

Proposition 2.3.2 (Bottom-up well-definedness)

Because a tree has no cycles (Definition 1.1.1) every recursion path ends at a `null` base case, so the combination ϕ is applied over a finite, acyclic call graph and $g(v)$ depends only on the strict subtree subtree(v) – the aggregate is always defined.

Proof

subtree(v) is finite and, excluding v, consists of the strict subtrees of v's children, which are strictly smaller. Induction on size(v) (Theorem 1.2.4 is exactly this for height): the children's g are defined by the hypothesis (smaller subtrees), and ϕ combines finitely many of them with v. The `null` base terminates the recursion.

■

2.3.3 The Template and the Global-Best Split

Code 2.3.1: Bottom-up skeleton: return the aggregate the parent needs, and separately update the answer that lives "through" the current node

```
// g(r) = the aggregate a parent can reuse (height, sum, chain length, pruned pointer).
// best = the real answer, which may pass THROUGH r and so cannot be a single-child return.
Aggregate dfs(TreeNode* r, Best& best) {
    if (r == nullptr) return IDENTITY;        // 0 for sums, 0/-1 for heights, ...
    Aggregate L = dfs(r->left,  best);
    Aggregate R = dfs(r->right, best);
    best = std::max(best, combine(L, R, r));  // answer using BOTH sides at r
    return fold(L, R, r);                     // single aggregate sent UP
}
```

Why the split is necessary

A recursion returns *one* value, but the best path/tilt/answer at r may use both children at once (a path bending through r). If you tried to return that, the parent would double-count. So you return the extendable part (one side) and record the through-part in a global – the exact move behind diameter (Chapter 9) and W2.3.3 below.

Complexity

One DFS, $O(n)$ time, $O(h)$ stack. The whole point of bottom-up is that each child's aggregate is computed once and reused by the parent, so an $O(n)$-per-node naive recomputation (like the $O(n^2)$ balanced-check warned about in W1.3.1) drops to $O(n)$.

2.3.4 Worked Examples

Example 2.3.3 (Reading returns off Figure 2.3.1)

The returned sums are $4, 5$ at 2's children, so node 2 returns 11; 3 returns 9; the root returns 21. If instead we wanted the max subtree sum, we would update a global with each return and read $\max(4, 5, 11, 6, 9, 21) = 21$.

Step-by-Step Trace

`dfs` for tilt visits $4, 5$ then 2: at 2 the tilt is $|5 - 4| = 1$, returns sum 11; at 3 tilt $|6 - 0| = 6$, returns 9; at the root tilt $|9 - 11| = 2$. Total tilt $1 + 6 + 2 = 9$.

2.3.5 Common Pitfalls

Returning the answer instead of the aggregate

For path-through-node problems, returning combine(L, R, r) (the through value) poisoning the parent is the most common error. Return only what a parent can extend – the better of the two children – and keep the through value in the global.

Wrong identity for the null base case

Sums return 0, node-heights return 0, edge-heights return -1, "contains a 1" returns `nullptr`/`false`. Using -1 where 0 is the identity makes every subtree off by one – the same off-by-one flagged in §1.2.

2.3.6 Solved Problems

W2.3.1 Binary Tree Tilt

Source: LeetCode 563 LC Easy
Techniques: bottom-up subtree sum, global accumulator

Binary Tree Tilt

The tilt of a node is $|\text{sum}(\text{left}) - \text{sum}(\text{right})|$. Return the sum of the tilts of all nodes.

➤ Understanding the Problem

Tilt needs each subtree's *sum*, which is a clean bottom-up aggregate; the total tilt is a value living at each node, so we return the sum and add the tilt into a global – exactly the split in Code 2.3.1.

➤ Implementation

Code 2.3.2: Binary Tree Tilt (LeetCode 563)

```
class Solution {
    int total = 0;
    int sum(TreeNode* r) {
        if (!r) return 0;                       // null contributes sum 0
        int L = sum(r->left), R = sum(r->right);
        total += abs(L - R);                    // tilt recorded on the way up
        return r->val + L + R;                  // return the subtree sum
    }
public:
    int findTilt(TreeNode* root) { sum(root); return total; }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$. Note the two distinct roles: `sum` returns the subtree total, `total` accumulates the answer – folding them into one return would lose the per-node tilt.

W2.3.2 Binary Tree Pruning

Source: LeetCode 814 LC Medium
Techniques: bottom-up decision, return subtree pointer

Binary Tree Pruning

The tree holds only 0 and 1. Remove every subtree that does not contain a 1, and return the root of the pruned tree.

➤ Understanding the Problem

Whether to keep a node depends only on its subtree: keep r iff r`->val`$= 1$ or one of its (already-pruned) children survived. That is a bottom-up predicate, so we recurse first, then decide, returning `nullptr` to delete a whole branch.

➤ Key Observations

Observation 1. After pruning the children, r must go iff val $= 0$ *and* both children are now `null`.

Observation 2. A 1 always survives even if both children are pruned away.

➤ Implementation

Code 2.3.3: Binary Tree Pruning (LeetCode 814)

```
class Solution {
public:
    TreeNode* pruneTree(TreeNode* r) {
        if (!r) return nullptr;
        r->left  = pruneTree(r->left);    // prune below first
        r->right = pruneTree(r->right);
        if (r->val == 0 && !r->left && !r->right) return nullptr;  // dead branch
        return r;
    }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$. The returned pointer doubles as the aggregate (survives or not) and the rebuilt child link – a bottom-up recursion that mutates the tree in place.

W2.3.3 Longest Univalue Path

Source: LeetCode 687 LC Easy
Techniques: bottom-up chain length, global best, path-through-node

Longest Univalue Path

Return the number of *edges* on the longest path whose nodes all share one value. The path need not pass the root and may bend at any node.

➤ Understanding the Problem

Mirror the diameter idea (W1.3.1's cousin in §1.2): `dfs` returns the longest univalue *chain* going straight down from r (extendable by r's parent), while the global answer takes the best path *through* r, which can join the left and right chains when they match r`->val`. This is the through-vs-return split of Code 2.3.1 verbatim.

➤ Implementation

Code 2.3.4: Longest Univalue Path (LeetCode 687)

```
class Solution {
    int best = 0;
    int chain(TreeNode* r) {
        if (!r) return 0;
        int L = chain(r->left), R = chain(r->right);
        int leftExt = 0, rightExt = 0;
```

```
        if (r->left  && r->left->val  == r->val) leftExt  = L + 1;   // extendable
    ↪ chains
        if (r->right && r->right->val == r->val) rightExt = R + 1;
        best = std::max(best, leftExt + rightExt);                   // path through r
        return std::max(leftExt, rightExt);                          // return one
    ↪ side up
    }
public:
    int longestUnivaluePath(TreeNode* root) { chain(root); return best; }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$. Returning $\max(\text{leftExt}, \text{rightExt})$ but scoring $\text{leftExt} + \text{rightExt}$ in the global is the whole trick – returning the sum would let the parent count a path that bends twice.

➤ Pitfalls and Variants

Counting edges, not nodes

The answer is edges; a chain of k matching nodes scores $k - 1$. Adding 1 only when we *extend* to a child (as above) keeps the count in edges automatically.

Variant: *Diameter of Binary Tree* is the same code with lengths in place of value matching; the general tree version is Chapter 9.

2.3.7 Practice Problems

Q2.3.1 Distribute Coins in Binary Tree – LeetCode 979 LC Medium

Hint: bottom-up: return the net surplus/deficit a subtree must move through its parent edge; add its absolute value to the answer.

Q2.3.2 House Robber III – LeetCode 337 LC Medium

Hint: return a pair (rob this node, skip this node) per subtree; combine the two children's pairs.

Q2.3.3 Delete Nodes And Return Forest – LeetCode 1110 LC Medium

Hint: post-order; when a child is deleted, its subtree roots become new trees – collect them.

Q2.3.4 Path Sum III – LeetCode 437 LC Medium

Hint 1: count downward paths (any start/end) summing to target; brute force is two DFSs.

Hint 2: prefix sums on the root-to-node path: carry a hash map of running-sum counts down, and for each node add how many earlier prefixes equal current−target. A top-down map plus a bottom-up count.

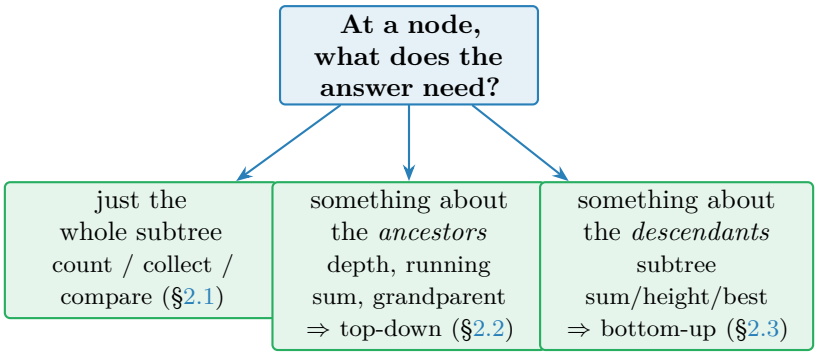
Chapter 2 Summary

This chapter reduced every binary-tree solution to one recursion and sorted the hard cases by the *direction information flows*. The tables collect the results and the exact condition each needs.

Result	Statement (with conditions)	Where
Null links	A binary tree on n nodes has $n-1$ non-`null` child pointers and $n+1$ `null` ones.	Theorem 2.1.3
Bottom-up definedness	With no cycles, the aggregate $g(v)$ depends only on subtree(v) and always terminates.	Proposition 2.3.2
Top-down flow	Answer at v needs ancestor state (depth, running value, grandparent), passed *in*.	Definition 2.2.1
Bottom-up flow	Answer at v is a function of the two children's return values, combined and *returned*.	Definition 2.3.1
Global-best split	Return the extendable one-side value; record the through-v value in a global.	Code 2.3.1

Algorithms from this chapter.

Algorithm	Time	Memory	Code
Structural recursion (count / leaves / sum)	$O(n)$	$O(h)$	Code 2.1.1
Build a tree by linking pointers	$O(n)$	$O(n)$	Code 2.1.2
Top-down DFS (accumulator / path)	$O(n)$ (or $O(nh)$ copying)	$O(h)$	Code 2.2.1
Bottom-up DFS (aggregate + global best)	$O(n)$	$O(h)$	Code 2.3.1



Choosing the recursion by data direction. If a node's answer needs the path above it, carry state down (2.2); if it needs the subtrees below, return an aggregate up (2.3); if it just needs to sweep the whole tree, plain structural recursion (2.1) suffices.

Common Mistakes Checklist

- **No `null` base case.** You meet $n+1$ nulls (Theorem 2.1.3); dereferencing one crashes. Fix: test `r == nullptr` first, every function.
- **Leaf vs null confused.** A leaf is a real node with two null children; a null is the absence of a node. Fix: give each its own return value.
- **Shared state not restored.** A pushed path or incremented counter leaks to the next sibling. Fix: undo after recursing, or pass primitives by value (W2.2.1).
- **Returning the through-node answer.** Returning the bend-at-v value makes the parent double-count. Fix: return one side, keep the through value in a global.
- **Wrong identity at null.** Sums want 0, edge-heights want -1, prune wants `nullptr`. Fix: pick the identity from the combine ϕ, then test a one-node tree.

- **Edges vs nodes again.** Univalue/diameter count edges; depth counts nodes. Same off-by-one as §1.2.
- **String / vector concatenation by value.** W2.1.2 and W2.1.3 become $O(nh)$ or $O(n^2)$. Fix: append into one shared buffer.

Quick Reference Card

Chapter 2 at a glance

The node. `TreeNode{val,left,right}`; empty tree = `nullptr`; two distinguished children. n nodes $\Rightarrow n-1$ live links, $n+1$ null links.

Structural recursion (2.1). Base at `null`; combine two correct subtree answers with the node. Count/leaves/sum/paths/serialise all share this one skeleton.

Top-down (2.2). $f(v, \text{state})$; state is ancestor info (depth, accumulator, path, grandparent) refined on the way *in*; restore shared state between siblings.

Bottom-up (2.3). Return $g(v) = \phi(v, g_L, g_R)$ *out*; when the answer passes through v but the parent needs one side, *return* the side, *record* the through value in a global (tilt, univalue path, diameter).

Code 2.3.5: Chapter 2 code kit: the two recursion directions side by side

```cpp
#include <bits/stdc++.h>
using namespace std;
struct TreeNode { int val; TreeNode *left, *right;
                  TreeNode(int x=0): val(x), left(nullptr), right(nullptr){} };

// TOP-DOWN: carry a value down; record at leaves. (Sum Root to Leaf Numbers, LC 129)
int topDown(TreeNode* r, int acc) {
    if (!r) return 0;
    acc = acc * 10 + r->val;                          // refine state for children
    if (!r->left && !r->right) return acc;            // leaf: emit
    return topDown(r->left, acc) + topDown(r->right, acc);
}

// BOTTOM-UP: return an aggregate, update a global best. (Longest Univalue Path, LC 687)
int best = 0;
int chain(TreeNode* r) {
    if (!r) return 0;
    int L = chain(r->left), R = chain(r->right);
    int le = (r->left  && r->left->val  == r->val) ? L + 1 : 0;   // extendable
    int re = (r->right && r->right->val == r->val) ? R + 1 : 0;
    best = max(best, le + re);                        // answer THROUGH r
    return max(le, re);                               // return ONE side up
}
```

Mixed Problem Set

These blend the chapter's ideas and never say which recursion direction to use; decide it yourself from what each node's answer depends on.

M2.1 Find Bottom Left Tree Value – LeetCode 513 LC Medium

Hint: carry depth down and keep the first node seen at each new maximum depth.

M2.2 Add One Row to Tree – LeetCode 623 LC Medium

Hint: top-down to depth $d-1$, then splice in two new children; special-case $d=1$.

M2.3 Find Duplicate Subtrees – LeetCode 652 LC Medium

Hint: bottom-up: serialise each subtree into a string, hash it, and report hashes seen twice.

M2.4 Flatten Binary Tree to Linked List – LeetCode 114 LC Medium

Hint: pre-order, but rewire right pointers; the in-place trick splices the old right subtree.

M2.5 Flip Equivalent Binary Trees – LeetCode 951 LC Medium

Hint: same or flipped: recurse comparing (a,b) and (b,a) child pairs at every node.

M2.6 Binary Tree Cameras – LeetCode 968 LC Hard

Hint 1: Stretch: greedy bottom-up – return each node's coverage state to its parent.

Hint 2: three states (has camera / covered / needs coverage); decide a camera when a child needs coverage (Chapter 10 reroot-free DP).

Related Topics

- Chapter 1 – the tree vocabulary and the height/size recursion this chapter generalises.
- Chapter 3 – preorder/inorder/postorder/level-order: the walks these recursions embody.
- Chapter 4 – array, adjacency-list and Euler-tour storage; how to *hold* the tree.
- Chapter 5 – binary search trees, where inorder order adds new bottom-up tricks.
- Chapter 9 – diameter and centroid: the return-plus-global idiom on general trees.
- Chapter 10 – tree DP formalises the bottom-up aggregate with richer returned states.
- Chapter 11 – LCA: carrying parent/depth information down, the ancestor tool teased here.

3

Chapter

Tree Traversals

Level: LC Easy–Hard • **Problems:** 29 (11 fully solved) • **Status:** Written

Chapter at a Glance

A tree stores its nodes in *space*, not in a line. To use a tree you first have to **traverse** it: walk over every node, *visiting* each one exactly once in some order. "Visit" just means *do your work on the node* – print its value, add it to a sum, check a condition. The only real choice is **when** you visit a node relative to its subtrees, and that one choice gives the four orders this chapter is about:

- **preorder**: visit the node *before* its two subtrees;
- **inorder**: visit it *between* the left subtree and the right subtree;
- **postorder**: visit it *after* both subtrees are done;
- **level order**: forget subtrees – go level by level, top to bottom, using a queue instead of recursion.

You will learn to

- produce all four orders by hand on any small tree, and know *why* each order is the natural choice for a different job (copy = preorder, delete = postorder, level-by-level = BFS);
- write each order recursively in three lines, then again with an explicit stack, and again with Morris threading in $O(1)$ extra memory;
- split a level-order walk into one vector per level – the template behind LeetCode 102/103/107/199/637;
- rebuild a whole tree from two of its traversal lists, and flatten a tree to a string and back (serialise/deserialise).

Prerequisites. Chapter 2 – the `TreeNode` struct (Definition 2.1.1) and the structural-recursion idea (§2.1). We also reuse the null-link count of Theorem 2.1.3 when we prove Morris traversal is linear.

In contests and interviews. Traversal questions are the most-asked tree family on LeetCode. The recursive answers are short; the follow-ups – "do it iteratively", "do it in $O(1)$ space", "rebuild the tree from these two lists" – are what separate a passing from a strong interview, and §3.2 and §3.4 train exactly those.

The Tree We Will Use All Chapter

Every trace in this chapter runs on the *same* six-node tree, so you can always come back to one picture instead of learning a new shape per section:

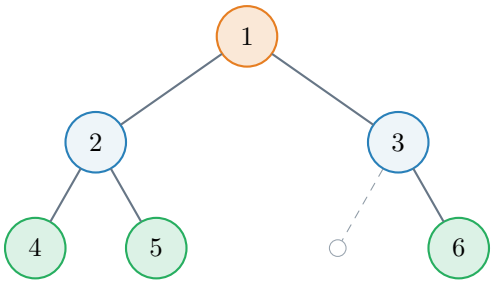
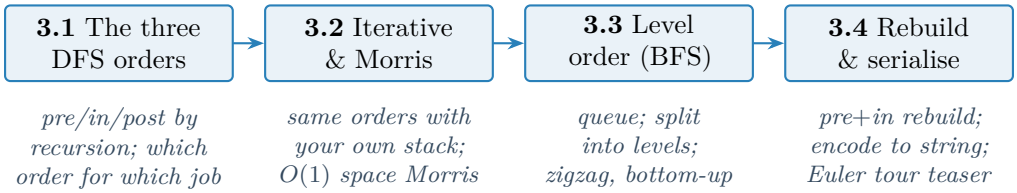


Figure 3.0.1: The running example T. Node 3 has *no left child* – deliberately kept, because one-child nodes are exactly where traversal mistakes show up. Its four orders: preorder $1, 2, 4, 5, 3, 6$; inorder $4, 2, 5, 1, 3, 6$; postorder $4, 5, 2, 6, 3, 1$; level order $1, 2, 3, 4, 5, 6$. We derive every one of these, step by step, in the chapter.



The four topics of this chapter. 3.1 defines the orders, 3.2 re-implements them without recursion, 3.3 adds the fourth (queue-based) order, and 3.4 turns the whole idea around: from orders back to trees.

Topic	Key results	Solved problems
§3.1 The three depth-first orders	recursive definitions; root position in each order	W3.1.1, W3.1.2, W3.1.3
§3.2 Iterative traversals & Morris	explicit-stack simulations; $O(1)$-space inorder	W3.2.1, W3.2.2
§3.3 Breadth-first / level order	queue with level splits; BFS assigns depths	W3.3.1, W3.3.2
§3.4 Rebuilding and serialising	pre + in $\Rightarrow$ unique tree; string round-trip	W3.4.1, W3.4.2, W3.4.3

Notation Used in This Chapter

Symbol	Meaning	Symbol	Meaning
$\mathrm{pre}(T)$	preorder list of T	$\mathrm{in}(T)$	inorder list of T
$\mathrm{post}(T)$	postorder list of T	$\mathrm{lev}(T)$	level-order list of T
S	the explicit stack (top at left)	Q	the FIFO queue (front at left)
$\mathrm{tin}(v)$	entry time of v in a DFS walk	$\mathrm{tout}(v)$	exit time of v
$v \cdot X$	value of v followed by list X	#	null marker in strings

Throughout, *left* and *right* are the distinguished children of Definition 1.3.1, heights stay in edges as in Chapter 1, and a **visit** never means "print" specifically – it means whatever the problem does with a node.

3.1 The Three Depth-First Orders

In This Section

This section defines the three classic *depth-first* orders – preorder, inorder, postorder – computes all three by hand on the running tree of Figure 3.0.1, and writes the three-line recursive code for each. By the end you can answer the question every traversal interview starts with: *"which order should I use, and why?"*

Prerequisites. §2.1 (the `TreeNode` struct and recursion on subtrees). Nothing else – this section starts traversal from zero.

After this section you can

- state the three orders and produce them by hand on any small tree, with a trace;
- write the recursive function for each order without looking anything up;
- say where the root sits in each order and use that to spot wrong answers;
- pick the right order for a job: copy → preorder, delete/evaluate → postorder.

3.1.1 What "Visit" Means, and the Three Orders

Definition 3.1.1 (Tree traversal (visit orders))

A **traversal** of a tree is a walk that *visits every node exactly once*. Visiting a node means performing whatever the problem asks at that node – printing its value, adding it into a sum, testing a property. A traversal is fully described by the **order** in which nodes are visited; two traversals that differ only in that order are different tools for different jobs.

Every depth-first traversal answers one question: *when do I visit the node itself, relative to its left subtree and its right subtree?* There are exactly three sensible answers, and they give the three classic orders.

Definition 3.1.2 (Preorder, inorder, postorder)

For a binary tree T with root value v, left subtree L and right subtree R:

- **preorder**: $\mathrm{pre}(T) = v \cdot \mathrm{pre}(L) \cdot \mathrm{pre}(R)$ – *node, then left, then right*;
- **inorder**: $\mathrm{in}(T) = \mathrm{in}(L) \cdot v \cdot \mathrm{in}(R)$ – *left, then node, then right*;
- **postorder**: $\mathrm{post}(T) = \mathrm{post}(L) \cdot \mathrm{post}(R) \cdot v$ – *left, then right, then node.*

Here $x \cdot X$ means "write x, then the list X". For the empty tree every list is empty – that is the base case of all three recurrences.

The names are the whole definition

Pre-order puts the node *before* its subtrees, **post**-order puts it *after* them, and **in**-order puts it *in between* the left and the right. If you can remember the names, you can rebuild the definitions. (Inorder only makes sense for trees with an ordered left/right – binary trees; general trees have preorder and postorder but no inorder.)

3.1.2 One Node's Lifecycle: the Three Moments

Definition 3.1.3 (The three moments of a node)

Imagine a depth-first walk that arrives at a node v, finishes the whole left subtree, finishes the whole right subtree, and then leaves v forever. The walk passes v at three moments: on *arrival* (before either subtree), on *turnaround* (between the two subtrees), and on *departure* (after both). Writing v at the arrival, turnaround, or departure moment gives exactly preorder, inorder, or postorder.

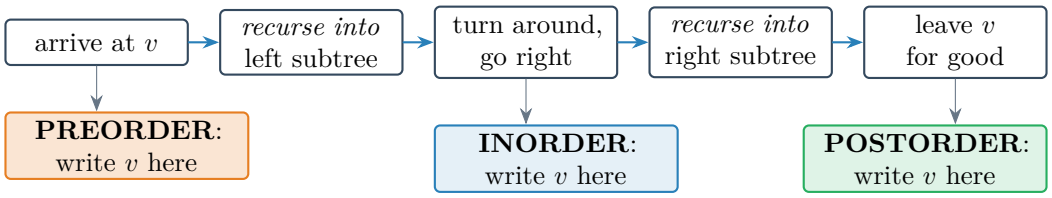


Figure 3.1.1: One node's lifecycle during a depth-first walk. The three orders differ *only* in which of the three moments they write the node down – everything else about the walk is identical. This single picture explains all of §3.1.

3.1.3 Computing All Three Orders by Hand

Example 3.1.4 (All three orders on the running tree)

Take T from Figure 3.0.1: root 1, left subtree rooted at 2 (children 4, 5), right subtree rooted at 3 (no left child, right child 6). Expand each recurrence subtree by subtree – read each line as "the node glued between its already-computed subtree lists":

$$\begin{aligned}\mathrm{pre}(T) &= 1 \cdot \mathrm{pre}(2) \cdot \mathrm{pre}(3) = 1 \cdot [\,2 \cdot 4 \cdot 5\,] \cdot [\,3 \cdot \varnothing \cdot 6\,] = \mathbf{1,2,4,5,3,6},\\ \mathrm{in}(T) &= \mathrm{in}(2) \cdot 1 \cdot \mathrm{in}(3) = [\,4 \cdot 2 \cdot 5\,] \cdot 1 \cdot [\,\varnothing \cdot 3 \cdot 6\,] = \mathbf{4,2,5,1,3,6},\\ \mathrm{post}(T) &= \mathrm{post}(2) \cdot \mathrm{post}(3) \cdot 1 = [\,4 \cdot 5 \cdot 2\,] \cdot [\,\varnothing \cdot 6 \cdot 3\,] \cdot 1 = \mathbf{4,5,2,6,3,1}.\end{aligned}$$

Notice the right subtree of node 3: writing 3 *before* its right child in pre and *after* its left child in in is exactly where the missing child changes nothing – an empty list contributes nothing, it does not shift anything. If you got $\mathrm{in}(T) = 4,2,5,1,6,3$ you placed 6 on the wrong side of 3: 6 is a *right* child, so it comes *after* 3 in inorder.

The recursive calls in time order (preorder)

Following the actual call sequence – indent shows call depth – the first DFS call unwinds like this. "**w**" marks the moment the node is written:

```
pre(1):  w1  pre(2):  w2  pre(4):  w4, left=null, right=null, return
      back at 2 → pre(5):  w5, return   2 done
            back at 1 → pre(3):  w3, left=null → pre(6):  w6, return
output stream:  1 2 4 5 3 6   (calls made:  6 nodes + 7 nulls = 13)
```

The output appears exactly in arrival order – node values are written on the way *down*, which is why preorder is also called **depth-first entry order**.

3.1.4 Definitions in Code

Code 3.1.1: The three depth-first traversals – each one is three lines; only the position of the visit call changes

```
#include <bits/stdc++.h>
```

```cpp
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

vector<int> out;   // shared output list, filled during the walk

// PREORDER: handle the node, then the children.
void preorder(TreeNode* r) {
    if (!r) return;              // empty tree: write nothing
    out.push_back(r->val);       // visit NOW  (arrival moment)
    preorder(r->left);
    preorder(r->right);
}

// INORDER: left, then node, then right. (Binary trees only.)
void inorder(TreeNode* r) {
    if (!r) return;
    inorder(r->left);
    out.push_back(r->val);       // visit BETWEEN the subtrees
    inorder(r->right);
}

// POSTORDER: children first, node last.
void postorder(TreeNode* r) {
    if (!r) return;
    postorder(r->left);
    postorder(r->right);
    out.push_back(r->val);       // visit AFTER both subtrees (departure)
}
```

Run each function on the tree of Figure 3.0.1 and you get precisely the three lists of Example 3.1.4 – check line by line that the position of the `push_back` call is the *only* difference between them.

Proposition 3.1.5 (Every visit-once DFS touches each node once)

Each of the three recursive traversals visits every node exactly once and makes exactly $2n+1$ calls on an n-node tree (n node-calls plus the $n+1$ null base cases of Theorem 2.1.3). Time is $O(n)$.

Proof

A node is visited when its unique call runs, and the call for node v is made exactly once – by v's parent (the root's call is made by the initial invocation). By Theorem 1.1.5 every node has exactly one parent chain, so no node is missed and none is reached twice. Null calls are the base cases: there are $n+1$ of them by Theorem 2.1.3, giving $2n+1$ calls, each doing $O(1)$ work.

■

3.1.5 Where the Root Sits (Instant Sanity Checks)

Proposition 3.1.6 (Root position in each order)

For any non-empty binary tree: pre(T) *starts* with the root, post(T) *ends* with the root, and in in(T) the root separates the left subtree's nodes (all before it) from the right subtree's nodes (all after it). Moreover each subtree's own list is a contiguous block inside the parent's list.

Proof

All three claims are the first step of the recurrences of Definition 3.1.2: preorder writes v before anything else in T, postorder writes v after everything, and inorder writes all of L, then v, then all of R. Contiguity is immediate from the same formulas: pre(L) appears as one unbroken piece at the front of pre(T) $\setminus \{v\}$, and similarly for the others.

■

Using the checks wrongly

The checks go one direction only. "Both lists start with the same value" does *not* prove two trees are equal – but "the root is not first in this preorder" *does* prove something is wrong. Use Proposition 3.1.6 to catch bugs fast, not to compare trees; the real equality tool is W1.1.1 (Chapter 1).

3.1.6 Which Order for Which Job?

Order = information availability

You visit a node when you know enough to answer about it.

- **Preorder** – the answer about v needs only ancestors and v itself (copy the tree, print structure, root-to-node paths: you hold the path on the way down). This is §2.2 wearing a different hat.
- **Postorder** – the answer about v needs the *completed* subtrees (delete the tree, evaluate expression nodes, subtree sizes/heights, serialise for a parent to consume). This is §2.3 exactly.
- **Inorder** – the answer interleaves left/node/right; on binary *search* trees it produces the values in sorted order (Chapter 5 proves it) – that is almost its only use, and the reason it exists at all.

Preorder vs postorder

Copy a tree	Delete a tree
create the new node first, then copy children into it – arrival moment	free the children first, then the node – you must never delete a node while its children still point into it – departure moment

Why do people care about inorder?

Because of one huge special case: on a binary *search* tree, `inorder` prints the keys in increasing order, so "sorted output" problems are secretly inorder traversals. We take this seriously in Chapter 5; for arbitrary trees inorder is the least useful of the three.

3.1.7 Solved Problems

W3.1.1 Binary Tree Preorder Traversal

Source: LeetCode 144 LC Medium
Techniques: DFS, preorder recursion, vector output

Preorder Traversal

Given the root of a binary tree, return the values in *preorder*: node, left subtree, right subtree.

➤ Understanding the Problem

This is Definition 3.1.2 verbatim, asked as a function. The output for the running tree of Figure 3.0.1 is the list computed in Example 3.1.4: $[1, 2, 4, 5, 3, 6]$.

➤ Key Observations

Observation 1. "Push the node's value, *then* recurse left, *then* right" – the order of two statements in the function body *is* the definition of the traversal.

Observation 2. The expected answer for an empty tree is the empty list, so the base case just returns.

➤ Implementation

Code 3.1.2: Preorder traversal (LeetCode 144) – recursive

```
class Solution {
    void dfs(TreeNode* r, vector<int>& res) {
        if (!r) return;
        res.push_back(r->val);      // node first: 1
        dfs(r->left, res);          // whole left subtree: 2 4 5
        dfs(r->right, res);         // whole right subtree: 3 6
    }
public:
    vector<int> preorderTraversal(TreeNode* root) {
        vector<int> res;
        dfs(root, res);
        return res;
    }
};
```

Trace on the running tree

Call order and pushes: `dfs(1)` pushes 1; `dfs(2)` pushes 2; `dfs(4)` pushes 4 (both children null, returns); `dfs(5)` pushes 5; back up, `dfs(3)` pushes 3, skips its null left child, `dfs(6)` pushes 6. Result `[1,2,4,5,3,6]` – matches Example 3.1.4.

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$ for the call stack ($O(\log n)$ balanced, $O(n)$ skewed) – Proposition 3.1.5. LeetCode's "follow-up: can you do it iteratively?" is §3.2; work through it after reading there.

W3.1.2 Binary Tree Inorder Traversal

Source: LeetCode 94 LC Medium
Techniques: DFS, inorder recursion

Inorder Traversal

Return the values in *inorder*: left subtree, node, right subtree.

➤ Understanding the Problem

One statement order changes: the push moves *between* the two recursive calls. On the running tree the answer is $[4, 2, 5, 1, 3, 6]$ – compute it yourself with Definition 3.1.2 before reading on; the tricky part is node 3 (no left child $\Rightarrow$ 3 is written immediately when reached, then 6).

➤ Key Observations

Observation 1. Inorder is defined only because children are *ordered* (left vs right). On a general tree there is no "between" – which is why LeetCode has 589/590 for n-ary trees but no inorder (see W3.2.1).

Observation 2. On a binary search tree this same three-line function emits sorted order; LeetCode grades 94 on arbitrary trees though, so do not use any ordering assumption.

➤ Implementation

Code 3.1.3: Inorder traversal (LeetCode 94) – recursive

```
class Solution {
    void dfs(TreeNode* r, vector<int>& res) {
        if (!r) return;
        dfs(r->left, res);          // 4 first (from the 2-subtree)
        res.push_back(r->val);      // node between its subtrees
        dfs(r->right, res);         // then right: from 1-subtree, 3 then 6
    }
public:
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> res;
        dfs(root, res);
        return res;
    }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$. The iterative and Morris versions of *this exact problem* are the two classics of §3.2 – revisit it there.

W3.1.3 Binary Tree Postorder Traversal

Source: LeetCode 145 LC Hard
Techniques: DFS, postorder recursion

Postorder Traversal

Return the values in *postorder*: left subtree, right subtree, node.

➤ Understanding the Problem

Answer for the running tree: $[4, 5, 2, 6, 3, 1]$ – every node appears *after* all of its descendants. (LeetCode tags this "Hard" purely because interviewers ask for the recursive-free version; the recursion itself is three lines.)

➤ Key Observations

Observation 1. In postorder, when you visit a node both subtrees are already fully processed – the "children are solved" guarantee of §2.3. Heights, sizes, subtree sums, tree deletion all use exactly this moment.

Observation 2. The last element of any postorder list is the root (Proposition 3.1.6) – a one-second way to check your output.

➤ Implementation

Code 3.1.4: Postorder traversal (LeetCode 145) – recursive

```cpp
class Solution {
    void dfs(TreeNode* r, vector<int>& res) {
        if (!r) return;
        dfs(r->left, res);          // children entirely first ...
        dfs(r->right, res);
        res.push_back(r->val);      // ... node last: 1 is the final push
    }
public:
    vector<int> postorderTraversal(TreeNode* root) {
        vector<int> res;
        dfs(root, res);
        return res;
    }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$. A neat iterative trick exists ("reverse of root-right-left preorder") – we build it in §3.2.

3.1.8 Practice Problems

Q3.1.1 Maximum Depth of N-ary Tree – LeetCode 559 LC Easy

Hint: postorder with a loop over the children vector: 1 + max of children's depths; generalises the Chapter 1 DFS solution directly.

Q3.1.2 Boundary Traversal of Binary Tree – GeeksforGeeks GFG Medium

Hint: left boundary (top-down, no leaves) + leaves (any DFS order) + right boundary (bottom-up, then reverse); watch the corner cases of Proposition 3.1.6.

Q3.1.3 Left View of Binary Tree – Geeksforgeeks GFG Easy

Hint: first node seen at each depth: preorder carrying the depth, and push when depth exceeds the best so far – a top-down (§2.2) shape.

3.2 Iterative Traversals and Morris Threading

In This Section

The recursive traversals of §3.1 hide their bookkeeping in the *call stack*. This section takes that stack into your own hands: you will rebuild preorder, inorder and postorder with an explicit `std::stack`, following the stack contents step by step on the running tree – exactly the trace an interviewer will ask you to show. Then we go one better: **Morris traversal** visits the tree in $O(1)$ *extra* memory by temporarily borrowing the $n+1$ dead null pointers proved to exist in Theorem 2.1.3.

Prerequisites. §3.1 (the three orders and their recursive code) and Theorem 2.1.3 (the $n+1$ null links we are about to recycle).

After this section you can

- write iterative preorder, inorder and postorder from memory, with a trace;
- explain precisely what the call stack was doing that your stack now does;
- run Morris inorder by hand on a small tree, creating and removing each thread;
- prove Morris is $O(n)$ time despite its inner `while` loop.

3.2.1 Why Replace Recursion at All?

1. **Crash safety.** Recursion costs one stack frame per level ($O(h)$ memory, §2.1); a 10^5-node chain overflows the default frame budget on many judges. An explicit stack lives in the heap and survives.
2. **Interview follow-ups.** "Now do it without recursion" is a standard second question; "now in $O(1)$ space" (Morris) is the third.
3. **Control.** Iterative versions can stop mid-walk (find the k-th element, timeouts), which a recursive walk cannot interrupt cleanly.

Definition 3.2.1 (The explicit-stack idea)

Every function call waits for its children and then resumes. An explicit stack stores exactly that *pending work*: each saved record says "this node still has parts of its walk left to do". Pushing, popping and resuming by hand reproduces the recursion step for step – the only design choice per order is **when the value is written** relative to pushing children.

We keep the stack readable as a list with the **top on the left**, e.g. `S = [2, 3]` means 2 is popped next.

3.2.2 Iterative Preorder: Pop, Write, Push Right, Push Left

The rule that makes preorder work iteratively: when you pop a node, write it immediately, then push its children *right first, then left* – so the stack hands you the left child next, exactly like the recursion would.

Code 3.2.1: Iterative preorder – pop/write/push; children go on *right-then-left* so left pops first

```cpp
vector<int> preorderIter(TreeNode* root) {
    vector<int> res;
    if (!root) return res;
    stack<TreeNode*> st;
    st.push(root);
    while (!st.empty()) {
        TreeNode* v = st.top(); st.pop();
        res.push_back(v->val);                    // write on POP = arrival
        if (v->right) st.push(v->right);          // push right FIRST ...
        if (v->left)  st.push(v->left);           // ... then left  (left pops first)
    }
    return res;
}
```

Iterative preorder on the running tree (top of stack on the left)

Step	Pop & write	Push	Stack after	Output
0	–	push root 1	[1]	
1	1	right 3, then left 2	[2, 3]	1
2	2	right 5, then left 4	[4, 5, 3]	1,2
3	4	none (leaf)	[5, 3]	1,2,4
4	5	none	[3]	1,2,4,5
5	3	right 6 (left is null)	[6]	1,2,4,5,3
6	6	none	[]	1,2,4,5,3,6 ✓

Every popped value is written once, and the right-then-left push order is *the whole trick*. Swap the two push lines and you traverse "right subtree first" – a valid but different order; that accident is actually useful in W3.2.2.

3.2.3 Iterative Inorder: Dive Left, Pop, Write, Go Right

Inorder must write a node *after* its whole left subtree – but the pop happens before we have finished the right side. The fix: keep a cursor `cur`; push (dive into) the entire left spine before writing anything, and after writing a popped node, continue from its right child.

Code 3.2.2: Iterative inorder – "go all the way left" is the loop invariant

```cpp
vector<int> inorderIter(TreeNode* root) {
    vector<int> res;
    stack<TreeNode*> st;
    TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }  // dive: push whole left spine
        cur = st.top(); st.pop();
        res.push_back(cur->val);                        // write on POP = turnaround
        cur = cur->right;                               // now do the right subtree
    }
    return res;
}
```

Iterative inorder on the running tree

Step	Action	Stack after	Output
1	dive 1 → 2 → 4: push 1,2,4; `cur=null`	[4, 2, 1]	
2	pop 4, write; `cur=4.right=null`	[2, 1]	4
3	pop 2, write; `cur=5`	[1]	4,2
4	dive 5: push 5; `cur=null`	[5, 1]	4,2
5	pop 5, write; `cur=null`	[1]	4,2,5
6	pop 1, write; `cur=3`	[]	4,2,5,1
7	dive 3: push 3; `cur=null`(no left child!)	[3]	4,2,5,1
8	pop 3, write; `cur=6`	[]	4,2,5,1,3
9	dive+pop 6	[]	4,2,5,1,3,6 ✓

Step 7 is the lesson: node 3 has *no left child*, so the dive loop pushes 3 and stops instantly – 3 is written as soon as it is popped, before its right child 6. That is exactly the inorder rule seen from the stack's point of view.

Pushing nulls

The recursive version *calls* itself on null children; the iterative version must not *push* them. Every push here is guarded (`if (v->right)`, `while (cur)`). A `nullptr` sitting on the stack is a crash waiting to be popped.

3.2.4 Iterative Postorder: Two Stacks

Postorder wants children before parents, and left before right. The cheapest way to get it is to build a "root → right → left" preorder with one stack, pushing every popped node onto a *second* stack; popping the second stack reverses everything into "left → right → root" per level.

Code 3.2.3: Iterative postorder with two stacks – reverse of a mirrored preorder

```
vector<int> postorderIter(TreeNode* root) {
    vector<int> res;
    if (!root) return res;
    stack<TreeNode*> in, outst;            // outst IS the reversal buffer
    in.push(root);
    while (!in.empty()) {
        TreeNode* v = in.top(); in.pop();
        outst.push(v);                     // collect in root-right-left order ...
        if (v->left)  in.push(v->left);    // ... so push LEFT first: right pops next
        if (v->right) in.push(v->right);
    }
    while (!outst.empty()) { res.push_back(outst.top()->val); outst.pop(); }
    return res;
}
```

Why the reversal works

"Mirrored preorder" writes v *before* its children and visits right before left: $1, 3, 6, 2, 5, 4$ on the running tree. Reversing gives $4, 5, 2, 6, 3, 1$ – which is postorder. The identity $\text{post}(T) = \text{reverse}(\text{root-right-left}(T))$ holds for every binary tree: both sides satisfy "left block, right block, node", each block itself reversed.

Two stacks on the running tree

Step	Pop from S1 / push to S2	Push to S1 (L then R)	S2 (top left)
1	1	2, then 3	[1]
2	3	6	[3, 1]
3	6	–	[6, 3, 1]
4	2	4, then 5	[2, 6, 3, 1]
5	5	–	[5, 2, 6, 3, 1]
6	4	–	[4, 5, 2, 6, 3, 1]

Pop S2 left to right: $4, 5, 2, 6, 3, 1$ = postorder ✓ (compare Example 3.1.4).

3.2.5 Morris Traversal: $O(1)$ Extra Memory

Stacks cost $O(h)$ memory. To reach $O(1)$ we need a way to get back from a finished left subtree to its parent without remembering the path – the tree itself must provide the way back. It already has spare parts: the $n + 1$ `null` pointers of Theorem 2.1.3.

Definition 3.2.2 (Thread)

A **thread** is a temporary pointer placed in a *node's null right child* so that it points to the node's **inorder successor** – the node that inorder would visit next. Morris traversal creates each thread just before descending into a left subtree and destroys it on the later visit, so the tree is *exactly restored* when the walk ends.

Definition 3.2.3 (Morris inorder)

Keep a cursor `cur` starting at the root. At each node with a left child, find the rightmost node `pred` of the left subtree (the inorder predecessor):

- if `pred->right == nullptr`: *create* the thread `pred->right = cur`, then descend (`cur = cur->left`);
- if `pred->right == cur`: the left subtree is done – *remove* the thread, *write* `cur`, and move right (`cur = cur->right`).

If there is no left child: write `cur` and move right. When a thread is written over, following `cur->right` rides it back up to the parent – that is the "way back" the $O(1)$ memory needed.

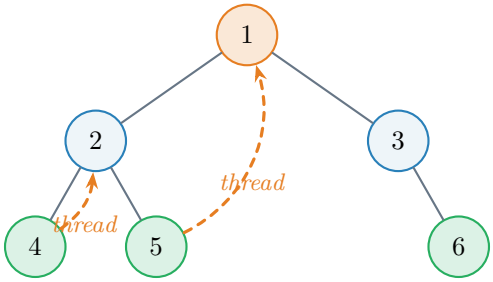


Figure 3.2.1: The running tree mid-Morris-walk, at the moment `cur` = 4: two live threads (dashed orange) `4->right = 2` and `5->right = 1`, written into null pointers. Riding thread $4 \to 2$ is how the walk returns to 2 with no stack. The real `nullptr`s are remembered so every thread can be undone.

Code 3.2.4: Morris inorder – every modification is undone before the walk passes on

```
vector<int> morrisInorder(TreeNode* root) {
    vector<int> res;
```

```
    TreeNode* cur = root;
    while (cur) {
        if (!cur->left) {                          // nothing left to do below
            res.push_back(cur->val);
            cur = cur->right;                      // real edge OR a thread back up
        } else {
            TreeNode* pred = cur->left;
            while (pred->right && pred->right != cur) pred = pred->right;
            if (!pred->right) {                    // first visit of cur:
                pred->right = cur;                 //   make the thread ...
                cur = cur->left;                   //   ... and descend
            } else {                               // came back via the thread:
                pred->right = nullptr;             //   undo the tree,
                res.push_back(cur->val);           //   write the node (turnaround)
                cur = cur->right;
            }
        }
    }
    return res;
}
```

Morris inorder on the running tree, line by line

Step	cur	Action	Output
1	1	left exists; pred=5 (rightmost of 2-subtree); thread 5→1; dive	
2	2	left exists; pred=4; thread 4→2; dive	
3	4	no left: **write 4**; right is thread → ride to 2	4
4	2	pred=4 found thread 4→2: **remove**, **write 2**, go right	4,2
5	5	no left: **write 5**; right is thread 5→1 (still live) → ride	4,2,5
6	1	pred=5 found thread: **remove**, **write 1**, go right	4,2,5,1
7	3	no left: **write 3**; go right	4,2,5,1,3
8	6	no left: **write 6**; right=null	4,2,5,1,3,6 ✓

At the end both threads are removed and every pointer equals its original value – run the trace twice in your head and check that claim node by node.

Theorem 3.2.4 (Morris cost)

Morris inorder runs in $O(n)$ time using $O(1)$ extra memory, and leaves the tree unchanged.

Proof

Space is immediate: only cur, pred and loop locals exist. For time, charge every pointer-following step to an edge or a thread: (a) cur crosses each tree edge at most twice (down, and once as a real right edge); (b) each thread is created once and ridden-then-removed once – threads correspond one-to-one to nodes that have a left child, at most n; (c) each step of an inner pred walk either traverses a real right edge that *later becomes that thread* or leads to an existing thread's end – so right-edge steps of type (c) are charged injectively to distinct thread creations. Total steps $\leq 2(n-1) + n + n = O(n)$.

■

Morris pitfalls

Only *null* right pointers may be threaded – never overwrite a real child. If two nodes share a value and the code searches pred->right != cur wrongly, the walk loops forever; the

stopping condition must compare against `cur` itself, as above. And if you remove the thread *after* writing and moving right, you leave the tree corrupted.

Preorder Morris, and revisiting 94

Swapping the two lines inside the `else` branch (write `cur` at thread-*creation* instead of at thread-*removal*) turns the same machine into Morris *preorder* – the lifecycle of Figure 3.1.1 again decides one line's position. Your homework after this section: solve LeetCode 94 (see W3.1.2) with Code 3.2.4 until the trace of the box above is something you can produce unaided.

3.2.6 Solved Problems

W3.2.1 N-ary Tree Preorder Traversal

Source: LeetCode 589 LC Medium
Techniques: iterative DFS, explicit stack, child order

N-ary Preorder

An n-ary node is `Node{val; vector<Node*> children;}` – children are ordered left to right but there is no fixed count. Return the preorder list (node, then each child's subtree in order).

➤ Understanding the Problem

Same idea as Code 3.2.1, but "push right then left" generalises to "push *all children in reverse order*" so the leftmost child is popped first.

➤ Key Observations

Observation 1. A stack reverses whatever order you push in. To visit children $c_1, \ldots, c_k$ left to right, push $c_k, \ldots, c_1$.

Observation 2. The recursive version needs no trick – `for (child : children) dfs(child);` already goes left to right. The follow-up on LeetCode is exactly the iterative version.

➤ Implementation

Code 3.2.5: N-ary preorder, iterative (LeetCode 589)

```
class Solution {
public:
    vector<int> preorder(Node* root) {
        vector<int> res;
        if (!root) return res;
        stack<Node*> st; st.push(root);
        while (!st.empty()) {
            Node* v = st.top(); st.pop();
            res.push_back(v->val);
            for (int i = (int)v->children.size() - 1; i >= 0; i--)
                st.push(v->children[i]);        // reverse push: child 0 pops first
        }
        return res;
    }
```

```
};
```

➤ Complexity

Complexity

Time $O(n)$ (each node pushed and popped once), memory $O(n)$ worst case for the stack (root with $n-1$ children). Compare the recursive version's $O(h)$.

W3.2.2 N-ary Tree Postorder Traversal

Source: LeetCode 590 LC Medium
Techniques: two-stack trick, order reversal

N-ary Postorder

Return the postorder list: every child subtree first (in order), then the node.

➤ Understanding the Problem

The two-stack trick of Code 3.2.3 generalises untouched: pop from S_1, push onto S_2, and push children onto $S1$ *left to right* so S_1 processes right-to-left; S_2 then reverses into correct postorder.

➤ Implementation

Code 3.2.6: N-ary postorder, two stacks (LeetCode 590)

```
class Solution {
public:
    vector<int> postorder(Node* root) {
        vector<int> res;
        if (!root) return res;
        stack<Node*> s1, s2; s1.push(root);
        while (!s1.empty()) {
            Node* v = s1.top(); s1.pop();
            s2.push(v);
            for (Node* c : v->children) s1.push(c);   // forward push: last child pops
    ↪ first
        }
        while (!s2.empty()) { res.push_back(s2.top()->val); s2.pop(); }
        return res;
    }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(n)$ (both stacks together never hold a node twice; the output vector is not counted). A one-stack variant exists (push nodes to `res`, reverse `res` at the end) – identical trick, less code.

W3.2.3 Find Diameter of N-ary Tree

Source: LeetCode 1522 LC Medium
Techniques: postorder recursion, top-two child heights, global best

Diameter of N-ary Tree

The diameter is the number of *edges* of the longest path between any two nodes (a path may pass through several children of a node). Return its length.

➤ Understanding the Problem

This is §2.3 on many children: the longest path through v joins the *two tallest* child subtrees. You may only use postorder here, because v's answer needs every child's finished height.

➤ Key Observations

Observation 1. At v: take the two largest child heights $h_1 \geq h_2$; candidate $= h_1 + h_2 + 2$ edges through v (the $+2$ is the two edges v to each child), and v returns $h_1 + 1$ upward (Proposition 2.3.2 in Chapter 2 guarantees this terminates).

Observation 2. A node with one child contributes $h_1 + 1$, not a "through" candidate – keep the two heights as $0, 0$ sentinels so a missing second branch costs nothing.

➤ Implementation

Code 3.2.7: Diameter of n-ary tree (LeetCode 1522) – postorder, two best children

```
class Solution {
    int best = 0;
    int height(Node* v) {
        int h1 = 0, h2 = 0;                          // top two child heights
        for (Node* c : v->children) {
            int h = height(c) + 1;                   // edge v->c plus c's height
            if (h > h1) { h2 = h1; h1 = h; }
            else if (h > h2) h2 = h;
        }
        best = max(best, h1 + h2);                   // path through v (edge counts)
        return h1;                                   // one branch upward only
    }
public:
    int diameter(Node* root) { height(root); return best; }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$ recursion stack. With two children the loop degenerates to the Chapter 1 diameter solution (§1.2); this is that code with the "top two" update generalised.

3.2.7 Practice Problems

Q3.2.1 Increasing Order Search Tree – LeetCode 897 LC Medium

Hint: inorder walk collecting values, relinking as one right spine; cleanest with the iterative inorder of Code 3.2.2 so you can splice as you pop.

Q3.2.2 Binary Search Tree Iterator – LeetCode 173 LC Medium

Hint: "inorder on demand": keep only the left spine on a stack, like lines 5-7 of Code 3.2.2; next() pops one and dives right-left again.

3.3 Breadth-First Traversal: Level Order

In This Section

Depth-first orders dive all the way down before moving sideways. The fourth classic order does the opposite: it visits the tree *row by row* – root, then all nodes at depth 1, then all at depth 2, and so on. This is **breadth-first search** (BFS), and its tool is a **queue** (first in, first out) instead of a stack. Level order is the most tested traversal family on LeetCode, because almost every "per level" problem – zigzag, right side view, level averages – is the same five-line template with one different line inside.

Prerequisites. Figure 3.0.1 (our running tree) and the notion of *depth* from §1.2. Stack-vs-recursion from §3.2 helps but is not required.

After this section you can

- run level order by hand on any tree, showing the queue at every step;
- split the flat walk into one list per level with the `sz = q.size()` trick;
- solve bottom-up and zigzag variants by changing *one line* of the template;
- explain why BFS discovers nodes in non-decreasing depth (shortest paths).

3.3.1 The Queue Idea

The office announcement

A manager tells the news to her two direct reports; then *each of them* tells *their* reports, in the order they heard it. Nobody tells anyone twice, and everyone hears it at their exact distance from the top. That rumour spreading is BFS: the queue is the "waiting to tell their own reports" list.

Definition 3.3.1 (Level order / breadth-first traversal)

Start by enqueuing the root. Repeat: dequeue the front node, *visit* it, then enqueue its existing children left to right. The walk visits nodes in non-decreasing depth, and within one depth, left to right. A **queue** is FIFO: elements leave in the order they entered – the exact opposite of a stack, and that single difference turns DFS into BFS.

Stack vs queue

Stack (DFS)	Queue (BFS)
last in, first out – re-expands the most recently discovered node: dives deep	first in, first out – re-expands the longest-waiting node: spreads wide

3.3.2 The Template, and the Level-Split Trick

The plain walk produces the flat list $1, 2, 3, 4, 5, 6$ on the running tree. Useful as-is, but LeetCode almost always wants *one vector per level*. The trick: at the top of each outer loop the queue holds *exactly one whole level, and nothing else*; freeze its size first, then do exactly that many dequeues.

Code 3.3.1: Level-order template: the frozen `sz` is where one level ends and the next begins

```cpp
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = (int)q.size();              // FREEZE: nodes waiting = this level
        vector<int> level;
        for (int i = 0; i < sz; i++) {       // consume exactly this level
            TreeNode* v = q.front(); q.pop();
            level.push_back(v->val);
            if (v->left)  q.push(v->left);   // children wait for the NEXT round
            if (v->right) q.push(v->right);
        }
        res.push_back(level);
    }
    return res;
}
```

Level order on the running tree, queue frozen per round

Queue written front-first. Round 1 starts with the queue holding only the root, so sz = 1; during the round we *add* children but the frozen sz never changes – that is the whole trick.

Round	Queue at freeze	sz	Emitted level
1	[1]	1	[1]
2	[2, 3]	2	[2, 3]
3	[4, 5, 6]	3	[4, 5, 6] ✓
4	[]	-	*done*

Detail of round 2: dequeue 2 (emit, enqueue $4, 5$), dequeue 3 (emit, enqueue 6 – its missing *left* child is simply never pushed). Queue at end of round: $[4, 5, 6]$ – the next level, in left-to-right order, assembled automatically.

Why the queue always holds exactly one level

Induction: suppose at freeze time the queue is level d in order. Each dequeue emits one level-d node and appends its children – all level $d + 1$, left before right – after anything already queued. So after exactly sz dequeues nothing of level d remains and the queue is precisely level $d + 1$ in order. One line of Code 3.3.1 ("push children") is doing all the ordering work.

You have met this before

LeetCode 102 (Binary Tree Level Order Traversal) and 199 (Binary Tree Right Side View) were practice lists in Chapter 1 – go back and redo them now with Code 3.3.1; they are two-line diffs from here. 102 is the template verbatim; 199 keeps only `level.back()` per round.

3.3.3 BFS Finds Shortest Distances

Proposition 3.3.2 (BFS orders by depth)

In the level-order walk, a node v with $\text{depth}(v) = d$ is visited for the first time in round $d+1$ (root in round 1), and no node of depth $> d$ is visited before it.

Proof

By the invariant of the insight box above, round r emits exactly the nodes of depth $r-1$ – and by Theorem 1.1.5 every node appears in exactly one depth class, so it is emitted in exactly one round. Nothing deeper can appear earlier: a node enters the queue only when its (unique) parent was dequeued, one round before.

■

Where this is used. Chapter 8 and graph BFS – On an unweighted *graph* the same argument gives true shortest paths (first visit = minimum edges); in a tree there is only one path anyway, so BFS is simply "by depth" – which is still exactly what "minimum depth" style problems want.

3.3.4 Per-Level Aggregates

Replacing `level.push_back` by a running number per round gives every "aggregate by level" variant in one edit.

Example 3.3.3 (Sums and averages per level of the running tree)

Levels from the trace above: [1], [2, 3], [4, 5, 6]. Sums: 1, 5, 15. Averages: 1.0, 2.5, 5.0 – LeetCode 637 verbatim. Max per level: 1, 3, 6 – LeetCode 515. Deepest-level sum: 15 – LeetCode 1302, which the Chapter 2 practice list also set *top-down with DFS*; same answer, two directions.

3.3.5 Solved Problems

W3.3.1 Binary Tree Level Order Traversal II

Source: LeetCode 107 LC Easy
Techniques: BFS template, output reversed

Level Order II (bottom-up)

Return level-order lists, but starting from the *deepest* level up to the root.

➤ Understanding the Problem

Exactly Code 3.3.1 with the output read backwards. The running tree wants `[[4,5,6], [2,3], [1]]`.

➤ Key Observations

Observation 1. Either call `reverse` once on `res` at the end, or insert every `level` at the front (`res.begin()`) – the single final reverse is cheaper and cleaner.

Observation 2. Do *not* reverse inside the walk by enqueueing right-to-left: that changes left/right order *within* each level, which the problem does not ask for.

➤ Implementation

Code 3.3.2: Bottom-up level order (LeetCode 107) – template + one reverse

```cpp
class Solution {
public:
    vector<vector<int>> levelOrderBottom(TreeNode* root) {
        vector<vector<int>> res;
        if (!root) return res;
        queue<TreeNode*> q; q.push(root);
        while (!q.empty()) {
            int sz = (int)q.size();
            vector<int> level;
            for (int i = 0; i < sz; i++) {
                TreeNode* v = q.front(); q.pop();
                level.push_back(v->val);
                if (v->left)  q.push(v->left);
                if (v->right) q.push(v->right);
            }
            res.push_back(level);
        }
        reverse(res.begin(), res.end());      // the only new line vs \coderef{c03-bfs
    ↪ }
        return res;
    }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(w)$ for the queue, where w = maximum level width: the queue never holds more than the rest of the current level plus the next one. Contrast the two memory profiles: DFS costs $O(h)$ (tall on skewed trees, tiny on balanced) and BFS costs $O(w)$ (tall = wide on balanced trees, tiny on skewed chains) – neither dominates, which is why both traversals exist.

W3.3.2 Binary Tree Zigzag Level Order Traversal

Source: LeetCode 103 LC Medium
Techniques: BFS template, reverse alternate levels

Zigzag Level Order

Level 0 left to right, level 1 right to left, level 2 left to right, alternating – return the lists.

➤ Understanding the Problem

The walk itself does not change at all – only the *reporting* of odd levels. Running tree: levels [1], [2, 3], [4, 5, 6] become [[1], [3, 2], [4, 5, 6]].

➤ Key Observations

Observation 1. Keep a round counter (or test `res.size() % 2`): after building `level` normally, `if (odd) reverse(level.begin(), level.end());`. One line.

Observation 2. A deque variant (push front on odd rounds) saves the reversal but is error-prone; the reverse version is what to write in an interview *and* on LeetCode.

➤ Implementation

Code 3.3.3: Zigzag level order (LeetCode 103) – template + conditional reverse

```cpp
class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        vector<vector<int>> res;
        if (!root) return res;
        queue<TreeNode*> q; q.push(root);
        bool leftToRight = true;
        while (!q.empty()) {
            int sz = (int)q.size();
            vector<int> level(sz);
            for (int i = 0; i < sz; i++) {
                TreeNode* v = q.front(); q.pop();
                int pos = leftToRight ? i : sz - 1 - i;   // the whole zigzag logic
                level[pos] = v->val;
                if (v->left)  q.push(v->left);
                if (v->right) q.push(v->right);
            }
            res.push_back(level);
            leftToRight = !leftToRight;
        }
        return res;
    }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(w)$. Writing into `level[pos]` instead of building then reversing avoids a per-level `reverse` call – same $O(w)$ total either way, because a level is reversed at most once.

3.3.6 Practice Problems

Q3.3.1 Average of Levels in Binary Tree – LeetCode 637 LC Easy

Hint: template with a `double sum` and counter per round instead of a vector; divide at freeze-size `sz`.

Q3.3.2 N-ary Tree Level Order Traversal – LeetCode 429 LC Medium

Hint: identical template; the inner `if (left/right)` pair becomes `for (c : children) q.push(c);`.

Q3.3.3 Find Largest Value in Each Tree Row – LeetCode 515 LC Medium

Hint: template with `mx = max(mx, v->val)` per round; start `mx` at `INT_MIN`, not 0 – values can be negative.

Q3.3.4 Populating Next Right Pointers in Each Node – LeetCode 116 LC Medium

Hint: level order where the work is a pointer, not a value: inside the `sz`-loop link `prev = q->previous node of this round; prev->next = v`.

Q3.3.5 Reverse Odd Levels of Binary Tree – LeetCode 2641 LC Medium

Hint 1: level BFS with the round number: on odd rounds, collect the level's node POINTERS, then swap their `val`s pairwise.

Hint 2: swapping values (not pointers) makes the tree surgery trivial – one vector of `TreeNode*`, two-pointer swap inside it.

3.4 Rebuilding and Serialising Trees

In This Section

Now we run traversal backwards: given the *lists* this chapter produces, can you recover the *tree*? This is the question behind LeetCode 105/106/889 and behind every "store a tree in a file / send it over a network" problem (serialise/deserialise, LC 297). The whole theory rests on one observation you already proved – Proposition 3.1.6 – used with a bit of care about which pairs of orders carry enough information.

Prerequisites. §3.1, especially Proposition 3.1.6 and the lists of Example 3.1.4.

After this section you can

- rebuild a tree from preorder + inorder with a full hand trace;
- prove why one inorder-free pair (pre + post) is *not* enough;
- serialise and deserialise a tree with null markers, by hand and in code;
- number a DFS walk's entry/exit times (tin / tout) and read subtree facts off them.

3.4.1 What Each Order Does and Does Not Tell You

Recall Proposition 3.1.6: preorder starts at the root, postorder ends at it, and *inorder splits* the remaining nodes into "all of the left" and "all of the right". That split property is the missing coordinate:

- **pre** alone: root first, then two unknown-length blocks glued together – you cannot see where the left block ends;
- **post** alone: same problem in mirror image;
- **in** alone: left block, root, right block – but you don't know which node is the root;
- **pre + in**: *pre hands you the root; in splits around it.* Every reconstruction step is just these two facts. (Same for post + in.)

Theorem 3.4.1 (Preorder + inorder determines the tree)

Let all node values be distinct. No two different binary trees have the same preorder list *and* the same inorder list. Moreover the tree is reconstructible from the two lists by the recursive rule: *the first preorder value is the root; the position of that value inside the inorder list splits inorder into left-subtree nodes and right-subtree nodes; the same-sized preorder chunks carry the subproblems.*

Proof

Constructibility is induction on n: the rule identifies the root v, the size $s = |L|$ of the left subtree (from the inorder split), the preorder prefix pre$[1..s]$ of the left subtree and the rest for the right. Both chunks are smaller trees with distinct values, so by induction each is

uniquely rebuildable – and v with its two rebuilt subtrees is one tree. Uniqueness: *any* tree with these lists must have root pre[0] (first visited in preorder) and its inorder forces every node before v into the left subtree; restricting the lists to each side repeats the argument. Two distinct trees would need distinct left or right subtrees, contradicting induction. ■

Two orders *without* inorder cannot split

Preorder keeps every subtree contiguous but never tells you *where the boundary* is; without the inorder split the shape is ambiguous whenever a node has exactly one child. The minimal example:

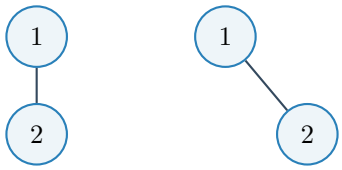


Figure 3.4.1: Two *different* trees – root 1 with a left child 2 versus with a right child 2 – whose preorder is $1, 2$ and whose postorder is $2, 1$ in both cases. Pre + post agree but the trees differ: exactly one-child nodes are where the ambiguity lives. This is why LeetCode 889 (construct from pre + post) asks for *any* valid tree.

3.4.2 Rebuilding by Hand

Rebuild the running tree from pre = [1,2,4,5,3,6

, in = [4,2,5,1,3,6]] Write each region as "pre={..} | in={..}". Every line applies exactly the two facts of Theorem 3.4.1: take pre's first value, find it in in, split.

1	root = pre[0] = 1; in: 1 at index 3	left = $\{4, 2, 5\}$, right = $\{3, 6\}$
2	left child: pre next = $[2, 4, 5]$, in = $[4, 2, 5]$	root 2; in split $\{4\} \mid \{5\}$
3	2's left: pre = $[4]$, in = $[4]$	leaf 4
4	2's right: pre = $[5]$, in = $[5]$	leaf 5
5	right child: pre = $[3, 6]$, in = $[3, 6]$	root 3; in split $\varnothing \mid \{6\}$
6	3's left = $\varnothing$; right: pre = $[6]$	leaf 6 as **right** child

Line 6 rebuilds precisely the odd shape of Figure 3.0.1 – 3 with *no* left child – because the inorder region $[3, 6]$ puts nothing before 3. An empty region means an empty (`null`) child; that base case is half the algorithm.

The only expensive step per node is "find the root inside in". Preprocess positions once into a hash map (`value` → `index in in`) and every node costs $O(1)$ – without the map you get $O(n^2)$ on skewed trees, the standard TLE of first attempts.

Code 3.4.1: Rebuild skeleton: index-by-hash, split, recurse on four ranges

```
unordered_map<int,int> pos;   // pos[value] = index inside inorder

TreeNode* build(vector<int>& pre, int pl, int pr,      // preorder range
                vector<int>& in,  int il, int ir) {    // inorder range
    if (pl > pr) return nullptr;                   // empty region = nil child
    TreeNode* r = new TreeNode(pre[pl]);           // pre[pl] is the root here
    int k = pos[r->val];                           // root inside inorder
    int leftSize = k - il;                         // everything before k is left
    r->left  = build(pre, pl+1,          pl+leftSize, in, il,    k-1);
    r->right = build(pre, pl+leftSize+1, pr,          in, k+1,   ir);
```

```
    return r;
}
```

3.4.3 Solved Problems

W3.4.1 Construct Binary Tree from Preorder and Inorder Traversal

Source: LeetCode 105 LC Medium
Techniques: divide and conquer, hash index, range arithmetic

Build from Pre + In

Given two integer arrays `preorder` and `inorder` (distinct values, produced by one binary tree), rebuild and return that tree.

➤ Understanding the Problem

Theorem 3.4.1 says the answer exists and is unique; Code 3.4.1 is the constructive proof. Running example: the trace above rebuilds Figure 3.0.1 itself.

➤ Key Observations

Observation 1. You never need to *slice* the arrays – carry four indices; the preorder boundary of the left chunk is derived as `pl + leftSize`, never read from inorder directly.

Observation 2. Distinct values are essential: the hash map `value` → `index` is meaningless with duplicates, which is why the problem guarantees them.

➤ Implementation

Code 3.4.2: Construct tree from preorder + inorder (LeetCode 105)

```
class Solution {
    unordered_map<int,int> pos;
    TreeNode* build(vector<int>& pre, int pl, int pr, vector<int>& in, int il, int ir)
    ↪ {
        if (pl > pr) return nullptr;
        TreeNode* r = new TreeNode(pre[pl]);
        int k = pos[r->val], ls = k - il;
        r->left  = build(pre, pl+1, pl+ls, in, il, k-1);
        r->right = build(pre, pl+ls+1, pr, in, k+1, ir);
        return r;
    }
public:
    TreeNode* buildTree(vector<int>& pre, vector<int>& in) {
        for (int i = 0; i < (int)in.size(); i++) pos[in[i]] = i;
        return build(pre, 0, (int)pre.size()-1, in, 0, (int)in.size()-1);
    }
};
```

➤ Complexity

Complexity

Time $O(n)$: one hash lookup and $O(1)$ work per node. Memory $O(n)$ for the map plus $O(h)$ recursion. Drop the map and re-scan inorder each call: $O(n^2)$ worst case (a chain), which LeetCode's larger tests punish.

W3.4.2 Construct Binary Tree from Inorder and Postorder Traversal

Source: LeetCode 106 LC Medium
Techniques: divide and conquer, mirror of LC 105

Build from In + Post

Rebuild the tree from `inorder` and `postorder` arrays (distinct values).

➤ Understanding the Problem

Mirror image: the *last* postorder value is the root (Proposition 3.1.6), the inorder position still splits, and now the right chunk of postorder is found counting *back*. Same skeleton, three index expressions change.

➤ Key Observations

Observation 1. Root = `post[pr]`; left size = `k - il`; the postorder ranges are $[\text{pl},\ \text{pl}+\text{ls}-1]$ for the left subtree and $[\text{pl}+\text{ls},\ \text{pr}-1]$ for the right – the final -1 removes the root that sits at the end.

Observation 2. Off-by-one here is guaranteed unless you trace one small case fully – do the trace of the subsection above with 1 moved from the front of pre to the back of post.

➤ Implementation

Code 3.4.3: Construct tree from inorder + postorder (LeetCode 106) – compare ranges with Code 3.4.2

```
class Solution {
    unordered_map<int,int> pos;
    TreeNode* build(vector<int>& post, int pl, int pr, vector<int>& in, int il, int ir
    ↪ ) {
        if (pl > pr) return nullptr;
        TreeNode* r = new TreeNode(post[pr]);        // LAST post value = root
        int k = pos[r->val], ls = k - il;
        r->left  = build(post, pl,       pl+ls-1, in, il, k-1);
        r->right = build(post, pl+ls,    pr-1,    in, k+1, ir);
        return r;
    }
public:
    TreeNode* buildTree(vector<int>& in, vector<int>& post) {
        for (int i = 0; i < (int)in.size(); i++) pos[in[i]] = i;
        return build(post, 0, (int)post.size()-1, in, 0, (int)in.size()-1);
    }
};
```

➤ Complexity

Complexity

$O(n)$ time and memory exactly as in W3.4.1. Pre + in and post + in both work; *any pair without in* fails by the ambiguity of Figure 3.4.1.

3.4.4 Serialising and Deserialising

Definition 3.4.2 (Serialise / deserialise)

A **serialisation** of a tree is a flat encoding (string or array) from which a **deserialisation** routine can rebuild *exactly* the original tree – shape and values. Formally: deserialize(serialize(T)) = T for every T.

Why one order plus markers is enough

Two orders were needed for reconstruction only because a *bare* preorder hides where subtrees end. Put an explicit marker (#) into the preorder every time the walk hits a `null`, and the boundary information is in the string itself: reading it left to right, "value → make a node; # → stop this branch" is unambiguous. This is the one place the $n + 1$ nulls of Theorem 2.1.3 become *data*.

Encode the running tree (preorder with #)

Walk preorder; write the value on arrival, write # at every null child:

1 2 4 # # 5 # # 3 # 6 # #

Reading back the same string: 1; its left 2; 2's left 4 closed by two #; 2's right 5 closed by two #; 1's right 3; # says 3 has no left child; 6 then two #. Rebuilds Figure 3.0.1 uniquely – 6 values +7 markers = 13 tokens = $2n + 1$, the same count as Proposition 3.1.5 (nice surprise: serialising and traversing are the same walk seen differently).

Code 3.4.4: Serialise: preorder with null markers, appended to one stream

```
void ser(TreeNode* r, ostringstream& out) {
    if (!r) { out << "# "; return; }        // the n+1 nulls, made explicit
    out << r->val << " ";
    ser(r->left, out);
    ser(r->right, out);
}
string serialize(TreeNode* root) {
    ostringstream out; ser(root, out); return out.str();
}
```

Code 3.4.5: Deserialise: read tokens left to right; the recursion shape IS the string shape

```
TreeNode* des(istringstream& in) {
    string tok; in >> tok;
    if (tok == "#") return nullptr;                // consumed null: branch ends
    TreeNode* r = new TreeNode(stoi(tok));
    r->left  = des(in);                             // same preorder order = same
    ↪ tokens
    r->right = des(in);
    return r;
```

```
}
TreeNode* deserialize(string data) { istringstream in(data); return des(in); }
```

W3.4.3 Serialize and Deserialize Binary Tree

Source: LeetCode 297 LC Hard
Techniques: preorder with null markers, stream I/O

Serialize and Deserialize

Design an encoding of a binary tree into a string and a decoding back. The only contract: `decode(encode(T))` must equal T for every tree – any format you like.

➤ Understanding the Problem

"Any format" is the trap: BFS with holes, or pre + in pairs, also work, but preorder with markers is shortest ($2n + 1$ tokens), needs one pass each way, and matches code you have *already written above* – Code 3.4.4 and Code 3.4.5 are the complete solution.

➤ Key Observations

Observation 1. Deserialisation must read *in exactly the order serialisation wrote.* Preorder writes node-left-right, so the decoder takes one token for the node, then recursively one region for the left, one for the right – the `istringstream` cursor advances itself.

Observation 2. Without the `#` markers the round trip *cannot* work: two different trees share a bare preorder (that exact fact is Figure 3.4.1). Markers are the missing second order, smuggled into the first one.

➤ Implementation

Code 3.4.6: LC 297 in full – the two helpers glued into the class

```
class Codec {
public:
    string serialize(TreeNode* root) {
        ostringstream out; ser(root, out); return out.str();
    }
    TreeNode* deserialize(string data) {
        istringstream in(data); return des(in);
    }
private:
    void ser(TreeNode* r, ostringstream& out) {
        if (!r) { out << "# "; return; }
        out << r->val << " "; ser(r->left, out); ser(r->right, out);
    }
    TreeNode* des(istringstream& in) {
        string tok; in >> tok;
        if (tok == "#") return nullptr;
        TreeNode* r = new TreeNode(stoi(tok));
        r->left = des(in); r->right = des(in);
        return r;
    }
};
```

➤ Complexity

Complexity

Time $O(n)$ each way; memory $O(n)$ for the string, $O(h)$ stack each pass. Values may contain any integer – tokenise with spaces (or commas) and never split on a character that can occur inside a value.

➤ Pitfalls and Variants

Serialising with inorder, or without markers

A bare inorder of a BST is just sorted values (no shape at all); a bare preorder of a BST *does* encode the shape (see LeetCode 449 in the practice list) – but only for BSTs, because the split point can be recovered from value ranges instead of a second array. General trees need markers or two orders; there is no way around Figure 3.4.1.

3.4.5 The Euler Tour: tin and tout (Teaser)

Definition 3.4.3 (Entry and exit times)

Number the moments of a single depth-first walk: $\text{tin}(v)$ is the count when the walk first *enters* v, and $\text{tout}(v)$ is the count when it *exits* v after finishing the whole subtree of v. Every node produces exactly two events – one entry, one exit – so the walk numbers the n nodes' events 1 through $2n$.

One DFS walk, twelve events, on the running tree

time	event	time	event
1	tin(1)	7	tout(2)
2	tin(2)	8	tin(3)
3	tin(4)	9	tin(6)
4	tout(4)	10	tout(6)
5	tin(5)	11	tout(3)
6	tout(5)	12	tout(1)

Proposition 3.4.4 (Subtrees are intervals)

For any nodes u, v: v is in the subtree of u if and only if $\text{tin}(u) \le \text{tin}(v) \le \text{tout}(v) \le \text{tout}(u)$. And $\text{size}(v) = (\text{tout}(v) - \text{tin}(v) + 1)/2$.

Proof

From the moment the walk enters u until the moment it exits u, it is inside u and never outside: it descends through each child edge, finishes that whole subtree, and only then moves on – so exactly the nodes of subtree(u) are entered (and exited) inside the span $[\text{tin}(u), \text{tout}(u)]$, giving the iff. For the count: a subtree of size s produces $2s$ events, so its span is $2s - 1$ time units plus the endpoint: $\text{tout}(v) - \text{tin}(v) + 1 = 2\,\text{size}(v)$.

■

Check on node 2: $[2, 7]$, size $(7 - 2 + 1)/2 = 3$ – nodes $\{2, 4, 5\}$ ✓. Node 1: $[1, 12]$, size 6 ✓. This one trick turns trees into intervals:

Where this is used. Chapters 11 and 16 – ancestor tests in $O(1)$ and subtree queries – the interval picture above is what makes them fast

3.4.6 Practice Problems

Q3.4.1 Maximum Width of Binary Tree – LeetCode 662 LC Medium

Hint: level order with *virtual indices*: number slots heap-style (children of i are $2i, 2i+1$), width = last index − first index +1 per round; use `long long` or strip leading zeros.

Q3.4.2 Serialize and Deserialize BST – LeetCode 449 LC Medium

Hint: plain preorder works with NO markers on BSTs: decode by recursing with a (min,max) value window – a preview of Chapter 5.

Q3.4.3 Construct Binary Tree from Preorder and Postorder Traversal – LeetCode 889 LC Medium

Hint 1: ambiguous in general – but you only need *some* valid tree: the next preorder value is the root of *a* child subtree; find it in post to get that child's size.

Hint 2: attach it as left if you like; left size = (index in post − post start +1); recurse the remainder as right child.

Chapter 3 Summary

One question organised this chapter: *when do you write the node down?* The tables collect the four answers, the facts that make them useful, and the cost of producing each.

Order	Rule and where the root sits	Defined code
Preorder	node, left, right; root is **first** – copy/serialise/paths down	Code 3.1.1
Inorder	left, node, right; root **splits** the list – BST sorted order	Code 3.1.1
Postorder	left, right, node; root is **last** – delete/evaluate/aggregate	Code 3.1.1
Level	by depth, left to right within a level; queue, not stack	Code 3.3.1

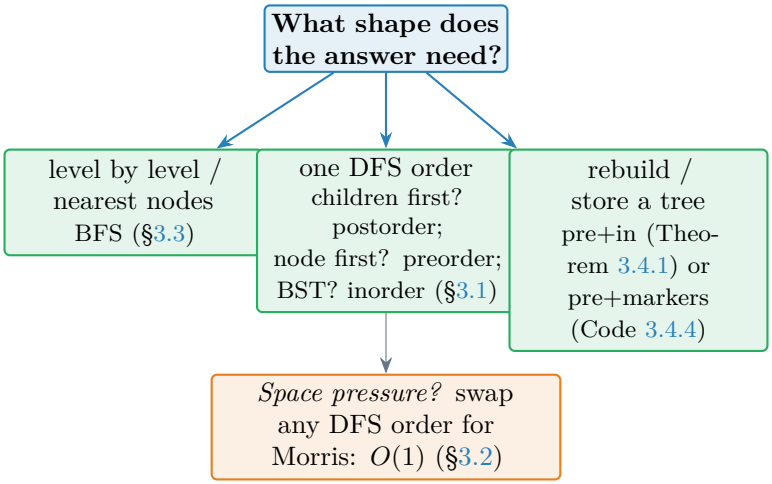
Results of this chapter.

Result	Statement	Where
Visit cost	Every DFS order takes $O(n)$ time, $O(h)$ stack ($2n+1$ calls).	Proposition 3.1.5
Root position	first in pre, last in post, splitter in inorder.	Proposition 3.1.6
Morris	$O(n)$ time, $O(1)$ extra space, tree fully restored.	Theorem 3.2.4
BFS orders depths	round r emits exactly depth $r-1$; first visit is nearest.	Proposition 3.3.2
Pre + in unique	distinct values reconstruct exactly one tree.	Theorem 3.4.1
Markers = shape	preorder + nulls round-trips: $2n+1$ tokens.	Code 3.4.4
Euler intervals	$v \in \text{subtree}(u) \iff \text{tin}(u) \le \text{tin}(v) \le \text{tout}(v) \le \text{tout}(u)$.	Proposition 3.4.4

Algorithms from this chapter.

Algorithm	Time	Memory	Code
Recursive pre/in/post	$O(n)$	$O(h)$	Code 3.1.1
Iterative pre / in / post	$O(n)$	$O(h)$	Code 3.2.1, Code 3.2.2, Code 3.2.3
Morris in (or pre) order	$O(n)$	$O(1)$	Code 3.2.4
Level order with splits	$O(n)$	$O(w)$	Code 3.3.1
Rebuild from pre + in	$O(n)$	$O(n)$	Code 3.4.1
Serialise / deserialise	$O(n)$	$O(n)$	Code 3.4.4, Code 3.4.5

Here h is the tree height and w its widest level – the two memory profiles are independent: skewed trees punish DFS, bushy trees punish BFS.



Choosing a traversal: breadth vs depth first, which writing moment, or run the walk backwards into a tree.

Common Mistakes Checklist

- **Push order wrong in iterative preorder.** Push *right then left* or the walk visits right subtrees first (Code 3.2.1).
- **Freeze too late.** Reading `q.size()` *inside* the `for` loop shuffles level boundaries – snapshot `sz` once per round (Code 3.3.1).
- **Writing on pop in inorder.** The value must be written *after* the pop *and* the left-dive is finished; writing on push gives preorder by accident.
- **Morris forgets to undo.** Remove the thread at the second visit before moving right – leftover threads corrupt the tree for every later algorithm.
- **Serialising without markers.** A bare preorder cannot distinguish the two trees of Figure 3.4.1; include the `#` nulls (or use two orders).
- **Rebuilding with duplicates or without the hash map.** Values must be distinct (Theorem 3.4.1), and scanning inorder per node gives $O(n^2)$ on chains – index the positions first (Code 3.4.1).
- **Zigzag parity off by one.** Level 0 goes left-to-right; test parity with the round counter, not the node depth, and check a two-level tree.
- **Confusing level order with "sorted" order.** Only *inorder on a BST* is sorted (Chapter 5); level order of a BST is generally not (Figure 3.0.1 levels $= [1][2,3][4,5,6]$ happens to be; that is luck).

Quick Reference Card

Chapter 3 at a glance

The four orders. pre = NLR, in = LNR, post = LRN, level = by depth. Root: first / splitter / last / first. Each DFS order is one three-line function – only the `push_back` position moves (Code 3.1.1).

Iterative. Preorder: pop-write, push R then L. Inorder: dive left, pop-write, go right. Postorder: mirrored preorder into a second stack, pop it reversed. Morris: borrow null right pointers as temporary threads back to the inorder successor.

BFS. `sz = q.size()` at the top of each round = one level. Everything "per level" (102/103/107/199/515/637) is this loop with one different line.

Backwards. pre + in (or post + in) rebuild uniquely; pre + post is ambiguous without inorder. One order + null markers serialises: `1 2 4 # # 5 # # 3 # 6 # #`.

Code 3.4.7: Chapter 3 code kit: the two must-memorise walkers side by side

```cpp
#include <bits/stdc++.h>
using namespace std;
struct TreeNode { int val; TreeNode *left, *right;
                  TreeNode(int x=0): val(x), left(nullptr), right(nullptr){} };

// ITERATIVE INORDER (LeetCode 94 follow-up) -- dive, pop, write, right.
vector<int> inorderIter(TreeNode* root) {
    vector<int> res; stack<TreeNode*> st; TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push(cur); cur = cur->left; }   // dive all the way left
        cur = st.top(); st.pop();
        res.push_back(cur->val);                          // turnaround moment
        cur = cur->right;
    }
    return res;
}

// LEVEL-ORDER TEMPLATE -- everything "by level" is this loop.
vector<vector<int>> levels(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = (int)q.size();                           // freeze the level
        vector<int> lev;
        for (int i = 0; i < sz; i++) {
            TreeNode* v = q.front(); q.pop();
            lev.push_back(v->val);
            if (v->left)  q.push(v->left);
            if (v->right) q.push(v->right);
        }
        res.push_back(lev);
    }
    return res;
}
```

Mixed Problem Set

The order is not stated – work out which traversal (and which variant) the task implies.

M3.1 Validate Preorder Sequence in Binary Search Tree – LeetCode 331 LC Medium

Hint: a preorder of a BST obeys a stack of shrinking lower bounds; simulate the walk on the array without building a tree.

M3.2 Print Binary Tree – LeetCode 655 LC Medium

Hint: compute the height once, then place nodes on an $h \times 2^h$ grid with the pre-order recursion writing its own coordinates.

M3.3 Subtree Queries – CSES 1138 CSES

Hint: flatten each subtree to the interval $[\text{tin}, \text{tout}]$ of Proposition 3.4.4 and answer sums on the flat array.

M3.4 Vertical Order Traversal of a Binary Tree – LeetCode 987 LC Hard

Hint 1: assign each node coordinates (col, row): left child $(^{-}1, +1)$, right child $(+1, +1)$; group by column, sort inside a column by row then value.

Hint 2: any walk works (BFS keeps rows increasing naturally); collect (row, value) pairs into one vector per column, then sort each vector – the tie-break rules are the whole problem.

M3.5 Diagonal Traversal of Binary Tree – GeeksforGeeks GFG Medium

Hint 1: same-diagonal nodes are reached by walking right edges freely and queueing the left child; one queue for the whole traversal.

Hint 2: think of the tree drawn with left edges going down-left: read top-right slant lines one by one; push `left` only when the right branch ends.

Related Topics

- Chapter 2 – the recursion directions these walks embody: preorder ≈ top-down, postorder ≈ bottom-up.
- Chapter 4 – arrays and heaps: *storing* the tree so that traversal becomes index arithmetic.
- Chapter 5 – binary search trees: inorder sortedness, the reason inorder exists.
- Chapter 7 – n-ary trees: pre/post carry over (589/590/429), inorder does not.
- Chapter 8 – graphs: BFS/DFS with a `visited` set; trees are the acyclic special case where the parent check suffices.
- Chapter 11, Chapter 16 – LCA and subtree queries: the tin / tout tour teased in §3.4, now load-bearing.

4

Chapter

Representing Trees in Memory

Level: LC Easy–Medium • **Problems:** 29 planned • **Status:** Outline

4.1 Pointers vs Arrays

Outline – 7 problems planned

Concepts to cover

- Linked nodes (`TreeNode*`) and when to use them
- Adjacency list `vector<vector<int>>` for a general tree
- The array / implicit-heap layout for complete binary trees
- Memory, cache locality, and allocation overhead

4.2 Rooting an Unrooted Tree

Outline – 8 problems planned

Concepts to cover

- Turning $n - 1$ undirected edges into parent/child pointers
- The parent-guard DFS and building a rooted tree
- Children lists, parent array, depth array in one pass
- Rerooting: moving the root to a different node

4.3 Euler Tour and Flattening

Outline – 8 problems planned

Concepts to cover

- The DFS entry/exit tour and the interval of a subtree
- Subtree as a contiguous range: the key to Fenwick/segment trees
- Flattening a tree into an array (forward reference to Chapter 13, Chapter 14)
- In-time / out-time ancestor tests
- The tour as the backbone for Mo's algorithm and virtual trees (Chapter 21)

4.4 Encodings and Formats

Outline – 6 problems planned

Concepts to cover

- Level-order array with `null` markers (LeetCode format)
- Parenthesised / Newick-style strings
- Edge lists and Prufer codes (forward reference to Chapter 17)
- Parsing a tree from contest input

Part II

Ordered Trees and Heaps

Search Trees, Heaps, Tries

LC Medium – CF 1600 • 4 chapters • 134 problems

Chapters in this part

5 Binary Search Trees 92

6 Balanced Search Trees 94

7 Heaps and Priority Queues 96

8 Tries 98

5

Chapter

Binary Search Trees

Level: LC Medium – CF 1500 • **Problems:** 32 planned • **Status:** Outline

5.1 The BST Property

Outline – 8 problems planned

Concepts to cover

- Definition: left < node < right, recursively on every subtree
- Inorder traversal yields the keys in sorted order
- Validating a BST (the min/max bound recursion)
- Searching in O(height) and the search path

5.2 Insertion and Deletion

Outline – 9 problems planned

Concepts to cover

- Insert by following the search path to a missing child
- Delete: leaf, one child, two children (inorder successor / predecessor)
- Finding the minimum and maximum of a subtree
- Successor and predecessor queries

5.3 BST Queries

Outline – 8 problems planned

Concepts to cover

- Rank, order statistics, and counting keys in a range
- k-th smallest element
- Lowest common ancestor in a BST (value-guided descent)
- Closest value and floor/ceil of a key

5.4 BSTs in Contests

Outline – 7 problems planned

Concepts to cover

- When a sorted structure turns $O(n^2)$ into $O(n \log n)$
- Using `std::set` / `std::map` as a ready-made BST
- Coordinate compression as a poor-man's BST
- Degeneration into a path and why we need balancing (Chapter 6)

6

Chapter

Balanced Search Trees

Level: CF 1600–1900 • **Problems:** 38 planned • **Status:** Outline

6.1 Why Balance?

Outline – 6 problems planned

Concepts to cover

- Worst case $O(n)$ on a skewed BST; balance restores $O(\log n)$
- Balance factor and the height bound of a balanced tree
- The many flavours: AVL, red–black, treap, scapegoat

6.2 Rotations

Outline – 8 problems planned

Concepts to cover

- Left and right rotation, drawn before and after
- Rotations preserve the inorder sequence
- The four rebalancing cases (LL, RR, LR, RL)
- Rotation as the primitive behind every balanced tree

6.3 AVL Trees

Outline – 7 problems planned

Concepts to cover

- Height invariant and the Fibonacci worst case
- Insert and delete with rebalancing
- Maintaining subtree sizes for order statistics

6.4 Treaps and Implicit Keys

Outline – 9 problems planned

Concepts to cover

- Random priorities give expected $O(\log n)$
- Split and merge as the two treap primitives
- Implicit treap: a balanced tree as a dynamic array

- Range insert, delete, reverse, and lazy values

6.5 Cartesian Trees

Outline – 8 problems planned

Concepts to cover

- A Cartesian tree of an array: heap-ordered by value, inorder by index
- Building it in $O(n)$ with a monotonic stack
- A treap is a Cartesian tree on $(\text{key}, \text{priority})$ – the link to the topic above
- RMQ $\leftrightarrow$ LCA on the Cartesian tree (forward to Chapter 11)
- Largest rectangle in a histogram as a Cartesian-tree problem

7

Chapter

Heaps and Priority Queues

Level: LC Medium – CF 1600 • **Problems:** 32 planned • **Status:** Outline

7.1 The Binary Heap

Outline – 8 problems planned

Concepts to cover

- Complete binary tree stored in an array; children at $2i$, $2i + 1$
- Min-heap and max-heap order property
- Sift-up and sift-down with diagrams of each swap
- Insert and extract-min in $O(\log n)$

7.2 Building and Using a Heap

Outline – 8 problems planned

Concepts to cover

- Floyd's build-heap in $O(n)$ (not $O(n \log n)$) and why
- `std::priority_queue`, `push`, `pop`, `top`
- Custom comparators and `pair` keys
- Heap as a greedy engine

7.3 Heaps as Trees

Outline – 7 problems planned

Concepts to cover

- A heap is a complete binary tree, so its height is $\lfloor \log_2 n \rfloor$
- Binomial and Fibonacci heaps (the idea, not the code)
- Leftist and skew heaps: mergeable heaps
- Why contests almost always use the plain binary heap

7.4 Classic Heap Problems

Outline – 9 problems planned

Concepts to cover

- k-th largest / smallest element
- Merge k sorted lists
- Running median with two heaps
- Top-k frequent elements
- Scheduling with a min-heap of finishing times

8

Chapter

Tries

Level: CF 1400–1900 • **Problems:** 32 planned • **Status:** Outline

8.1 The Trie of Strings

Outline – 8 problems planned

Concepts to cover

- A rooted tree where each edge is a character
- Insert, search, and prefix-count
- Node as an array of 26 children vs a map
- Memory bounds and when to compress

8.2 Applications of the String Trie

Outline – 8 problems planned

Concepts to cover

- Longest common prefix of many strings
- Prefix matching and autocomplete
- Word search and dictionary problems
- Aho–Corasick in one paragraph (multi-pattern search)

8.3 The Binary Trie

Outline – 9 problems planned

Concepts to cover

- Storing integers bit by bit, most significant bit first
- Maximum XOR of two numbers in a set
- Counting pairs with XOR below a threshold
- k-th smallest XOR and persistent tries

8.4 Trie Practice

Outline – 7 problems planned

Concepts to cover

- Implement Trie (prefix tree)
- Maximum XOR subarray via prefix XORs
- Designing the node for speed and memory

Part III

Rooted Tree Algorithms

DFS, Tree DP, Ancestors

CF 1400–2000 • 4 chapters • 133 problems

Chapters in this part

9 Height, Diameter and Centroid 101

10 Tree Dynamic Programming 103

11 Lowest Common Ancestor 105

12 Binary Lifting 107

9

Chapter

Height, Diameter and Centroid

Level: CF 1400–1700 • **Problems:** 32 planned • **Status:** Outline

9.1 Depth and Height on a General Tree

Outline – 8 problems planned

Concepts to cover

- Depth, height and subtree size in one DFS
- The two-pass height algorithm
- Marking every node with its depth and size
- Iterative DFS when the tree is a path

9.2 Tree Diameter

Outline – 9 problems planned

Concepts to cover

- Definition: the longest path between any two nodes
- Two-sweep method: BFS/DFS from any node, then from the farthest
- Proof that two sweeps find a diameter
- Diameter by tree DP (down1 + down2 at each node)
- Diameter of a weighted tree

9.3 Centroid and Centroid of a Tree

Outline – 8 problems planned

Concepts to cover

- A centroid splits the tree into parts of at most $n/2$ nodes
- Finding all centroids (one or two adjacent nodes)
- Proof that a centroid always exists
- Centroid as the "balance point" for divide and conquer

9.4 Radius, Eccentricity and Centre

Outline – 7 problems planned

Concepts to cover

- Eccentricity of a node, radius and centre of a tree
- Relationship between diameter, radius and centroid
- Minimising the maximum distance to any node

10

Chapter

Tree Dynamic Programming

Level: CF 1600–2000 • **Problems:** 34 planned • **Status:** Outline

10.1 DP on Subtrees

Outline – 9 problems planned

Concepts to cover

- The pattern: state per node, combine children in post-order
- Maximum independent set on a tree (house robber III)
- Maximum path sum and diameter as DP
- Subtree sums, counts and aggregates

10.2 DP with Two States

Outline – 8 problems planned

Concepts to cover

- In/out of a set: $dp[v][0]$ and $dp[v][1]$
- Vertex cover and dominating set on trees
- Tree colouring and matching
- Designing the state so children are independent

10.3 Rerooting DP

Outline – 9 problems planned

Concepts to cover

- Compute the answer rooted at one node, then slide the root
- The "remove the child's contribution, add it back" trick
- Sum of distances to all other nodes
- Rerooting in $O(n)$ for many aggregates

10.4 Tree DP Practice

Outline – 8 problems planned

Concepts to cover

- Binary tree cameras
- Longest path with a constraint
- DP over the DFS order / Euler tour
- Combining tree DP with knapsack on children

11

Chapter

Lowest Common Ancestor

Level: CF 1500–1900 • **Problems:** 33 planned • **Status:** Outline

11.1 Ancestors and the LCA

Outline – 8 problems planned

Concepts to cover

- Ancestor, descendant, and the definition of $\mathrm{lca}(u, v)$
- Climbing by depth until the nodes meet: $O(\text{height})$
- LCA on a path, a star, and a balanced tree
- Why LCA answers path questions

11.2 LCA by Euler Tour and RMQ

Outline – 9 problems planned

Concepts to cover

- The Euler tour and first-occurrence array
- Reducing LCA to a range-minimum query on depths
- Sparse table RMQ in $O(1)$ per query after $O(n \log n)$
- ± 1 RMQ for linear preprocessing (the idea)

11.3 LCA by Binary Lifting

Outline – 8 problems planned

Concepts to cover

- The 2^k-th ancestor table (full treatment in Chapter 12)
- Lifting the deeper node, then jumping together
- $O(\log n)$ per query, the contest default

11.4 Distances and Paths via LCA

Outline – 8 problems planned

Concepts to cover

- $\text{dist}(u, v) = \text{depth}(u) + \text{depth}(v) - 2\,\text{depth}(\text{lca}(u, v))$
- Checking whether a node lies on the path $u \to v$
- Aggregates along a path (min/max/xor) with lifting
- LCA on virtual trees and forests

12

Chapter

Binary Lifting

Level: CF 1700–2100 • **Problems:** 34 planned • **Status:** Outline

12.1 Jumping by Powers of Two

Outline – 9 problems planned

Concepts to cover

- The up$[v][k] = 2^k$-th ancestor table
- Building the table in $O(n \log n)$
- Climbing to a given depth in $O(\log n)$
- k-th ancestor queries

12.2 Binary Lifting for LCA

Outline – 8 problems planned

Concepts to cover

- Equalise depths, then jump both nodes together
- The highest bit that keeps the nodes different
- LCA in $O(\log n)$ with a small constant

12.3 Lifting Arbitrary Path Data

Outline – 9 problems planned

Concepts to cover

- Store a value (min, max, sum, xor) per 2^k jump
- Combine on the way up to answer path queries
- Weighted edges and the distance to the LCA
- Jumps on a functional graph (successor lifting)

12.4 Binary Lifting Practice

Outline – 8 problems planned

Concepts to cover

- k-th ancestor of a node (LeetCode)
- Maximum edge on a path
- Binary lifting vs Euler+RMQ: when to use which

Part IV

Range and Query Structures

Segment and Indexed Trees

CF 1600–2300 • 4 chapters • 133 problems

Chapters in this part

13 Segment Trees 110

14 Fenwick Trees 112

15 Lazy Propagation 114

16 Heavy–Light Decomposition 116

13

Chapter

Segment Trees

Level: CF 1600–2100 • **Problems:** 34 planned • **Status:** Outline

13.1 The Segment Tree

Outline – 9 problems planned

Concepts to cover

- A binary tree of intervals; each node stores an aggregate
- Build, point update, range query in $O(\log n)$
- Array layout ($4n$) vs pointer layout
- The merge function and identity element

13.2 Range Queries and Updates

Outline – 9 problems planned

Concepts to cover

- Sum, min, max, gcd over a range
- Point update, range query
- Range update with lazy propagation (full treatment in Chapter 15)
- Querying a segment tree by hand on a small array

13.3 Advanced Segment Trees

Outline – 8 problems planned

Concepts to cover

- Dynamic and sparse segment trees
- Merge sort tree and segment tree of counts
- Segment tree over the Euler tour for subtree queries
- 2D segment trees (the idea)

13.4 Segment Tree Practice

Outline – 8 problems planned

Concepts to cover

- Range sum query - mutable
- Count of elements in a range
- First element greater than x (binary search on the tree)

14

Chapter

Fenwick Trees

Level: CF 1500–2000 • **Problems:** 32 planned • **Status:** Outline

14.1 The Binary Indexed Tree

Outline – 9 problems planned

Concepts to cover

- Prefix sums and the lowbit $i \,\&\, (-i)$
- `update` and `query` in $O(\log n)$
- Why the Fenwick tree is a hidden binary tree over the array
- Building in $O(n)$

14.2 Fenwick Variants

Outline – 8 problems planned

Concepts to cover

- Range update, point query
- Range update, range query with two BITs
- Fenwick for prefix min/max (with care)
- Coordinate compression

14.3 Fenwick on Trees

Outline – 8 problems planned

Concepts to cover

- Euler tour + Fenwick for subtree updates and queries
- Counting inversions and offline queries
- Fenwick vs segment tree: when to choose which

14.4 Fenwick Practice

Outline – 7 problems planned

Concepts to cover

- Range sum query - immutable and mutable
- Count of smaller numbers after self
- Inversion count

15

Chapter

Lazy Propagation

Level: CF 1800–2300 • **Problems:** 34 planned • **Status:** Outline

15.1 Range Updates Done Right

Outline – 9 problems planned

Concepts to cover

- The problem: updating a whole range in $O(\log n)$, not $O(n)$
- A pending value per node, pushed down only when needed
- Build, lazy update, lazy query
- Tracing a range update on a small tree

15.2 Composing Lazy Values

Outline – 9 problems planned

Concepts to cover

- Assign vs add vs multiply and how they combine
- Two lazy tags (multiply then add) in one node
- Order of composition and the identity
- Applying a tag to a node's aggregate and its children

15.3 Lazy on Trees

Outline – 8 problems planned

Concepts to cover

- Euler tour turns a subtree into a range; lazy update the subtree
- Path updates via heavy–light + lazy (forward to Chapter 16)
- Offline vs online updates

15.4 Lazy Practice

Outline – 8 problems planned

Concepts to cover

- Range add, range sum
- Range assign, range max
- Rectangle add and query (2D, the idea)

16

Chapter

Heavy–Light Decomposition

Level: CF 2000–2400 • **Problems:** 33 planned • **Status:** Outline

16.1 Splitting a Tree into Chains

Outline – 9 problems planned

Concepts to cover

- Heavy child (largest subtree) and light edges
- A path crosses at most $O(\log n)$ light edges
- Decomposing the tree into disjoint chains
- The head of each chain

16.2 HLD for Path Queries

Outline – 9 problems planned

Concepts to cover

- Assign a DFS-order position to every node
- A segment tree over the positions
- Path query and update by jumping chain to chain
- Combining with LCA

16.3 HLD Applications

Outline – 8 problems planned

Concepts to cover

- Path max/sum/assign with a lazy segment tree
- Subtree queries as one contiguous range
- Edge values on the deeper endpoint

16.4 HLD Practice

Outline – 7 problems planned

Concepts to cover

- Path queries on a tree (CSES / CF)
- Subtree and path updates together
- Debugging the chain decomposition on a small tree

Part V

Expert Trees

Decompositions, Persistence and Advanced Queries

CF 1800–2800 • 7 chapters • 261 problems

Chapters in this part

17 Union-Find and Minimum Spanning Trees . . . 119

18 Centroid Decomposition . . . 121

19 Persistent Data Structures . . . 123

20 Advanced Tree DP . . . 125

21 Mo's Algorithm and Virtual Trees . . . 127

22 Tree Isomorphism and Hashing . . . 129

23 Link-Cut Trees . . . 131

A Formula Reference . . . 134

B Code Templates . . . 136

C Problem-Solving Strategies . . . 139

List of Solved Problems . . . 141

17

Chapter

Union-Find and Minimum Spanning Trees

Level: CF 1500–2100 • **Problems:** 42 planned • **Status:** Outline

17.1 Disjoint Set Union

Outline – 9 problems planned

Concepts to cover

- A forest of parent pointers; `find` and `union`
- Path compression and union by size/rank
- Almost-$O(1)$ amortised: the inverse Ackermann function
- Detecting a cycle while adding edges

17.2 Minimum Spanning Tree

Outline – 9 problems planned

Concepts to cover

- Spanning tree of a connected graph; $n - 1$ edges
- Kruskal with DSU, and the cut property
- Prim's algorithm with a priority queue
- MST on a tree is the tree itself

17.3 DSU Trees and Offline Tricks

Outline – 8 problems planned

Concepts to cover

- DSU tree (Kruskal reconstruction tree)
- Small-to-large merging on trees
- DSU on tree for subtree queries
- Rollback DSU and divide-and-conquer over time

17.4 Pruefer Sequences and Counting Trees

Outline – 9 problems planned

Concepts to cover

- The Pruefer code: a labeled tree on n vertices $\leftrightarrow$ a sequence of length $n-2$
- Encoding and decoding in $O(n)$ / $O(n \log n)$
- Cayley's formula n^{n-2} as a one-line corollary (home of the bijection; Chapter 1 states the result)
- Kirchhoff's matrix-tree theorem: counting spanning trees of any graph
- Counting trees and forests with degree constraints via multinomials

17.5 Spanning Tree Practice

Outline – 7 problems planned

Concepts to cover

- Second-best MST (the idea)
- Counting spanning trees: Cayley and Kirchhoff (matrix tree theorem)
- Number of connected components after deletions

18

Chapter

Centroid Decomposition

Level: CF 2100–2500 • **Problems:** 33 planned • **Status:** Outline

18.1 Building the Centroid Tree

Outline – 9 problems planned

Concepts to cover

- Find a centroid, remove it, recurse on the parts
- The decomposition has height $O(\log n)$
- Proof that each level halves the component
- Building the centroid tree in $O(n \log n)$

18.2 Counting Paths

Outline – 9 problems planned

Concepts to cover

- Paths through a centroid split into two subtrees
- Counting pairs at distance k
- The "subtract the same-subtree pairs" correction
- Classic CF path-counting problems

18.3 Centroid Decomposition Applications

Outline – 8 problems planned

Concepts to cover

- Nearest marked node with dynamic updates
- Storing distance tables per centroid
- Divide and conquer on trees beyond paths

18.4 Centroid Practice

Outline – 7 problems planned

Concepts to cover

- Count pairs with a given distance
- Closest coloured node queries
- Combining centroid decomposition with a Fenwick tree

19

Chapter

Persistent Data Structures

Level: CF 2000–2500 • **Problems:** 33 planned • **Status:** Outline

19.1 Persistence by Path Copying

Outline – 9 problems planned

Concepts to cover

- Keep every version; copy only the $O(\log n)$ touched nodes
- Persistent array and persistent segment tree
- Memory analysis and node pools
- Fat nodes vs path copying

19.2 Persistent Segment Trees

Outline – 9 problems planned

Concepts to cover

- Point update creating a new root
- Querying any past version
- k-th smallest in a range (the classic)
- Chairman tree and prefix versions

19.3 Persistent Trees on Trees

Outline – 8 problems planned

Concepts to cover

- Persistent segment tree over the root-to-node path
- Counting values on a path with LCA
- Persistent stacks and queues

19.4 Persistence Practice

Outline – 7 problems planned

Concepts to cover

- k-th number in a subarray
- Path queries on a tree with persistent structures
- Rollback and offline versions

20

Chapter

Advanced Tree DP

Level: CF 2200–2800 • **Problems:** 34 planned • **Status:** Outline

20.1 DP on the DFS Order

Outline – 9 problems planned

Concepts to cover

- Knapsack over children with subtree sizes
- The $O(n^2)$ bound for "merge children" DPs
- Ordering states to keep the merge cheap
- Counting connected subgraphs and independent sets

20.2 Rerooting at Scale

Outline – 9 problems planned

Concepts to cover

- Prefix/suffix merges over a node's children
- Answering "rooted at every node" in $O(n)$
- Rerooting with non-invertible operations
- Expected-value rerooting

20.3 DP with Data Structures

Outline – 8 problems planned

Concepts to cover

- Small-to-large merging of maps and multisets
- Convex hull trick on trees
- DSU on tree (sack) for subtree queries

20.4 Capstone Problems

Outline – 8 problems planned

Concepts to cover

- Tree diameter with node weights and constraints
- Counting colourings and labelings modulo a prime
- A full CF round problem solved end to end
- Choosing the right tree tool under time pressure

21

Chapter

Mo's Algorithm and Virtual Trees

Level: CF 1800–2400 • **Problems:** 35 planned • **Status:** Outline

21.1 Mo's Algorithm on Subtrees

Outline – 9 problems planned

Concepts to cover

- Offline queries: sort them, sweep, add/remove in $O(1)$
- Flatten the tree with the Euler tour of Chapter 4 so a subtree is a range
- Block size $\approx \sqrt{n}$ and the $O((n+q)\sqrt{n})$ bound
- Distinct colours / values in a subtree
- Handling the enter-and-exit Euler array (each node appears twice)

21.2 Mo's Algorithm on Paths

Outline – 9 problems planned

Concepts to cover

- Paths are not contiguous in the Euler tour – the fix
- Use $\mathrm{lca}(u, v)$ (Chapter 11) to split a path into two root paths
- The "toggle a node" trick and parity of occurrences
- Ordering queries by blocks with the LCA special case
- Path distinct-count and path XOR/sum queries

21.3 Virtual (Auxiliary) Trees

Outline – 9 problems planned

Concepts to cover

- When only $k \ll n$ nodes matter: compress the tree to those nodes plus their LCAs
- Sort by tin, add pairwise LCAs, rebuild the small tree with a stack
- Size $O(k)$ and why the compressed tree preserves ancestor relations
- Running a DP on the virtual tree instead of the whole tree
- Combining with Chapter 16 (HLD) and Chapter 13 (segment tree)

21.4 Applications and Practice

Outline – 8 problems planned

Concepts to cover

- Counting pairs at distance $\leq k$ offline
- Queries that mix subtree and path restrictions
- When to choose Mo's vs HLD vs centroid decomposition
- Debugging the Euler-toggle invariant on a small tree

22

Chapter

Tree Isomorphism and Hashing

Level: CF 1800–2300 • **Problems:** 41 planned • **Status:** Outline

22.1 Rooted Tree Isomorphism and AHU

Outline – 9 problems planned

Concepts to cover

- Two rooted trees are isomorphic if a bijection preserves the parent–child relation
- The AHU algorithm: canonical labels bottom-up by sorted child labels
- Assigning ids to sorted child-label tuples (map of vectors)
- $O(n \log n)$ with sorting, $O(n)$ with radix-style bucketing
- Handling unrooted trees: try the centre(s) from Chapter 9

22.2 Canonical Forms and the Tree Centre

Outline – 8 problems planned

Concepts to cover

- The centre of a tree (one node or an edge) and why it is the right root
- Canonical form / encoding of a tree as a balanced-parentheses string
- Comparing two trees by their canonical strings
- Rooting at the centre makes isomorphism canonical

22.3 Tree Hashing

Outline – 9 problems planned

Concepts to cover

- A randomized polynomial hash of a subtree: combine child hashes commutatively
- XOR/add of a random value per child, or a sorted-tuple hash
- Collision risk and using 64-bit / double moduli
- Comparing subtrees of one big tree in $O(1)$ after $O(n)$ preprocessing
- Hashing vs AHU: when a hash is enough

22.4 Counting Distinct Subtrees and Shapes

Outline – 8 problems planned

Concepts to cover

- Number of distinct rooted subtree shapes in a tree
- Counting unlabeled trees and the link to Catalan/Cayley (Chapter 1, Chapter 17)
- Distinct-substring-style counting on the tree
- Isomorphism classes of forests

22.5 Isomorphism Practice

Outline – 7 problems planned

Concepts to cover

- Subtree of another tree via hashing
- Find all isomorphic subtrees / repeated structure
- Counting non-isomorphic trees with n nodes (small n tables)

23

Chapter

Link-Cut Trees

Level: CF 2300–2800 • **Problems:** 43 planned • **Status:** Outline

23.1 Dynamic Trees and the Represented Tree

Outline – 8 problems planned

Concepts to cover

- A forest that changes: `link`, `cut`, and path queries online
- Represented tree vs auxiliary (splay) tree
- Why HLD (Chapter 16) is static and link–cut is dynamic
- The operations we want in $O(\log n)$ amortised

23.2 Splay Trees: the Building Block

Outline – 9 problems planned

Concepts to cover

- Splay rotations and the access-by-splaying idea (from Chapter 6)
- Amortised $O(\log n)$ argument (the potential, informally)
- Maintaining aggregates and lazy flags through rotations
- `is_root` in the auxiliary tree and pushing down

23.3 Preferred Paths and Access

Outline – 9 problems planned

Concepts to cover

- Preferred child and preferred path decomposition
- `access(v)`: make the root-to-v path preferred, splay v
- `makeroot(v)`: reroot via a path reverse lazy tag
- How `access` reorganises the auxiliary splay trees

23.4 Link, Cut and Queries

Outline – 9 problems planned

Concepts to cover

- `link(u,v)` and `cut(u,v)` with `makeroot` + `access`
- Path aggregate (sum/min/max/xor) between u and v
- Online connectivity and dynamic LCA
- Subtree aggregates (the harder variant)

23.5 Applications and Practice

Outline – 8 problems planned

Concepts to cover

- Dynamic MST and online bridge-finding (the idea)
- Path updates with lazy propagation on a changing forest
- Link–cut vs Euler-tour trees vs HLD: when to use which
- A full dynamic-connectivity problem solved end to end

Appendices

A

Appendix

Formula Reference

Each entry lists the conditions under which it holds and, where the chapter is written, the place where it is proved. A formula without its conditions is a wrong-answer verdict waiting to happen.

A.1 Tree Basics

Counting nodes, edges, paths (Chapter 1)

Edges of a tree. A tree with n nodes has exactly $n - 1$ edges. (Theorem 1.1.4)

Unique path. Between any two nodes of a tree there is exactly one simple path. (Theorem 1.1.5)

Height recursion. With height(`null`) $= -1$: for a node v, height$(v) = 1 + \max_{c \in \text{children}(v)} \text{height}(c)$, so height(leaf) $= 0$ and height$(T) =$ height(root). (Theorem 1.2.4)

Size recursion. size$(v) = 1 + \sum_{c \in \text{children}(v)} \text{size}(c)$, and size(`null`) $= 0$.

Depth vs height. depth(v) counts edges from the root down to v; height(v) counts edges from v down to the deepest leaf below it. For the root, depth $= 0$ and height$(T) =$ height(root).

Binary tree shapes (Chapter 1)

Perfect binary tree. Height h (edges from root to deepest leaf) gives $2^{h+1} - 1$ nodes and 2^h leaves; level ℓ holds 2^ℓ nodes. (Theorem 1.3.7)

Full (proper) binary tree. Every internal node has exactly two children, and #leaves $=$ #internal $+ 1$. (Theorem 1.3.8)

Height bounds. A binary tree with n nodes satisfies $\lfloor \log_2 n \rfloor \le \text{height} \le n - 1$: minimum for a perfect tree, maximum for a skewed (path) tree. (Corollary 1.3.9)

Array layout. In a 1-indexed heap array, node i has children $2i$ and $2i + 1$ and parent $\lfloor i/2 \rfloor$. (Chapter 4, Chapter 7.)

A.2 Counting Trees

How many trees are there? (Chapter 1)

Binary tree shapes. The number of distinct *shapes* of a binary tree with n nodes is the Catalan number $C_n = \dfrac{1}{n+1}\binom{2n}{n}$, with $C_0 = 1, 1, 2, 5, 14, 42, \ldots$. (Theorem 1.4.3)

Labeled trees (Cayley). The number of distinct trees on n *labeled* vertices is n^{n-2}. (Theorem 1.4.4)

Labeled forests / rooted trees. Rooting a labeled tree at any of its n vertices gives n^{n-1} labeled rooted trees.

A.3 Distances and Ancestors

Paths via LCA (Chapters 9 and 11)

Distance from LCA. For nodes u, v of a rooted tree, $\text{dist}(u, v) = \text{depth}(u) + \text{depth}(v) - 2\,\text{depth}(\text{lca}(u, v))$.

Diameter. $\text{diam}(T) = \max_{u,v} \text{dist}(u, v)$; it equals the largest $\text{depth}(a) + \text{depth}(b)$ over pairs whose LCA is the branching node, computable by two DFS/BFS sweeps or by a tree DP.

On a path. u is an ancestor of v iff $\text{tin}(u) \le \text{tin}(v)$ and $\text{tout}(v) \le \text{tout}(u)$ in the Euler (DFS-order) tour. (Chapter 11.)

Convention used in this book

height and depth are counted in **edges**, so a single node has height 0 and $\text{height}(\texttt{null}) = -1$. Some references count in *nodes* (a single node has height 1); the two differ by exactly 1. Always check which convention a problem uses before you submit.

B

Appendix

Code Templates

These are the reference versions of the routines derived in the chapters. Copy them once, trust them forever, and spend contest time on the idea instead of the boilerplate.

B.1 Node Types

Code B.1.1: Binary tree node and n-ary tree node (Chapter 1, Chapter 2)

```cpp
// Binary tree (LeetCode style).
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

// N-ary tree (LeetCode style): any number of children.
class Node {
public:
    int val;
    vector<Node*> children;
    Node(int x = 0) : val(x) {}
};
```

B.2 General Trees from an Edge List

Most contest trees are given as $n - 1$ undirected edges, not as pointers. Store them in an adjacency list and always pass the parent down so a DFS never walks back.

Code B.2.1: Build adjacency and DFS with a parent guard (Chapter 1)

```cpp
int n;
vector<vector<int>> adj;

void build(const vector<pair<int,int>>& edges) {   // nodes 0..n-1 or 1..n
    adj.assign(n, {});
    for (auto [u, v] : edges) { adj[u].push_back(v); adj[v].push_back(u); }
}

vector<int> depth_, parent_, subtree;
void dfs(int v, int p) {                            // O(n) over the whole tree
    parent_[v] = p;
    subtree[v] = 1;
    for (int to : adj[v]) {
        if (to == p) continue;                      // never go back to the parent
        depth_[to] = depth_[v] + 1;
        dfs(to, v);
```

```
        subtree[v] += subtree[to];
    }
}
```

The parent guard is not optional

On a general tree stored as an adjacency list, a DFS that does not skip the parent revisits it and loops forever. The single line `if (to == p) continue;` is what turns an undirected graph traversal into a rooted-tree traversal.

B.3 Height and Depth of a Binary Tree

The recursive form is the one to memorise (Code 1.2.1); the iterative form avoids the call stack when the tree can be a path of 10^5 nodes.

Code B.3.1: Recursive and iterative height (Chapter 1)

```
int height(TreeNode* r) {                          // height(null) = -1
    if (r == nullptr) return -1;
    return 1 + max(height(r->left), height(r->right));
}

int heightIterative(TreeNode* root) {              // post-order with an explicit
    ↪ stack
    if (!root) return -1;
    stack<pair<TreeNode*, int>> st;                // node, child height seen so far
    unordered_map<TreeNode*, int> h;
    st.push({root, 0});
    int best = 0;
    while (!st.empty()) {
        auto [v, state] = st.top();
        if (!v->left && !v->right) { h[v] = 0; st.pop(); best = max(best, 0); continue
    ↪ ; }
        if (state == 0) { st.top().second = 1; if (v->left) st.push({v->left, 0}); }
        else if (state == 1) { st.top().second = 2; if (v->right) st.push({v->right,
    ↪ 0}); }
        else {
            int hl = v->left ? h[v->left] : -1, hr = v->right ? h[v->right] : -1;
            h[v] = 1 + max(hl, hr); best = max(best, h[v]); st.pop();
        }
    }
    return best;
}
```

B.4 Level-Order Traversal (BFS)

Code B.4.1: Breadth-first traversal, level by level (Chapter 3)

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> levels;
    if (!root) return levels;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();                          // the whole current level
```

```
        vector<int> cur;
        for (int i = 0; i < sz; ++i) {
            TreeNode* v = q.front(); q.pop();
            cur.push_back(v->val);
            if (v->left)  q.push(v->left);
            if (v->right) q.push(v->right);
        }
        levels.push_back(cur);
    }
    return levels;
}
```

B.5 Three Binary-Tree Traversals

Code B.5.1: Preorder, inorder, postorder in one recursive pass (Chapter 3)

```
void preorder (TreeNode* r) { if (!r) return; visit(r); preorder(r->left);  preorder(r
    ↪ ->right); }
void inorder  (TreeNode* r) { if (!r) return; inorder(r->left);  visit(r); inorder(r->
    ↪ right); }
void postorder(TreeNode* r) { if (!r) return; postorder(r->left); postorder(r->right);
    ↪  visit(r); }
```

More templates appear as their chapters are written

Binary search trees, rotations and balancing (Chapters 5–6); heaps and `priority_queue` tricks (Chapter 7); tries (Chapter 8); diameter, centroid and rerooting DP (Chapters 9, 18, 20); LCA and binary lifting (Chapters 11–12); segment trees, Fenwick trees and lazy propagation (Chapters 13–15); heavy–light decomposition (Chapter 16); DSU and MST (Chapter 17); persistent structures (Chapter 19).

C

Appendix

Problem-Solving Strategies

C.1 Recognising a Tree Problem

Problem indicators

It is a tree. Words such as *node, root, parent, child, leaf, subtree, sibling, ancestor, descendant, hierarchy*; "n nodes and $n-1$ edges"; "connected and acyclic"; an org chart, a file system, a family tree, a parse expression.

Binary specifically. Each node has a `left` and a `right`; a BST, a heap, a segment tree; LeetCode `TreeNode*` input.

Subtree queries. "For every node, the sum / max / count over its subtree"; ask for a value that aggregates a whole branch. Root the tree and run one post-order DFS.

Path queries. "Between nodes u and v", "the longest path", "maximum path sum". Think LCA and the identity $\text{dist}(u,v) = \text{depth}(u) + \text{depth}(v) - 2\,\text{depth}(\text{lca}(u,v))$, or heavy–light decomposition.

Rerooting. "Compute the answer for every node as if it were the root". Two DFS passes: gather from children, then push corrected values back down.

C.2 Constraints Suggest the Method

Constraint / wording	Usually means
$n \le 2 \cdot 10^5$, $n-1$ edges	adjacency list, one DFS/BFS, $O(n)$ or $O(n \log n)$
recursion depth can reach n (a path)	write the traversal iteratively, or raise the stack
many queries on one static tree	preprocess once: depths, parents, Euler tour, binary lifting
"subtree of every node"	post-order DFS, or Euler tour + Fenwick/segment tree
"path between two nodes"	LCA ($O(\log n)$ per query after $O(n \log n)$ preprocessing)
values on edges / weights	root the tree, keep the parent to orient each edge
"closest marked node", "distance to nearest"	multi-source BFS or a rerooting DP
counting labeled trees / shapes	Cayley n^{n-2}, Catalan C_n (Chapter 1)
tree is actually a BST in disguise	use the in-order = sorted property (Chapter 5)

C.3 A Five-Step Framework

From statement to accepted

1. Root it. Pick a root (node 1, or the node the problem names). Almost every tree algorithm is a DFS that carries the parent down and combines child answers on the way up.

2. Draw a small tree. Six to ten nodes. Mark the depth, height and size of each. Most tree recurrences become obvious once you can see the subtree of one node.

3. State the recurrence. Write what a node needs from its children (the DP state) and what it returns to its parent. Name the base case: usually `null` or a leaf.

4. Implement with a parent guard. On an adjacency list, skip the parent. On pointers, handle `null` first. Choose `long long` when a subtree sum or a count of shapes can exceed 2^{31}.

5. Check the shapes. The empty tree, a single node, a path (max depth), a star (one root, many leaves), and a perfect tree. Off-by-one in depth vs height shows up on exactly these.

List of Solved Problems

Every fully solved problem in the book, grouped by chapter. The number is a link to the solution.

Chapter 1 Tree Basics and Terminology

Chapter 2 Binary Trees

Chapter 3 Tree Traversals

Index

m-ary tree, 17
(height-)balanced, 18

ancestor, 5

binary tree, 17, 33
bottom-up, 47
breadth-first search, 73

child, 5
complete, 17

degenerate, 17
degree, 5
depth, 10
depth-first entry order, 59
descendant, 5
deserialisation, 82
distance, 5

forest, 5
full, 17

height, 10, 11

inorder, 56, 58
internal, 5

labelled tree, 24
leaf, 5
length, 5
level, 10
level order, 56

Morris traversal, 65

ordered, 24

parent, 5
path, 5
perfect, 17
postorder, 56, 58
preorder, 56, 58
proper, 17

queue, 73

restored, 42
root, 5
rooted tree, 5

serialisation, 82
shape, 24
size, 10, 11
skewed, 17
subtree, 5

thread, 68
top-down, 41
traversal, 58
traverse, 56
tree, 2, 5